



Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia informática

Unidade Curricular de Laboratórios de Informática IV

Ano Lectivo de 2025/2026

Au Au... Sou um cão!

Hugo Cunha, David Alves, Diogo Pais, Gonçalo Sá

Grupo 20

Fevereiro, 2026

LI4

Data de Recepção	
Responsável	
Avaliação	
Observações	

Au Au... Sou um cão!

David Alves (A107324), Diogo Pais (A107368),

Gonçalo Sá (A107376) e Hugo Cunha (A106808)

Maio, 2026

Resumo

O crescimento do setor de cuidados para animais de companhia exige soluções tecnológicas cada vez mais especializadas e integradas. O presente relatório descreve a conceitualização, desenho e implementação da plataforma "AuAuSouUmCao", um sistema de gestão centralizado para um hotel canino, concebido para harmonizar as exigências operacionais de tutores, funcionários (*staff*), veterinários e equipa de gestão.

O trabalho realizado englobou todo o ciclo de vida do desenvolvimento de software. Ao nível do planeamento arquitetural, adotou-se uma abordagem inspirada na Arquitetura Hexagonal, recorrendo de forma estrita aos padrões de desenho *Facade* e *Data Access Object* (DAO). Esta decisão permitiu garantir um forte desacoplamento entre a complexa lógica de negócio (*core*) e as infraestruturas externas. O sistema foi desenvolvido com recurso a uma *stack* tecnológica moderna assente em TypeScript, compreendendo Node.js (Express) no *backend*, Prisma ORM para modelação relacional, e React no *frontend*.

Entre as principais funcionalidades desenvolvidas destacam-se: um motor transacional de reservas com alocação inteligente de *boxes* (condicionado pelo estado de higienização e reatividade do animal), um módulo clínico para acompanhamento e gestão de quarentenas, um "Diário de Bordo" interativo integrado com armazenamento *cloud* seguro (AWS S3), e a implementação de trabalhadores autónomos em segundo plano (*Cron Jobs*) para auditoria de tarefas, alertas críticos de saúde e mitigação de *no-shows*. De realçar, ainda, a adoção de um paradigma moderno de engenharia através da utilização contínua de Modelos de Linguagem de Grande Escala (LLM) como co-pilotos e consultores arquiteturais, acelerando significativamente o ciclo de desenvolvimento e *refactoring*.

Atualmente, o projeto encontra-se num estado funcional sólido e escalável. A resiliência da aplicação foi validada através de uma rigorosa bateria de testes automatizados, assegurando uma cobertura superior a 95% na camada de negócio. Embora algumas integrações secundárias tenham sido catalogadas para iterações futuras, o fluxo transacional primário encontra-se totalmente operacional, entregando um protótipo robusto capaz de digitalizar, proteger e otimizar as operações logísticas do hotel canino.

Para aceder ao site clique neste link: "<https://main.dgvaudjmhakvj.amplifyapp.com/> "

Para aceder ao repositório onde se encontra todo o código raiz

clique neste link: "<https://github.com/DiogoPais/AuAuSouUmCao>"

Área de Aplicação: Engenharia de Software e Desenvolvimento de Sistemas de Informação *Full-Stack*.

Palavras-Chave: Engenharia de Software, Padrões de Desenho (*Facade*, DAO), Aplicações *Web Full-Stack*, TypeScript, Base de Dados, Prisma ORM, Armazenamento *Cloud* (AWS S3), Desenvolvimento Assistido por LLM, Automação de Processos (*Cron Jobs*).

Índice

Resumo	1
Índice	3
Índice de Figuras	7
Índice de Tabelas	9
1. Introdução	1
1.1. Contextualização	1
1.2. Motivação (Viabilidade, Objetivos)	2
1.3. Plano de Execução	2
2. Análise do Caso de Estudo	8
2.1. Contextualização	8
2.2. Apresentação do Caso de Estudo	9
2.3. Motivação e Objectivos	9
2.3.1. Motivação	9
2.3.2. Objetivos	10
2.4. Recursos	11
2.4.1. Recursos Físicos	11
2.4.2. Recursos Materiais e Software	11
2.5. Prompts e geração dos resultados	12
3. Conceção e Engenharia de Requisitos Assistida	15
3.1. Identificação de Stakeholders	15
3.2. Análise de contexto e restrições	16
3.3. Eliciação de Requisitos	17
3.3.1. Relatório de Observação Direta	17
3.3.2. Leitura de Contrato	17
3.4. Entrevistas com stakeholders	17
3.4.1. Entrevista com a Fundadora e Gestora Técnica (Diana Silva)	18
3.4.2. Entrevista com um Cliente	18
3.4.3. Entrevista com um membro do Staff	19
3.4.4. Conclusões do Levantamento de Informação	20
3.5. Requisitos funcionais e não funcionais	20
3.6. Refinamento de requisitos ambíguos	24
3.6.1. Requisitos Funcionais	24
3.6.2. Requisitos Não Funcionais	27
3.7. Geração de user stories	28
3.7.1. Diana Gonçalves Silva (Veterinária e Gestora)	28
3.7.2. Clientes (Tutores de Animais)	29
3.7.3. Staff (Cuidadores e Operacionais)	30
3.7.4. Receção e Administrativo	31
3.8. Casos de Uso	31
3.9. Diagramas de Atividades	33
3.10. Prompts e geração dos resultados	37
3.10.1. Simulação de entrevista	37

3.10.1.1. Prompts Iniciais	37
3.10.1.2. Avaliação Crítica do Resultado Inicial e Refinamento	38
3.10.2. Simulação de eliciação	39
3.10.2.1. Prompt Inicial	39
3.10.2.2. Avaliação Crítica do Resultado Inicial e Refinamento	39
3.10.3. Levantamento de Requisitos	40
3.10.3.1. Prompt Inicial	40
3.10.3.2. Avaliação Crítica do Resultado Inicial e Refinamento	40
3.10.4. Refinamento de Requisitos	41
3.10.4.1. Prompt Inicial	41
3.10.4.2. Avaliação Crítica do Resultado Inicial e Refinamento	41
4. Arquitetura e Design do Software	43
4.1 Análise de Alternativas Arquiteturais	43
4.1.1. Justificação da Escolha	44
4.2. Definição da Arquitetura Global	45
4.2.1. Camada de Apresentação (Frontend / Clientes)	45
4.2.2. Camada de Lógica de Negócio (Application Server / Backend)	46
4.2.3. Camada de Dados (Database Tier)	46
4.3. Decisões Arquiteturais e Stack Tecnológica	46
4.3.1. Padrão de Desenho Interno do Servidor (Arquitetura Hexagonal)	47
4.3.2. Tecnologias de Frontend (Camada de Apresentação)	47
4.3.3. Controlo de Acessos e Segurança (RBAC)	48
4.3.4. Base de Dados e Infraestrutura Cloud (AWS)	48
4.4. Análise e justificação das escolhas tecnológicas	49
4.4.1. Escolha da infraestrutura Cloud	49
4.4.2. Análise Comparativa e Justificação da Stack Tecnológica	50
4.4.2.1. Camada de Apresentação (Frontend)	50
4.4.2.2. Camada de Lógica de Negócio (Backend)	51
4.4.2.3. Camada de Base de Dados	52
4.5. Modelação do Sistema (UML)	53
4.5.1. Diagrama de Componentes	53
4.5.2. Diagrama de Classes	55
4.5.3. Diagramas de Sequência	56
4.6. Design de Interfaces (UI/UX)	62
4.6.1. Módulo Web do Cliente (Tutor)	62
4.6.2. Dashboard de Gestão (Desktop)	66
4.6.3. Módulo Operacional do Staff (Mobile-First Web)	67
4.6.4. Módulo do Veterinário	68
4.7. Especificação de API e Troca de Dados	69
4.7.1. Definição do Protocolo de Troca de Dados	69
4.7.2. Documentação dos Endpoints Principais	70
4.8. Prompts e geração dos resultados	73
4.8.1. Definição da Stack Tecnológica e Arquitetura Hexagonal	73
4.8.1.1. Prompt Inicial	73

4.8.1.2. Avaliação Crítica do Resultado Inicial e Refinamento	73
4.8.2. Geração de Esboço Estrutural para o Diagrama de Classes	74
4.8.2.1. Prompt Inicial	74
4.8.2.2. Avaliação Crítica do Resultado Inicial e Refinamento	74
4.8.3. Desenho da Infraestrutura Cloud (AWS Free Tier)	75
4.8.3.1. Prompt Inicial	75
4.8.3.2. Avaliação Crítica do Resultado Inicial e Refinamento	75
5. Implementação e Desenvolvimento Assistido por LLM	77
5.1. Configuração do Ambiente de Trabalho e Infraestrutura Cloud	78
5.2. Orquestração e Implementação na Cloud (AWS)	79
5.2.1. Amazon RDS e Prisma: Persistência e Segurança de Dados	79
5.2.2. Amazon S3: Gestão Eficiente e Segura de Media	79
5.2.3. Alojamento, Distribuição e Integração Contínua	80
5.3. Desenvolvimento da Camada de Dados (Data Layer)	81
5.3.1. Modelação e Esquema Lógico da Base de Dados	81
5.3.2.1. Relações Principais	83
5.3.2.2. Escolhas de Design	84
5.3.2.3. Modelo Lógico	85
5.3.2. Abstração Relacional com Prisma ORM e Padrão DAO	85
5.3.3. Integração de Armazenamento com S3StorageAdapter	88
5.4. Desenvolvimento da Camada de Lógica de Negócio	89
5.4.1. O Núcleo Lógico e o Padrão Facade	89
5.4.2. Segurança e Controlo de Acessos (Middleware)	90
5.4.3. Automatização de Tarefas (Cron Jobs)	91
5.4.4. Integração de Serviços de Terceiros	91
5.5. Desenvolvimento da Camada de Apresentação	92
5.5.1. Estrutura da Aplicação e Roteamento	93
5.5.2. Validação Visual com a Stakeholder e Refinamento da Interface	93
5.5.3. Módulo de Autenticação	94
5.5.3.1. Página de Login (LoginPage.tsx)	95
5.5.3.2. Página de Registo (RegisterPage.tsx)	96
5.5.4. Página Pública - Homepage (HomePage.tsx)	96
5.5.5. Módulo da Gestora - Dashboard da Gestora (GestoraPage.tsx e GatewayGestoraPage.tsx)	97
5.5.6. Módulo da Recepção (RececaoPage.tsx)	98
5.5.7. Módulo Veterinário (VeterinariaPage.tsx)	100
5.5.8. Módulo do Staff Operacional (StaffPage.tsx)	101
5.5.9. Módulo de Marcações (MarcacoesPage.tsx)	103
5.5.10. Diário de Bordo (DiarioBordoPage.tsx)	103
5.5.11. Portal do Tutor (PortalTutor.tsx)	104
5.5.12. Princípios Transversais de Desenvolvimento da Interface	105
5.6. Revisão de Código	106
5.6.1. Execução da Metodologia Semi-Profissional no Projeto	106
5.6.2. Análise Crítica de Casos de Estudo e Code Review	107

5.7. Prompts e Geração dos Resultados	108
5.7.1. Refatoração e Alinhamento com a Arquitetura Hexagonal	108
5.7.1.1. Prompt Inicial	108
5.7.1.2. Avaliação Crítica e Refinamento do Resultado	109
5.7.2. Lógica Transacional e Regras de Negócio (Prisma)	109
5.7.2.1. Prompt Inicial	109
5.7.2.2 Avaliação Crítica e Refinamento do Resultado	110
5.7.3. Desenvolvimento de Interface Complexa (Frontend)	110
5.7.3.1. Prompt Inicial	110
5.7.3.2. Avaliação Crítica e Refinamento do Resultado	110
6. Verificação, Validação e Avaliação da Qualidade do Software Produzido	112
6.1. Realização de Testes Automatizados	112
6.1.1. Testes Unitários	112
6.1.2. Testes de Integração	113
6.2. Testes de Sistema e de Aceitação: Simulação em Cenário Real	114
6.2.1. Testes Automatizados End-to-End (E2E) com Playwright	114
6.2.2. Testes Beta e de Usabilidade (Manuais)	115
6.3. Matrizes de Rastreabilidade	116
6.3.1 Matriz de Rastreabilidade RF	116
6.3.2. Matriz de Rastreabilidade RNF	120
6.3.3. Requisitos não Atendidos	122
6.4. Cobertura de Código e Análise de Qualidade	123
6.5. Prompts e geração dos resultados	124
6.5.1. Planeamento da Fase de Testes e Validação	124
6.5.1.1. Prompt Inicial	124
6.5.1.2. Avaliação Crítica e Refinamento do Resultado	125
6.5.1.3. Refinamento do Resultado (Novo Prompt)	125
7. Conclusões e Trabalho Futuro	127
7.1. Aspetos Positivos e Valor Entregue	127
7.2. Aspetos Negativos e Desafios	128
7.3. Trabalho Futuro	129
7.4. O Paradigma do Desenvolvimento Assistido por LLMs - Uma Reflexão	130
7.4.1. O que a IA realmente foi	130
7.4.2. O que a IA não conseguiu ser	130
7.4.3. A questão que fica	131
Referências Bibliográficas	132
Lista de Siglas e Acrónimos	133
Apêndices	134

Índice de Figuras

Figura 1: Diagrama de Gantt objetivo – Capítulo 1.	1
Figura 2: Diagrama de Gantt objetivo – Etapa 1.	3
Figura 3: Diagrama de Gantt final – Etapa 1.	4
Figura 4: Diagrama de Gantt objetivo – Etapa 2.	5
Figura 5: Diagrama de Gantt final – Etapa 2.	5
Figura 6: Diagrama de Gantt objetivo – Etapa 3.	5
Figura 7: Diagrama de Gantt final – Etapa 3.	6
Figura 8: Diagrama de Gantt objetivo – Etapa 4.	6
Figura 9: Diagrama de Gantt final – Etapa 4.	6
Figura 10: Diagrama de Gantt final – Capítulo 1.	7
Figura 11: Diagrama de Gantt objetivo – Capítulo 2.	8
Figura 12: Diagrama de Gantt final – Capítulo 2.	14
Figura 13: Diagrama de Gantt objetivo – Capítulo 3.	15
Figura 14: Diagrama de Use Cases.	33
Figura 15: Diagrama de Atividade - Efetuar Reserva e Alocação.	34
Figura 16: Diagrama de Atividade - Executar Checklist Diário.	35
Figura 17: Diagrama de Atividade - Visualizar Diário de Bordo.	36
Figura 18: Diagrama de Gantt final – Capítulo 3.	42
Figura 19: Diagrama de Gantt objetivo – Capítulo 4.	43
Figura 20: Diagrama de Componentes.	54
Figura 21: Diagrama de Classes.	56
Figura 22: Diagrama de Sequência - faturaFinal.	57
Figura 23: Diagrama de Sequência - cadastrarAnimal.	57
Figura 24: Diagrama de Sequência - prescreverMedicação.	58
Figura 25: Diagrama de Sequência - adicionarTarefa.	59
Figura 26: Diagrama de Sequência - criarConta.	59
Figura 27: Diagrama de Sequência - adicionarServico.	60
Figura 28: Diagrama de Sequência - reporStock.	61
Figura 29: Diagrama de Sequência - efetuarReserva.	61
Figura 30: Divisão em passos lógicos.	62
Figura 31: Opções de Utilização.	63
Figura 32: Diário de Bordo.	64
Figura 33: Marcação de Dias.	65
Figura 34: Dashboard com Tarefas.	66
Figura 35: Foto do Stock com os filtros de lado.	66
Figura 36: UI para One-Hand Operation.	67
Figura 37: Dashboard do Veterinário.	68
Figura 38: Dashboard do Veterinário.	69
Figura 39: Diagrama de Gantt final – Capítulo 4.	76
Figura 40: Diagrama de Gantt objetivo– Capítulo 5.	77
Figura 41: Esquema da arquitetura cloud.	81
Figura 42: Diagrama Entidade-Relacionamento.	85

Figura 43: Exemplo de regra implementada.	87
Figura 44: Código de link temporário.	88
Figura 45: Middleware de autenticação JWT.	91
Figura 46: Exemplo de email de 2FA.	92
Figura 47: Página de Login.	95
Figura 48: Página de Login com o 2FA.	95
Figura 49: Página de Registo.	96
Figura 50: Home Page.	97
Figura 51: Página de escolha da Gestora.	98
Figura 52: Página exclusiva da Gestora.	98
Figura 53: Página da Rececionista - Check-In.	99
Figura 54: Pré-Visualização da Fatura - Check-Out.	100
Figura 55: Página do Veterinário - Verificar Cães.	101
Figura 56: Página do Veterinário - Tratamento e Prescrições.	101
Figura 57: Página do Staff - CheckList de Tarefas.	102
Figura 58: Página do Staff - Higienização Boxes.	102
Figura 59: Página do Tutor - Marcações.	103
Figura 60: Diário de Bordo do cão Marega.	104
Figura 61: Página Principal do Tutor.	105
Figura 62: Histórico de Faturação do Tutor.	105
Figura 63: Diagrama de Gantt final – Capítulo 5.	111
Figura 64: Diagrama de Gantt objetivo – Capítulo 6.	112
Figura 65: Testes Backend unitários e de Integração.	113
Figura 66: Testes com o Playwright 14/14.	115
Figura 67: Diagrama de Gantt final – Capítulo 6.	126

Índice de Tabelas

Tabela 1: Avaliação Crítica dos Prompts - Capítulo 2.	14
Tabela 2: Tabela de requisitos inicial.	23
Tabela 3: Tabela de requisitos funcionais.	27
Tabela 4: Tabela de requisitos não funcionais.	28
Tabela 5: Prompts da simulação de entrevistas.	38
Tabela 5: Prompts da simulação de eliciações.	39
Tabela 6: Prompts de levantamento de entrevistas.	41
Tabela 7: Prompts de refinamento de requisitos.	42
Tabela 8: Prós e Contras de serviços Cloud.	49
Tabela 9: Prós e Contras das opções frontend.	50
Tabela 10: Prós e Contras das opções backend.	51
Tabela 11: Prós e Contras das opções da base de dados.	52
Tabela 12: Prompts da Arquitetura e Stack Tecnológica .	74
Tabela 13: Prompts do Diagrama de Classes.	75
Tabela 14: Prompts da Infraestrutura AWS.	76
Tabela 15: Entidades Principais da Base de Dados.	83
Tabela 16: Resumo dos Data Access Objects (DAOs) implementados.	86
Tabela 17: Resumo das Facades implementadas e respectivas responsabilidades.	90
Tabela 18: Prompts da Refatorização da Arquitetura - Backend.	109
Tabela 19: Prompts do Prisma.	110
Tabela 20: Prompts do Frontend.	111
Tabela 21: Rastreabilidade dos requisitos funcionais.	120
Tabela 22: Rastreabilidade dos requisitos não funcionais.	122
Tabela 23: Requisitos não respondidos.	123
Tabela 24: Cobertura e Testes.	124
Tabela 25: Prompts da Fase de Testes.	125

1. Introdução

Este capítulo introdutório apresenta o enquadramento do projeto "Au Au... Sou um Cão!", detalhando o contexto do domínio, as características do caso de estudo em análise, bem como as motivações e os objetivos fundamentais que guiaram a equipa na proposta deste sistema de gestão.

O presente capítulo foi planeado para ser desenvolvido entre os dias 09/02 e 16/02, correspondendo à fase inicial da Etapa 1. A figura seguinte apresenta o diagrama de Gantt objetivo para as secções que o constituem.



Figura 1: Diagrama de Gantt objetivo – Capítulo 1.

1.1. Contextualização

O presente trabalho prático desenvolve-se na Unidade Curricular de Laboratórios de Informática IV (LI4), do 2º semestre do Ano Letivo 2025/2026. Num cenário onde a Engenharia de *Software* enfrenta uma revolução na forma como os sistemas são idealizados e mantidos, este projeto foca-se na utilização sistemática de *Large Language Models* (LLM) como agentes colaboradores.

O caso de estudo selecionado para aplicar estas metodologias é o Tema 4: Sistema de gestão para um hotel de animais. O hotel "Au Au... Sou um Cão!", fundado pela estudante de veterinária Diana Silva em Braga, serve como o domínio real para a conceção, especificação e desenvolvimento de um sistema que suporte a transição de um modelo de gestão manual para uma infraestrutura digital resiliente e orientada por dados.

1.2. Motivação (Viabilidade, Objetivos)

Inspirada pelas diretrizes da equipa docente da escola de engenharia da Universidade do Minho (EEUM), a equipa estabeleceu os seguintes objetivos principais:

- Gestão de Ciclo de Vida Completo: Realizar todas as fases do *Software Development Life Cycle* (SDLC), desde a análise de requisitos até ao *deployment* e manutenção.
- Integração de Agentes Inteligentes: Compreender e aplicar o papel dos LLMs em todas as etapas, desde a geração de código à validação arquitetural. Do ponto de vista de análise de comportamento
- Profissionalização da Gestão: Facilitar o controlo de reservas, registos clínicos veterinários e faturação automática certificada pela Autoridade Tributária (AT).
- Qualidade e Ética: Saber produzir peças de *software* comparáveis ao nível profissional, assegurando práticas éticas no uso da inteligência artificial generativa (IAG).

1.3. Plano de Execução

A equipa adotou a metodologia ágil *Scrum* para governar a execução deste projeto. Esta escolha justifica-se pela necessidade de gerir um desenvolvimento iterativo e incremental, permitindo adaptações rápidas aos requisitos selecionados junto da *stakeholder* Diana Silva e garantindo entregas de valor contínuas.

Dada a natureza académica do projeto, onde os membros da equipa não se encontram dedicados a tempo inteiro, procedeu-se a uma adaptação dos eventos padrão do *Scrum* para alinhar com o regime de uma a duas reuniões por semana. Desta forma, os encontros síncronos assumem um carácter híbrido: a primeira reunião semanal serve essencialmente para o planeamento da iteração e para a revisão do trabalho concluído; a segunda reunião, quando necessária, funciona como um ponto de sincronização rápido para partilha de progresso, discussão de obstáculos e eventual resolução conjunta de problemas.

O projeto tem um horizonte temporal total definido entre 09 de fevereiro e 27 de maio de 2026, estando estruturado em quatro etapas sequenciais de engenharia, cada uma com objetivos e entregáveis bem definidos. A Etapa 1, que engloba os capítulos 1, 2 e 3 deste relatório, decorreu entre 09/02 e 02/03 e incidiu sobre a conceção, análise do domínio e engenharia de requisitos. A Etapa 2, correspondente ao capítulo 4, decorreu entre 02/03 e 23/03 e foi dedicada à arquitetura e design do sistema. A Etapa 3, correspondente ao capítulo 5, decorreu entre 23/03 e 13/04 e focou-se na implementação assistida por LLM. Por fim, a Etapa 4 decorreu entre 13/04 e 11/05 e foi dedicada à verificação, validação e avaliação da qualidade do *software* produzido.

A distribuição de responsabilidades dentro da Etapa 1 foi a seguinte: a introdução ficou a cargo do Gonçalo Sá, a análise do caso de estudo foi liderada pelo Hugo Cunha com base nas entrevistas realizadas, a conceção e engenharia de requisitos foi conduzida pelo Diogo Pais, e a arquitetura e *design* do *software* foi desenvolvida pelo David Alves.

A figura seguinte apresenta o diagrama de Gantt completo da Etapa 1, com a comparação entre o plano objetivo e a execução real, permitindo identificar os desvios ocorridos ao longo do processo.

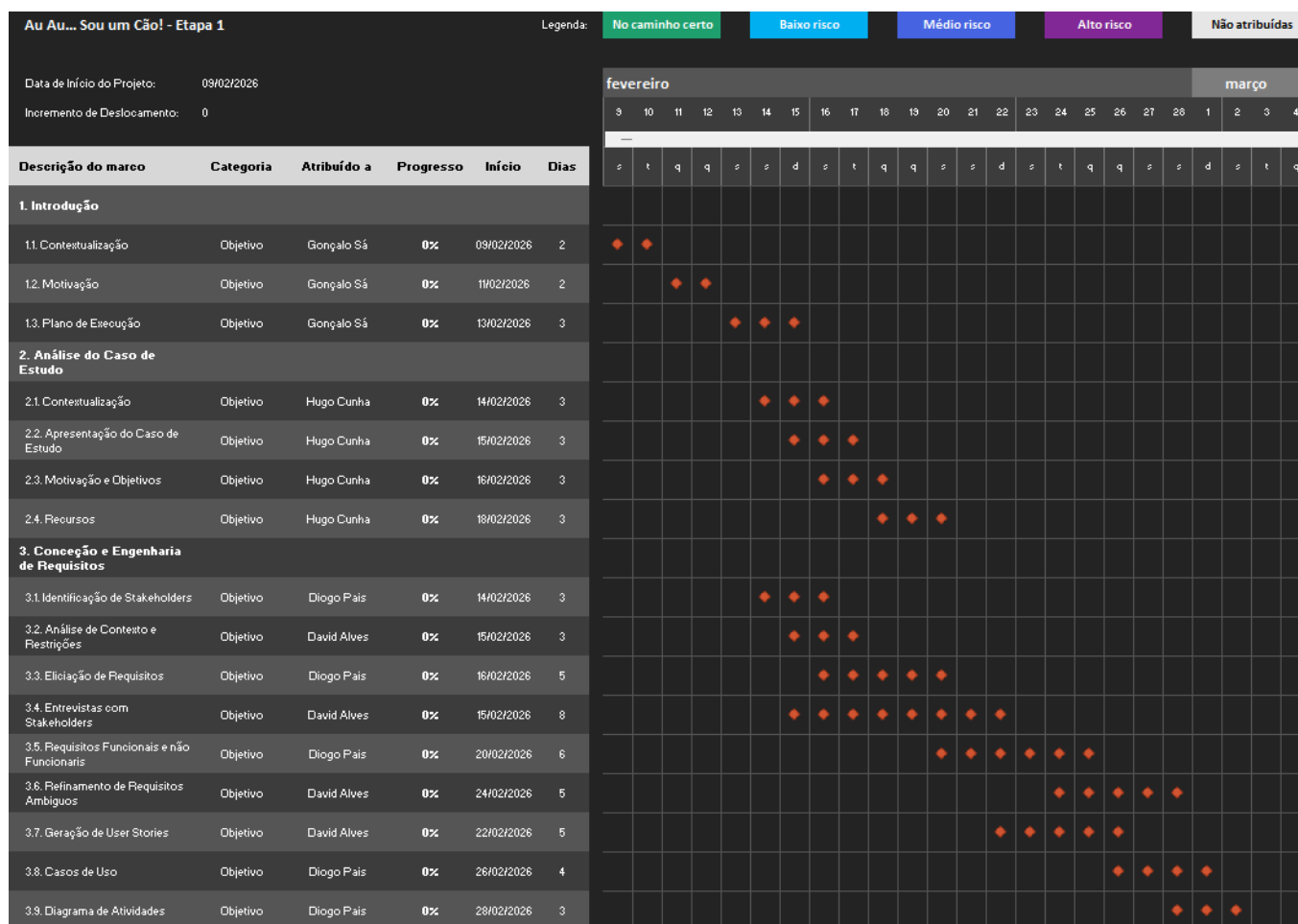


Figura 2: Diagrama de Gantt objetivo – Etapa 1.

A Etapa 1 foi concluída dentro do prazo global estabelecido de 09/02 a 02/03, ainda que com alguns desvios pontuais face ao plano objetivo. O capítulo 1 registou apenas um atraso de um dia na secção 1.3, sem consequências para as fases seguintes. O capítulo 2 foi o único a ser executado integralmente dentro dos prazos previstos, refletindo a boa preparação da equipa relativamente ao domínio do problema. O capítulo 3 concentrou a maioria dos desvios da etapa, com atrasos de um dia nas entrevistas ao staff, na elaboração dos requisitos funcionais e não funcionais, no refinamento de

requisitos ambíguos e na geração de user stories, todos justificados pela complexidade e pelo volume de trabalho associados a estas tarefas. Apesar destes deslizamentos, as secções de casos de uso e diagramas de atividades foram concluídas dentro do prazo, permitindo fechar a etapa na data prevista de 02/03.

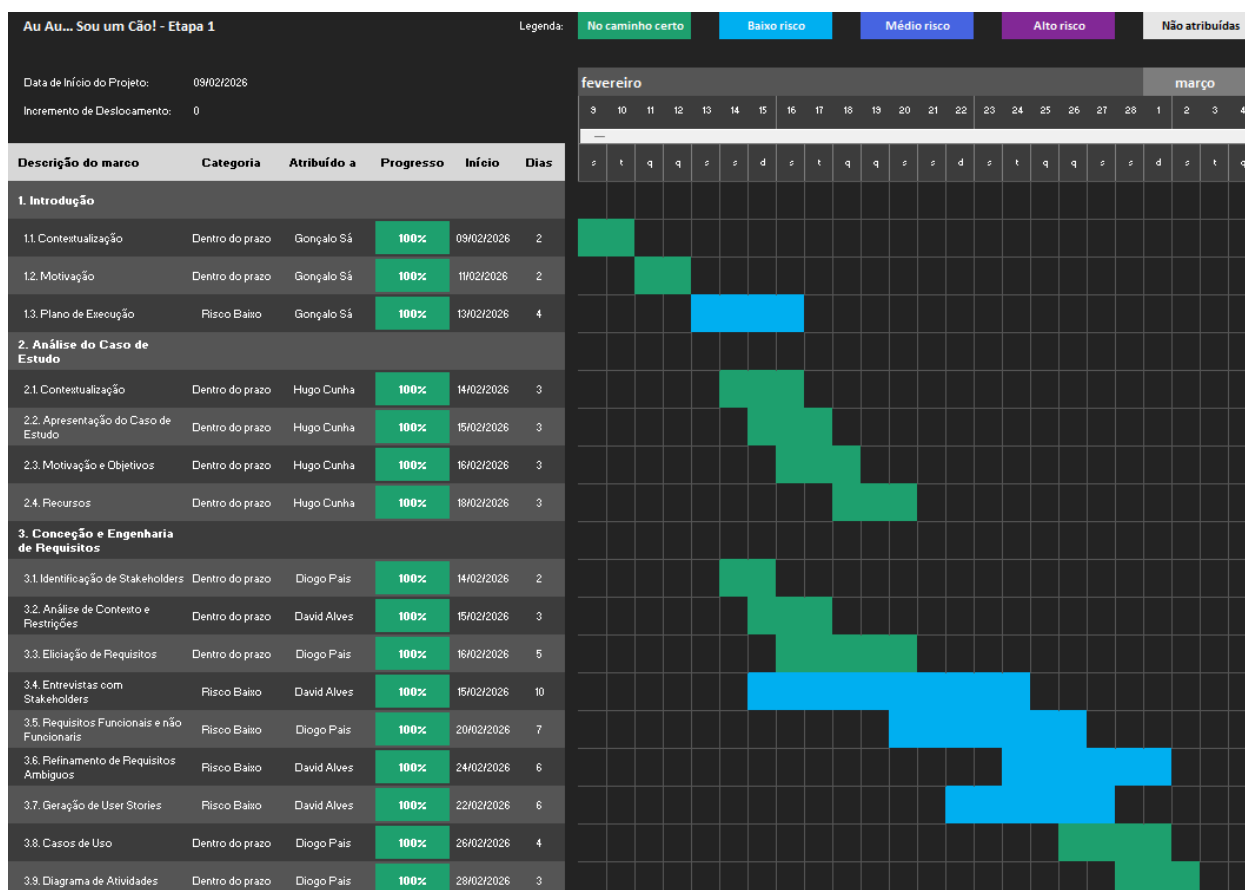


Figura 3: Diagrama de Gantt final – Etapa 1.

A Etapa 2, compreendida entre 02/03 e 23/03, foi dedicada à arquitetura e design do sistema, correspondendo ao capítulo 4. O Gonçalo Sá ficou responsável pela análise de alternativas e definição da arquitetura global, o David Alves pelas decisões arquiteturais, stack tecnológica, análise comparativa e especificação de API, o Hugo Cunha pela modelação UML, e o Diogo Pais pelo design de interfaces. A etapa decorreu de forma bastante controlada, com desvios máximos de um dia nas secções de maior complexidade técnica e de validação, sem comprometer a data de entrega global de 23/03.

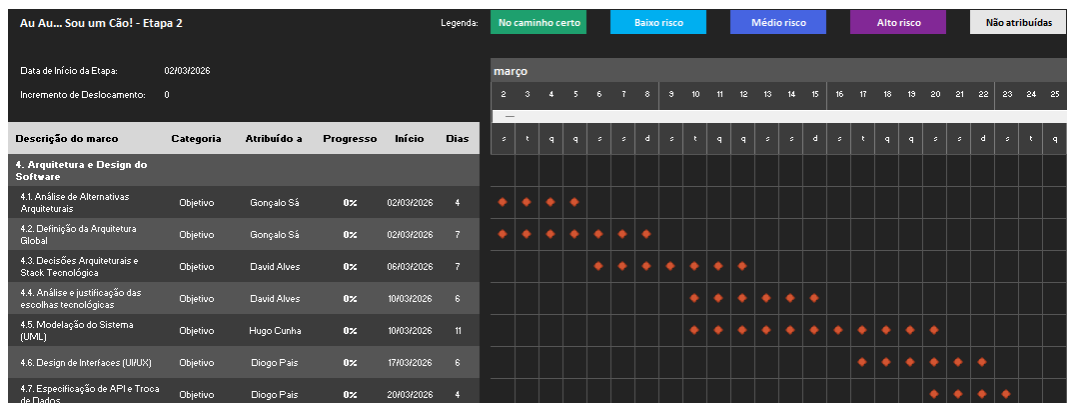


Figura 4: Diagrama de Gantt objetivo – Etapa 2.

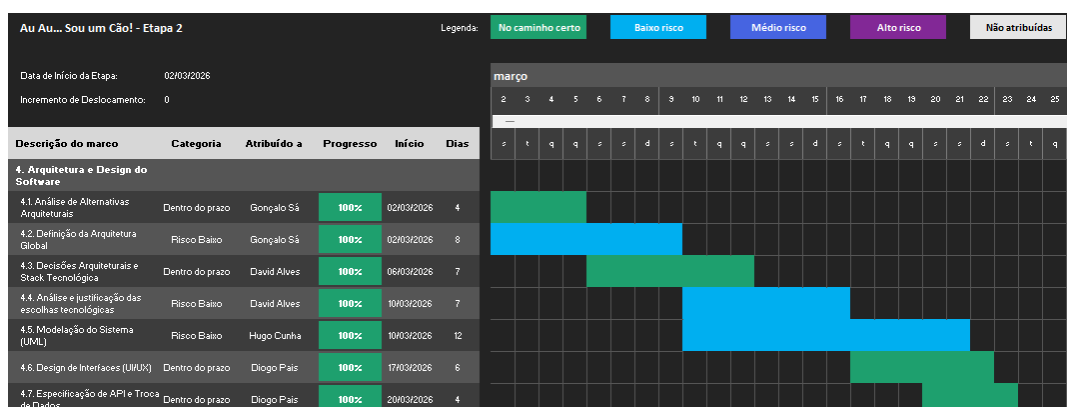


Figura 5: Diagrama de Gantt final – Etapa 2.

A Etapa 3, compreendida entre 23/03 e 13/04, foi dedicada à implementação e desenvolvimento assistido por LLM, correspondendo ao capítulo 5. O David Alves ficou responsável pela configuração do ambiente e pela orquestração cloud, o Hugo Cunha pelo desenvolvimento da camada de dados, o Gonçalo Sá pela camada de lógica de negócio, e o Diogo Pais pela camada de apresentação e todos os módulos frontend. Foi a etapa com mais atrasos pontuais, todos de um dia, concentrados nas fases de integração cloud e no frontend, sem comprometer a data de entrega.

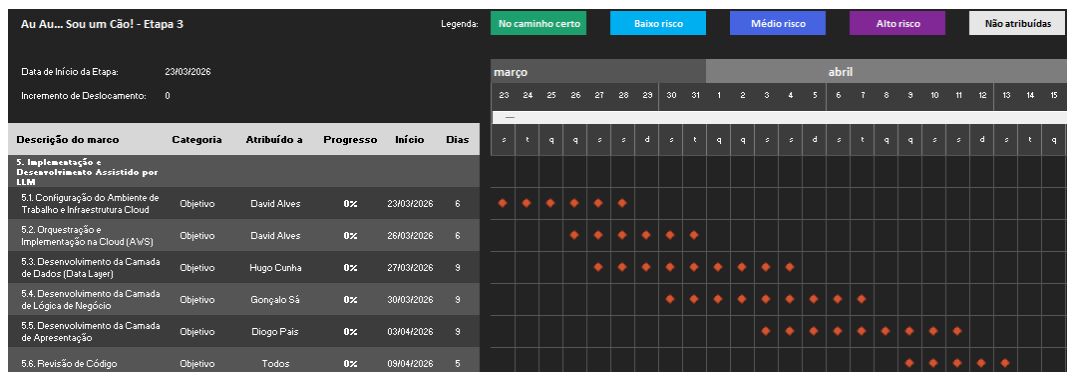


Figura 6: Diagrama de Gantt objetivo – Etapa 3.

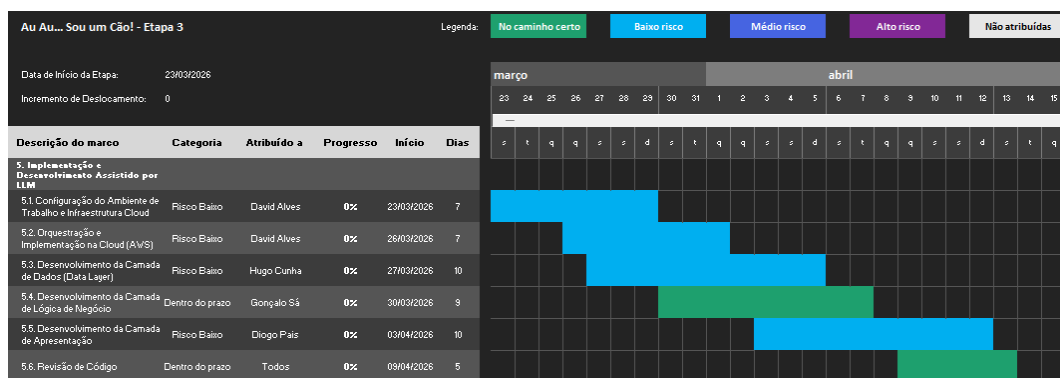


Figura 7: Diagrama de Gantt final – Etapa 3.

A Etapa 4, compreendida entre 13/04 e 11/05, foi dedicada aos testes, validação e garantia de qualidade, correspondendo ao capítulo 6. O Hugo Cunha liderou os testes automatizados, o Diogo Pais os testes de sistema e aceitação, o Gonçalo Sá a verificação dos requisitos e o David Alves a análise de cobertura e qualidade. A etapa decorreu de forma estável, com desvios máximos de um dia nas fases de testes de integração e aceitação, sendo o projeto concluído dentro do prazo global estabelecido pelo docente.

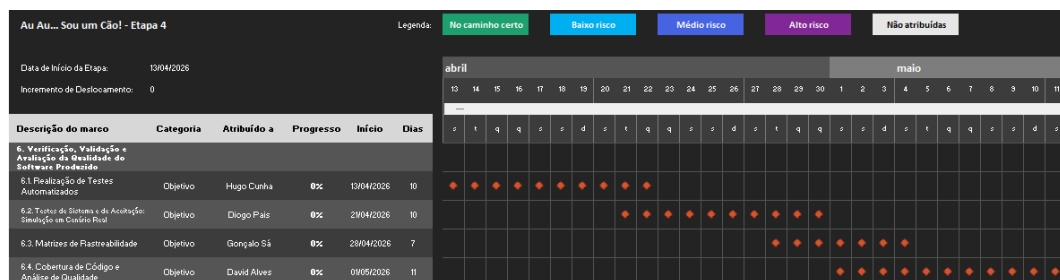


Figura 8: Diagrama de Gantt objetivo – Etapa 4.

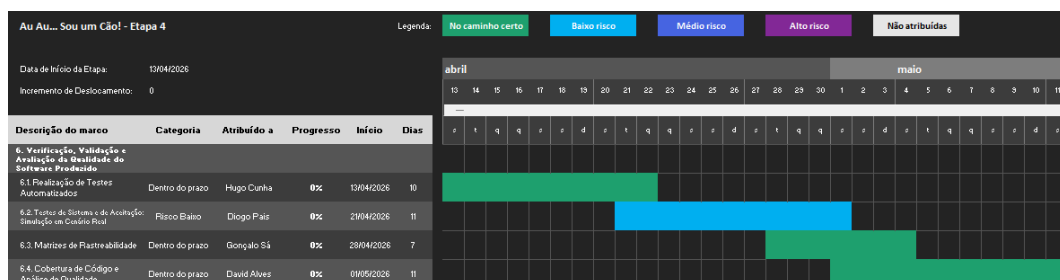


Figura 9: Diagrama de Gantt final – Etapa 4.

A execução do capítulo 1 decorreu de forma bastante próxima do planeado. As secções 1.1 e 1.2 foram concluídas dentro dos prazos definidos, entre os dias 09/02 e 13/02. A secção 1.3, relativa ao plano de execução, registou um atraso de um dia face

ao previsto, tendo sido concluída a 17/02 em vez de 16/02. Este desvio ficou a dever-se ao tempo adicional necessário para consolidar a estrutura do plano de projeto e alinhar a divisão de tarefas entre os membros da equipa. No global, o capítulo foi concluído sem impacto relevante nas etapas seguintes.

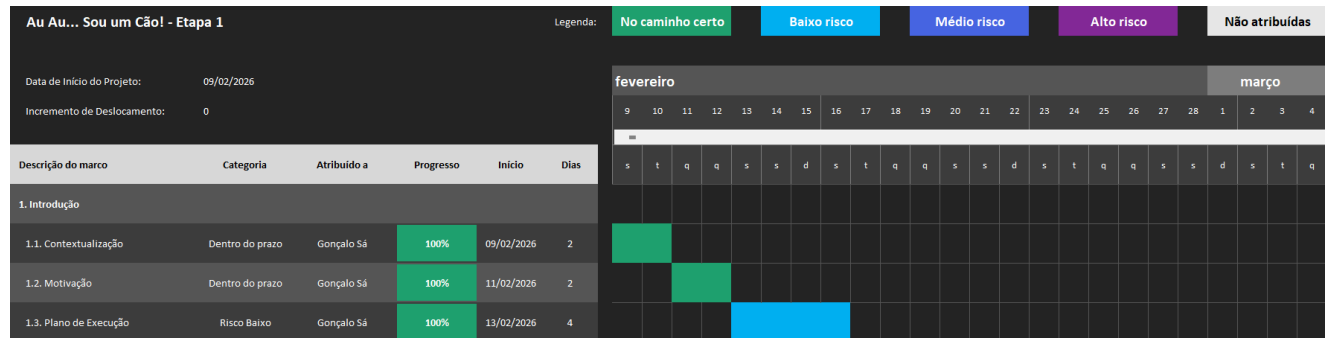


Figura 10: Diagrama de Gantt final – Capítulo 1.

2. Análise do Caso de Estudo

A análise do caso de estudo foi planeada para decorrer entre os dias 15/02 e 20/02, em sobreposição parcial com os capítulos 1 e 3, dado que a recolha de informação junto da *stakeholder* Diana Silva ocorreu em simultâneo com a redação introdutória. A figura seguinte apresenta o diagrama de Gantt objetivo para este capítulo.



Figura 11: Diagrama de Gantt objetivo – Capítulo 2.

2.1. Contextualização

O hotel de animais “Au Au... Sou um cão!” é um negócio criado por Diana Gonçalves Silva, estudante de Medicina Veterinária na Universidade de Trás-os-Montes e Alto Douro, em 2021. A ideia surgiu após as dificuldades sentidas durante a pandemia de COVID-19 e da dificuldade em encontrar trabalho na sua área perto de casa. Depois de uma conversa com o pai, decidiram avançar com a criação de um hotel para animais, unindo o gosto pelos animais aos conhecimentos adquiridos no curso.

No início, a equipa era composta apenas por Diana e pelo seu namorado, que a ajudava nos cuidados diários e na organização do espaço. O bom atendimento e a forma carinhosa como tratavam os animais fizeram com que o negócio crescesse rapidamente, conquistando a confiança dos clientes.

Com o aumento da procura, foi necessário melhorar a organização e adaptar as instalações, criando espaços mais adequados e seguros para os animais. Atualmente, Diana é responsável pela gestão e supervisão do hotel, contando com o apoio do namorado e de funcionários que auxiliam no atendimento e nos cuidados diários, garantindo um serviço de qualidade e bem-estar aos animais.

2.2. Apresentação do Caso de Estudo

O presente relatório foca-se no projeto e desenvolvimento do sistema de gestão para o hotel de animais "Au Au... Sou um Cão!", uma unidade hoteleira canina situada em Braga. Fundado por Diana Gonçalves Silva, estudante de Medicina Veterinária na Universidade de Trás-os-Montes e Alto Douro, o hotel nasceu da visão de aliar o rigor clínico ao bem-estar animal, oferecendo um serviço diferenciado na região que combina alojamento, cuidados clínicos e atividades de recreação.

A infraestrutura física do hotel compreende 40 quartos perfeitamente equipados. Contudo, o maior desafio operacional reside na acentuada dinâmica de ocupação sazonal:

- Época Baixa: 10 a 20 animais.
- Época Média: 20 a 30 animais.
- Época Alta: Picos críticos onde a lotação pode alcançar os 40 animais em simultâneo (recorrendo a gestão de espaços complementares e multi-ocupação controlada).

Para garantir a viabilidade deste modelo de negócio, o sistema deve centralizar a gestão de registos de animais (espécie, raça, plano vacinal, alergias) e proprietários, o controlo de reservas e estadias, e a coordenação de serviços especializados: planos alimentares, monitorização veterinária, cuidados de higiene (*grooming*) e passeios. Adicionalmente, o software assegurará a gestão financeira (pagamentos) e a geração de relatórios operacionais para apoio à decisão.

2.3. Motivação e Objectivos

A presente secção expõe os motivos fundamentais que impulsionaram a proposta e o desenvolvimento deste projeto, bem como os objetivos concretos que a equipa pretende alcançar com a sua implementação. Para garantir que o sistema de gestão a desenvolver responde às reais necessidades operacionais do negócio, as motivações e metas aqui descritas foram delineadas com base numa entrevista direta realizada com a fundadora do hotel, Diana Gonçalves Silva. Através deste levantamento presencial junto da principal *stakeholder*, foi possível compreender em primeira mão os desafios do dia a dia e definir um rumo tecnológico perfeitamente alinhado com a visão de crescimento, segurança e profissionalização do "Au Au... Sou um cão!".

2.3.1. Motivação

A motivação central para o desenvolvimento deste sistema reside na transição necessária de um modelo de gestão artesanal para uma infraestrutura digital resiliente.

Com o crescimento exponencial da operação do Hotel de Animais desde 2021 atingindo picos de 40 animais em épocas altas a dependência de processos manuais, como quadros brancos e registos em papel, tornou-se incomportável. A escala atual exige que a intuição e a memória da equipa sejam substituídas por sistemas de informação que garantam a integridade dos dados e a continuidade do serviço sem falhas humanas.

Em segundo lugar, a motivação é impulsionada pelo rigor clínico e etológico que define a identidade da instituição. Sendo a fundadora médica veterinária, a ausência de um controlo digital sobre stocks de medicação crítica, validades e protocolos de administração representa um risco operacional e jurídico elevado. A necessidade de garantir que cada animal recebe o cuidado específico (dietas pancreáticas, medicação para epilepsia ou isolamento sanitário) de forma rastreável é o motor principal para a procura de uma solução tecnológica de alta precisão.

Por fim, existe uma motivação de eficiência comunicacional e administrativa. O fluxo atual de interações via canais informais (*WhatsApp*) e a gestão manual de faturação complexa — que inclui políticas de cancelamento e descontos progressivos consomem recursos temporais preciosos da equipa. A automação destes processos não é apenas uma questão de conveniência, mas uma estratégia de sobrevivência organizacional, permitindo que o foco da equipa regresse exclusivamente ao bem-estar animal e à supervisão técnica.

2.3.2. Objetivos

O objetivo primordial deste projeto é o desenvolvimento de uma plataforma de gestão integrada que centralize todo o ciclo de vida da estadia do animal, desde a reserva até ao *check-out*. Para materializar esta visão, o projeto persegue os seguintes objetivos específicos:

- **Centralização da Informação:** Criar um repositório único de verdade, onde o histórico clínico, comportamental e de incidentes de cada hóspede seja facilmente consultável e exportável, eliminando a fragmentação de dados.
- **Apoio à Decisão e Segurança Clínica:** Dotar a equipa de ferramentas de apoio à decisão em tempo real, garantindo a rastreabilidade e a segurança clínica em larga escala.
- **Gestão Inteligente de Ocupação:** Implementar algoritmos para a logística interna, incluindo um motor de alocação de quartos baseado em regras de vizinhança (compatibilidade de porte, reatividade e estado de saúde).
- **Controlo de Inventário Clínico:** Desenvolver um sistema de monitorização de *stock* clínico e dietético, dotado de alertas automáticos para prevenir ruturas.

- **Automação Financeira e de Faturação:** Automatizar o processamento de variáveis complexas (estadias, penalizações e serviços extra) de forma instantânea, minimizando erros de cálculo e tempos de espera no atendimento ao cliente.
- **Modernização da Comunicação:** Melhorar a interface com o cliente através de um "Diário de Bordo" digital, elevando a confiança dos tutores ao fornecer transparência total via notificações automatizadas e relatórios de incidentes estruturados.
- **Segurança e Controlo de Acessos:** Implementar um sistema de permissões de acesso diferenciado (Cuidadores, Gestão e Veterinário) para garantir a estrita segurança da informação e a conformidade com as normas do regulamento geral da proteção de dados (RGPD).

2.4. Recursos

2.4.1. Recursos Físicos

Pessoal Externo:

- Os quatro alunos de LEI (Hugo Cunha, David Alves, Diogo Pais e Gonçalo Sá) são os responsáveis diretos pelo levantamento de requisitos, modelação conceptual/lógica e implementação.
- A supervisora Diana que orienta o grupo na aplicação das metodologias clássicas enriquecidas com práticas modernas

Pessoal Interno(*Stakeholders* e Supervisão):

- Diana Gonçalves Silva (Proprietária): Estudante de Veterinária e gestora do hotel em Braga; é a principal fonte de requisitos técnicos e operacionais.
- *Staff* composta pelo namorado dela e funcionários;
- Equipa Veterinária;

2.4.2. Recursos Materiais e Software

- Recursos Materiais: Computadores portáteis para a receção e para o grupo de Trabalho tal como dispositivos móveis para uso diário do Staff nas suas tarefas mundanas.
- Recursos *Software*:
 - Ferramentas de Engenharia: Sistema de Gestão de Base de Dados (SGBD) para gestão de dados como o MySQL, editores de código

- (VScode), ferramentas de modelação UML(Visual Studio Paradigm) e Google Docs para escrever o relatório.
- Metodologias: Normas IEEE 830/29148 para SRS e ISO/IEC 25010 para avaliação de qualidade.

2.5. Prompts e geração dos resultados

A presente secção documenta a estratégia de interação com os LLMs durante as fases iniciais do projeto, detalhando o *prompt* formulado, a avaliação crítica das respostas geradas e o processo iterativo de refinamento aplicado para atingir os resultados finais.

2.5.1. Prompt Inicial

A elaboração do prompt inicial procurou enquadrar o assistente de Inteligência Artificial no contexto académico e nas necessidades operacionais específicas do caso de estudo. O conteúdo submetido, transcrito na íntegra abaixo, foi o seguinte:

“Contexto do Projeto:

Atua como um Consultor de Engenharia de *Software* Sênior. Estou a realizar o trabalho prático da UC de Laboratórios de *Informática IV na Licenciatura em Engenharia Informática da Universidade do Minho (2025/2026)*. O objetivo é desenvolver um sistema de software utilizando LLMs como agentes artificiais de forma sistemática em todas as fases do ciclo de vida. O projeto segue uma metodologia sequencial e incremental inspirada em modelos como Waterfall ou V-Model, integrada com práticas de *AI-assisted SE*.

Caso de Estudo: Hotel de Animais "Au Au... Sou um Cão!"

Este hotel, localizado em Braga e fundado por Diana Gonçalves Silva (estudante de Veterinária na UMinho), necessita de um sistema de gestão integrado. O hotel possui 40 quartos e enfrenta uma flutuação sazonal significativa: época baixa (10-20 animais), época média (20-30 animais) e época alta (40 animais).

Tarefa: Com base nestes dados e no Tema 4 do enunciado de LI4, redige as seguintes secções para a introdução do meu relatório técnico, utilizando um tom académico e profissional:

1. **Contextualização:** Apresenta o cenário atual da Engenharia de Software, onde a complexidade dos sistemas e a integração de Inteligência Artificial Generativa (IAG) estão a transformar a análise, especificação e desenvolvimento de software. Explica como os LLMs deixaram de ser meros geradores de código para se

tornarem colaboradores ativos no ciclo de vida do software.

2. **Apresentação do Caso de Estudo:** Descreve o hotel "Au Au... Sou um Cão!", a sua localização em Braga e a visão da fundadora Diana Gonçalves Silva. Detalha a infraestrutura (40 quartos) e a dinâmica de ocupação sazonal. Refere que o sistema deve gerir registos de animais, proprietários, reservas, estadias, serviços (alimentação, veterinária, banhos, passeios), pagamentos e relatórios operacionais.

3. **Motivação e Objetivos:**

Motivação: A necessidade de uma gestão eficiente para lidar com picos de procura (até 120+ animais), garantindo a qualidade dos cuidados e a otimização dos recursos. A oportunidade de aplicar metodologias modernas (*AI-assisted SE*) para reduzir o "time-to-market" e aumentar a qualidade do produto final.

Objetivos: Conceber, especificar, desenvolver e validar um sistema robusto assistido por LLM. Os objetivos técnicos incluem a criação de um SRS (IEEE 830), design de arquitetura, implementação modular e verificação de qualidade (ISO/IEC 25010).

4. **Estrutura do Relatório:** Resume o que será abordado nos capítulos seguintes, correspondendo às etapas do projeto: Engenharia de Requisitos (Etapa 1), Arquitetura e Design (Etapa 2), Implementação (Etapa 3) e Verificação/Validação (Etapa 4)."

2.5.2. Avaliação Crítica do Resultado Inicial e Refinamento

Avaliação Crítica do Resultado	Refinamento Aplicado / Ação Tomada
Ambiguidade de Âmbito: Ausência de uma demarcação clara entre a vertente estritamente académica do projeto e o âmbito aplicacional direcionado à gestão do hotel.	Exigiu-se ao modelo a separação rigorosa entre a componente académica do projeto e o produto a ser desenvolvido para o hotel.
Omissão de Recursos: Falta de identificação e caracterização dos recursos necessários para a concretização do projeto.	Solicitou-se a introdução detalhada dos recursos alocados, com especial incidência nos Recursos Humanos (<i>stakeholders</i>). A equipa interveio manualmente para filtrar a informação

	pertinente e "humanizar" as pessoas envolvidas.
Desalinhamento de Objetivos: Apresentação de metas difusas, carentes de foco na resolução dos problemas operacionais reais do hotel.	Os objetivos foram reajustados, instruindo a IA para ancorar as metas em exemplos reais de impacto positivo na rotina do hotel.
Sobrefoco Tecnológico: Destaque desproporcional ao papel da Inteligência Artificial na conceção do trabalho, em detrimento do modelo de negócio e da solução de engenharia propriamente dita.	Determinou-se a supressão do foco excessivo na assistência prestada pela IA, redirecionando a tónica para a Engenharia de Software.

Tabela 1: Avaliação Crítica dos Prompts - Capítulo 2.

A execução do capítulo 2 decorreu integralmente dentro dos prazos planeados, entre os dias 15/02 e 20/02. As secções de contextualização, apresentação do caso de estudo, motivação e objetivos, e recursos foram todas concluídas nos intervalos previstos, sem registo de desvios significativos. Este resultado reflete a boa preparação prévia da equipa relativamente ao domínio do problema, bem como a disponibilidade da *stakeholder* Diana Silva para as sessões de recolha de informação realizadas presencialmente.



Figura 12: Diagrama de Gantt final – Capítulo 2.

3. Conceção e Engenharia de Requisitos Assistida

Este capítulo detalha a fase inicial do ciclo de vida do desenvolvimento do sistema "Au Au... Sou um cão!". Aqui são apresentados os intervenientes do projeto, o contexto de negócio, e os processos de levantamento, análise e especificação de todos os requisitos funcionais e não funcionais necessários para o sucesso da plataforma.

A conceção e engenharia de requisitos foi planeada para decorrer entre os dias 15/02 e 02/03, sendo a fase de maior duração e complexidade dentro da Etapa 1. A figura seguinte apresenta o diagrama de Gantt objetivo para este capítulo.

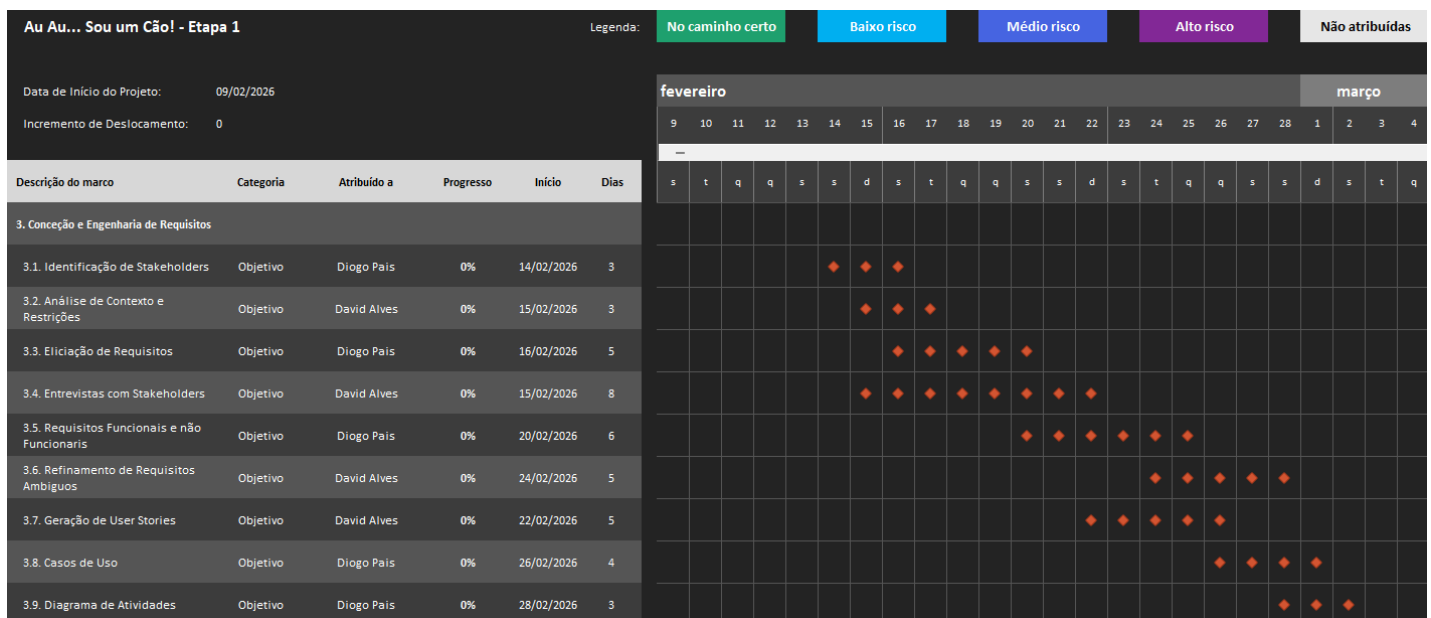


Figura 13: Diagrama de Gantt objetivo – Capítulo 3.

3.1. Identificação de Stakeholders

- Diana Gonçalves Silva (Proprietária e Gestora): Fundadora do hotel e estudante de Medicina Veterinária. Atua como a principal responsável pela gestão, supervisão e tomada de decisões estratégicas.
- *Staff* (Funcionários de Apoio): Equipa composta atualmente por dois funcionários e o namorado da proprietária. São responsáveis pelos cuidados diários, alimentação, passeios e higiene (*grooming*) dos animais. Utilizarão o sistema para registar atividades e consultar o plano de cuidados de cada hóspede.
- Equipa Médica / Veterinária: Liderada pela Diana, integra o rigor clínico no serviço hoteleiro. O sistema deve permitir a esta equipa a gestão de planos

vacinais, alergias e a monitorização de cuidados críticos e administração de medicação.

- **Receção:** Função responsável pelo atendimento ao cliente, gestão de reservas e *check-in/check-out*. Este perfil necessita de uma interface ágil para controlar a ocupação sazonal.
- **Clientes (Proprietários de Animais):** Indivíduos que confiam os seus animais ao hotel. São os beneficiários indiretos do sistema, que garante a segurança dos dados dos seus animais e a fiabilidade dos serviços contratados.
- **Fornecedores:** Entidades responsáveis pelo abastecimento de rações, produtos de higiene e material médico. O sistema poderá auxiliar na gestão de inventário e necessidades de reposição.

3.2. Análise de contexto e restrições

Como detalhado no Capítulo 1, o sistema "Au Au... Sou um cão!" operará num ambiente de forte flutuação sazonal e de prestação de cuidados integrados (alojamento e saúde veterinária). O projeto visa substituir a atual gestão manual por uma infraestrutura digital robusta, capaz de centralizar toda a informação operacional, clínica e financeira do hotel, garantindo a escalabilidade necessária para acompanhar o crescimento do negócio.

Para responder às necessidades operacionais e de engenharia, o sistema está sujeito às seguintes restrições:

- **Restrições Físicas e de Lotação:** O motor de reservas tem de otimizar a alocação e gerir a multi-ocupação, dado o limite estrutural de 40 quartos para responder a picos sazonais que alcança os 40 animais.
- **Restrições Clínicas e de Segurança:** O sistema deve bloquear automaticamente reservas ou *check-ins* de animais sem o plano vacinal atualizado e garantir alertas perfeitamente visíveis para alergias e necessidades de medicação.
- **Restrições Técnicas:** A especificação de requisitos tem de cumprir rigorosamente as diretrizes da norma IEEE 830/29148 para o documento SRS (*Software Requirements Specification*), e a base de dados tem de garantir a centralização e segurança da informação para uma correta geração de relatórios. – É mais objetivo do trabalho

3.3. Eliciação de Requisitos

Nesta eliciação usamos 2 métodos muito comuns na área: a observação do funcionamento do hotel e análise de um contrato feito pelo hotel com um dono.

3.3.1. Relatório de Observação Direta

- **Caos Comunicacional:** O staff utiliza um quadro branco na zona dos canis para marcar quem já comeu, o que pode levar um funcionário a se esquecer de uma medicação porque o marcador pode estar gasto e ilegível.
- **Gestão de Stress (*Check-in*):** No momento da entrada, o namorado da Diana perdeu 15 minutos a procurar a ficha física de um cliente antigo. O cão estava ansioso e a ladrar, o que aumentou a tensão no balcão.
- **Logística de Limpeza:** Existe uma rotina rígida. Se o *software* não indicar quais quartos estão "vazios e sujos" vs "vazios e prontos", a equipa perde tempo a verificar porta a porta.
- **O "Puzzle" da Vizinhaça:** Vimos a Diana pedir para trocar um Yorkshire de quarto porque o vizinho era um Pastor Alemão muito reativo. Essa troca não foi registada em lado nenhum, apenas feita no momento.

3.3.2. Leitura de Contrato

Em seguida pedimos a equipa da Diana para que nos deixasse examinar um contrato e retiramos a seguinte informação: cláusulas jurídicas em regras de negócio automáticas, garantindo que o sistema valide obrigatoriedades como o estado vacinal no check-in e aplique rigorosamente as taxas de cancelamento e serviços extra na faturação final.

3.4. Entrevistas com *stakeholders*

Esta secção detalha o processo de auscultação dos principais intervenientes no ecossistema do hotel "Au Au... Sou um Cão!". O objetivo foi capturar não apenas necessidades funcionais, mas a filosofia do negócio, as dificuldades humanas na gestão de grandes volumes de animais e as expectativas emocionais dos clientes.

3.4.1. Entrevista com a Fundadora e Gestora Técnica (Diana Silva)

A entrevista com a Diana Silva foi pautada pelo seu perfil pragmático e pela sua dupla responsabilidade: gestora de um negócio em rápida expansão e Médica Veterinária.

- **Contexto e Motivações da Criação:** A Diana partilhou que o hotel nasceu em 2021, num momento de incerteza laboral devido à pandemia. O "clique" surgiu da união entre uma necessidade pessoal (dificuldade em encontrar alojamento canino em Braga que cumprisse padrões clínicos rigorosos) e a observação de um aumento nas adoções durante o confinamento. A sua formação académica na UTAD moldou o negócio: o objetivo nunca foi ser apenas uma "creche", mas sim uma unidade de hotelaria com suporte clínico e etológico (estudo do comportamento animal).
- **A Evolução e o Desafio da Escala:** Inicialmente planeado para uma escala familiar de 40 quartos, o hotel rapidamente escalou para gerir 40 animais em épocas altas. A Diana descreveu esta transição como "caótica mas gratificante", sublinhando que o método atual de gestão (cadernos físicos e quadros brancos) atingiu o seu limite de segurança. A preocupação central da Diana não é apenas a organização, mas a rastreabilidade: saber exatamente quem medicou, o quê e quando, sem margem para erro humano.
- **Dores Críticas Identificadas:**
 1. **Maneio Clínico:** A gestão de animais com patologias crónicas (diabetes, epilepsia) exige um controlo que o papel não garante de forma imutável.
 2. **O "Puzzle" de Alocação:** A Diana explicou a complexidade de gerir a vizinhança entre cães reativos, seniores e de porte minúsculo, o que atualmente exige um esforço mental exaustivo da sua parte para evitar incidentes.
 3. **Fadiga Comunicacional:** A gestão de expectativas via *WhatsApp* foi identificada como o maior "ladrão de tempo", onde o staff gasta horas a enviar fotos individuais em vez de se focar no cuidado direto.

3.4.2. Entrevista com um Cliente

A entrevista com o cliente (representando tutores de animais com necessidades especiais) revelou o lado emocional e a confiança depositada na estrutura.

- **CrITÉRIOS de Fidelização:** O entrevistado reforçou que a principal razão para escolher este hotel em Braga é a paz de espírito conferida pela presença de uma

técnica de veterinária. Para este perfil de cliente, o hotel funciona como uma extensão da segurança de uma clínica.

- A Experiência do Utilizador e Frustrações: O cliente descreveu a experiência de reserva em época alta como "opaca". A necessidade de esperar por uma resposta manual via telefone ou mensagem gera ansiedade. Foi reportado que, durante o Verão, a comunicação diminui devido ao volume de trabalho da equipa, o que faz com que os donos se sintam "desconectados" do estado do animal.
- Expectativas de Interação:
 1. Transparência Digital: O cliente manifestou o desejo de consultar uma "Ficha de Saúde" atualizada durante a estadia, onde pudesse ver confirmações de que o cão comeu e tomou a medicação.
 2. Agilidade no *Check-in*: Foi descrita a frustração de chegar ao hotel com um animal agitado e ter de preencher papeis à mão ou repetir informações clínicas já fornecidas anteriormente. O cliente anseia por um processo onde, ao chegar, a equipa já conheça o histórico completo do animal através de um perfil digital prévio.
 3. Visualização de Serviços: Existe o desejo de "ver" os serviços extra (banhos, passeios) de forma visual e clara, evitando surpresas de faturação no momento do *check-out*.

3.4.3. Entrevista com um membro do Staff

De forma a complementar a análise de requisitos técnicos e administrativos, foi realizada uma entrevista semiestruturada a um membro da equipa de cuidados diários. O objetivo foi identificar as dificuldades práticas que o crescimento da ocupação (para 40 animais) impôs à operação. Os pontos principais extraídos da entrevista incluíram:

- Falha na Sincronização: A dependência de métodos analógicos (quadro branco com marcadores gastos) gera insegurança na transmissão de turnos.
- Sobrecarga Comunicacional: O staff despende entre 1 a 2 horas diárias no *WhatsApp* para envio individual de fotos, prejudicando o tempo de cuidado direto.
- Risco Clínico: A falta de alertas digitais para medicação crítica (epilepsia/diabetes) é identificada como a maior fonte de ansiedade e risco de erro humano.
- Gestão de Serviços: A ausência de registo digital imediato de banhos e passeios causa erros na faturação final e perda de receita.

3.4.4. Conclusões do Levantamento de Informação

As entrevistas demonstraram uma convergência crítica de interesses entre os três pilares do hotel: enquanto a Diana (Gestão) necessita de ferramentas de controlo e segurança clínica para gerir a carga de trabalho, e o Cliente (Utilizador) procura transparência e autonomia no acompanhamento do animal, o *Staff* (Operacional) carece urgentemente de mecanismos que eliminem a ambiguidade dos registos manuais e a sobrecarga de comunicação informal.

O ponto comum mais forte entre todos os intervenientes foi a necessidade imperativa de profissionalizar a operação, movendo o hotel de um modelo de "confiança baseada na memória" e em suportes físicos voláteis para um ecossistema digital de "confiança baseada em dados reais e registos auditáveis". Esta transição é vista não apenas como uma melhoria logística, mas como a única salvaguarda contra o erro humano perante a elevada densidade de animais em épocas altas

3.5. Requisitos funcionais e não funcionais

Com base nas diversas técnicas de eliciação aplicadas nas fases anteriores, incluindo a observação direta, a análise do contrato de prestação de serviços, as entrevistas estruturadas com os *stakeholders*, procedeu-se ao levantamento sistemático das necessidades do sistema. Esta secção apresenta a consolidação preliminar de todos os requisitos identificados, listados na tabela abaixo no seu estado bruto.

ID	Descrição do Requisito	Data	Fonte	Analista
R01	Motor de Reservas: Calendário em tempo real manipulável com as datas.	15/02/2026	Entrevista (Diana)	Diogo Pais
R02	Reservas Inteligentes: Gestão de <i>Overbooking</i> .	15/02/2026	Entrevista (Diana)	Diogo Pais
R03	Alocação por Perfil e Porte: Sugestão de quartos por reatividade e tamanho do animal.	15/02/2026	Entrevista (Diana)	Diogo Pais
R04	Políticas de Cancelamento: Taxas automáticas de retenção conforme antecedência.	15/02/2026	Entrevista (Diana)	Diogo Pais
R05	Validação de <i>Check-in</i> : Bloqueio de entrada sem vacinação.	15/02/2026	Entrevista (Diana)	Diogo Pais
R06	Validação de <i>Check-in</i> : Bloqueio sem contacto de emergência.	15/02/2026	Entrevista (Diana)	Diogo Pais

R07	Peak Pricing: Preços diferenciados automáticos para Verão e Natal(filtro por preço).	16/02/2026	Entrevista (Cliente)	Diogo Pais
R08	Lista de Espera Automática: Notificações de vagas para datas esgotadas.	16/02/2026	Entrevista (Cliente)	Diogo Pais
R09	Extensões de Estadia: Recálculo de fatura para atrasos imprevistos.	18/02/2026	Eliciação (Contrato)	Hugo Cunha
R10	<i>Check-in</i> Digital Antecipado: <i>Upload</i> de documentos e ficha clínica via portal.	16/02/2026	Entrevista (Cliente)	Diogo Pais
R11	OCR de Documentos: Validação automática de datas em boletins de vacinas.	16/02/2026	Entrevista (Cliente)	Diogo Pais
R12	Ficha Animal/Tutor: Registo de <i>microchip</i> .	15/02/2026	Entrevista (Diana)	Diogo Pais
R13	Ficha Animal: Necessidade de historial vacinal e alergias.	15/02/2026	Entrevista (Diana)	Diogo Pais
R14	Gestão de Medicação (<i>e-Prescription</i>): Alertas de posologia para o staff.	15/02/2026	Entrevista (Diana)	Diogo Pais
R15	Log de Administração Imutável: Registo digital de medicação não editável.	15/02/2026	Entrevista (Diana)	Diogo Pais
R16	Protocolo de Quarentena: Isolamento automático no sistema para sintomas suspeitos.	15/02/2026	Entrevista (Diana)	Diogo Pais
R17	Monitorização de Consumo: Registo de ingestão alimentar com alerta de inapetência.	15/02/2026	Entrevista (Diana)	Diogo Pais
R18	Alerta de inapetência: Quando um cão não come.	15/02/2026	Entrevista (Diana)	Diogo Pais
R19	<i>Stock</i> Clínico: Controlo de inventário de medicamentos e rações terapêuticas.	16/02/2026	Entrevista (Cliente)	Diogo Pais
R20	Histórico Exportável: Geração de relatórios de incidentes em PDF.	15/02/2026	Entrevista (Diana)	Diogo Pais
R21	<i>Checklist</i> Digital Móvel: Tarefas diárias (passeios/limpeza) em tempo real.	18/02/2026	Eliciação (Staff)	Hugo Cunha
R22	Preferências na UI: Exibição de tipo de trela e sensibilidade no ecrã da tarefa.	18/02/2026	Entrevista (Staff)	Hugo Cunha
R23	Módulo de Serviços Extra: Carrinho visual para Banhos e Treinos na reserva.	16/02/2026	Entrevista (Cliente)	Diogo Pais

R24	Rastreabilidade de Incidentes: Registo de intercorrências com fotos.	15/02/2026	Entrevista (Diana)	Diogo Pais
R25	Estado de Higienização (Boxes): Estados Suja/Em Limpeza/Higienizada.	18/02/2026	Entrevista (Staff)	Hugo Cunha
R26	Mapa de Alocação Interativo: Visualização gráfica e mudança de box.	18/02/2026	Entrevista (Staff)	Hugo Cunha
R27	Etiquetas de Dieta: Geração de etiquetas para taças.	18/02/2026	Eliciação (Operacional)	Hugo Cunha
R28	Centralização de <i>Media</i> : Upload único de fotos associadas ao animal.	18/02/2026	Eliciação (Staff)	Hugo Cunha
R29	Assinatura Digital: Termo de responsabilidade digital antecipado.	16/02/2026	Entrevista (Cliente)	Diogo Pais
R30	Pagamentos Integrados: Suporte a MB WAY e Referência Multibanco.	16/02/2026	Entrevista (Cliente)	Diogo Pais
R31	Faturação Certificada: Emissão automática conforme normas da AT.	15/02/2026	Entrevista (Diana)	Diogo Pais
R32	Diário de Bordo Digital: Notificações automáticas para os tutores.	15/02/2026	Entrevista (Diana)	Diogo Pais
R33	Portal do Cliente: Histórico de faturas e gastos acumulados.	16/02/2026	Entrevista (Cliente)	Diogo Pais
R34	Notificações Estruturadas: Alertas de medicação, refeição e media.	16/02/2026	Entrevista (Cliente)	Diogo Pais
R35	Alertas Críticos: Notificação imediata de sintomas graves.	16/02/2026	Entrevista (Cliente)	Diogo Pais
R36	<i>Dashboards</i> de Gestão: Relatórios de rentabilidade e serviços.	18/02/2026	Eliciação (Enunciado)	Hugo Cunha
R37	Encriptação (RGPD): Dados sensíveis protegidos via AES-256.	16/02/2026	Entrevista (Cliente)	Diogo Pais
R38	Controlo de Acessos (RBAC): Segregação entre staff e área financeira.	15/02/2026	Entrevista (Diana)	Diogo Pais
R39	2 <i>Factor Authentication</i> (2FA): Obrigatório para dados de saúde.	16/02/2026	Entrevista (Cliente)	Diogo Pais
R40	Auditabilidade: <i>Logs</i> de auditoria mantidos por 3 anos.	16/02/2026	Entrevista (Cliente)	Diogo Pais
R41	Conformidade Fiscal: Documentos com NIF/NIPC conforme AT.	20/02/2026	Eliciação (Legal)	Hugo Cunha
R42	Tempos de Resposta: Latência inferior a 2 segundos em operações críticas.	15/02/2026	Entrevista (Diana)	Diogo Pais

R43	Latência do Mapa: Atualização visual em menos de 1 segundo.	18/02/2026	Entrevista (Staff)	Hugo Cunha
R44	Escalabilidade de Pico: Suporte a 500+ acessos simultâneos.	16/02/2026	Entrevista (Cliente)	Diogo Pais
R45	Disponibilidade (Uptime): Garantia de 99.9% de uptime.	15/02/2026	Entrevista (Diana)	Diogo Pais
R46	Recuperação de Desastres: <i>Backups</i> com RPO < 1h e RTO < 4h.	15/02/2026	Entrevista (Diana)	Diogo Pais
R47	Modo Offline: Registo em zonas sem rede com sincronização posterior.	18/02/2026	Eliciação (Staff)	Hugo Cunha
R48	Dupla Confirmação: Verificação extra para fármacos de alta toxicidade.	18/02/2026	Entrevista (Staff)	Hugo Cunha
R49	Interface <i>One-Hand: Design</i> otimizado para uso com uma só mão.	18/02/2026	Entrevista (Staff)	Hugo Cunha
R50	Alta Visibilidade: Ícones grandes para ambientes de canil.	18/02/2026	Entrevista (Staff)	Hugo Cunha
R51	Redução de Carga Cognitiva: Sinalética visual para processos rápidos.	18/02/2026	Entrevista (Staff)	Hugo Cunha
R52	UX para Tutores: Navegação intuitiva otimizada para smartphones.	16/02/2026	Entrevista (Cliente)	Diogo Pais
R53	Exportação de Logs: Logs exportáveis em CSV/PDF para reconciliação.	18/02/2026	Entrevista (Staff)	Hugo Cunha
R54	Localização: Suporte integral a PT-PT e Moeda EUR.	16/02/2026	Entrevista (Cliente)	Diogo Pais
R55	Interface Simples: O sistema deve ser muito intuitivo e fácil de usar para não perder tempo.	15/02/2026	Entrevista (Diana)	Diogo Pais
R56	Segurança Adequada: Devem ser aplicadas as medidas de segurança normais para proteger os dados dos clientes.	20/02/2026	Eliciação (Legal)	Hugo Cunha
R57	Necessidade de Página Pública com descrição de serviço, galeria de fotos das instalações e “Sobre Nós”.	28/02/2026	Eliciação (Outros)	Hugo Cunha
R58	Comentários positivos deixados por outros donos.	28/02/2026	Eliciação (Outros)	Hugo Cunha

Tabela 2: Tabela de requisitos inicial.

É importante notar que esta listagem representa o *input* inicial da fase de análise, contendo ainda duplicações, ambiguidades e requisitos pendentes de validação técnica, que serão alvo de tratamento e limpeza na secção de refinamento que se segue.

3.6. Refinamento de requisitos ambíguos

Nesta fase, procedeu-se à análise crítica dos requisitos eliciados para remover subjetividades. Requisitos como "rápido" ou "simples" foram convertidos em métricas verificáveis conforme a norma IEEE 830/29148.

A necessidade de refinar os requisitos R48, R55 e R56 surgiu do facto de estes terem sido inicialmente capturados como expressões qualitativas e subjetivas durante as sessões de eliciação, onde os *stakeholders* se focaram na experiência de utilização e em conceitos genéricos de conformidade legal, sem definir métricas técnicas ou protocolos verificáveis.

Para mitigar esta ambiguidade e permitir uma validação objetiva, a equipa procedeu à conversão destes conceitos em valores quantitativos e normas técnicas rigorosas:

- O requisito de "Interface Simples" (R55) foi traduzido no seguinte requisito ergonomia *One-Hand* (RNF.13), pois este é o que melhor se adapta e após mais interpretação da entrevista onde foi retirado era neste contexto que a Diana se referia.
- O requisito de "Segurança Adequada" (R56) foi detalhado através da especificação de protocolos de encriptação AES-256 (RNF.01), obrigatoriedade de 2FA (RNF.03) e cumprimento estrito das normas do RGPD (RNF.01).
- O requisito de "Dupla Confirmação" (R48) foi de certa maneira traduzido no "2 Factor" (R39) pois para entrarmos no modo de veterinário antes temos que passar por este teste logo testar 2 vezes a mesma coisa seria fútil.

Esta adaptação transformou intenções subjetivas em especificações de engenharia mensuráveis, assegurando que o sistema cumpra os padrões de qualidade e segurança exigidos num contexto profissional.

3.6.1. Requisitos Funcionais

Novo ID	Nome do Requisito	ID Original	Descrição da Função Específica
RF.01	Calendário de Disponibilidade	R01	Visualização de datas livres e ocupadas por box em tempo real.

RF.02	Validação de Conflitos	de	R01	Impedir tecnicamente a sobreposição de datas para o mesmo espaço.
RF.03	Bloqueio de <i>Overbooking</i>	de	R02	Impedir reservas quando a capacidade máxima do hotel for atingida.
RF.04	Alerta Ocupação Crítica	de	R02	Notificar a gerência quando a ocupação atingir 90%.
RF.05	Alocação Espécie	por	R03	Restringir boxes específicas apenas para gatos ou apenas para cães.
RF.06	Alocação Perfil	por	R03	Sugerir boxes com base no comportamento.
RF.07	Cálculo Reembolso	de	R04	Cálculo automático do valor a devolver com base na data do cancelamento.
RF.08	Emissão de Vale/Crédito	de	R04	Opção de converter reembolso em crédito para estadias futuras.
RF.09	Validação de Vacinação	de	R05	Bloquear o check-in se as vacinas obrigatórias estiverem fora de validade.
RF.10	Contactos de Emergência	de	R06	Obrigatoriedade de registo de um contacto alternativo no <i>check-in</i> .
RF.11	Tabela de Preços Dinâmica	de	R07	Definição automática de preços diferenciados para épocas altas/feriados.
RF.12	Gestão de Fila de Espera	de	R08	Registo de clientes interessados em datas que já estão lotadas.
RF.13	Notificação de Vaga	de	R08	Envio automático de alerta quando ocorre uma desistência na fila.
RF.14	Verificação de Prolongamento	de	R01	Checar disponibilidade da box para estender a estadia atual.
RF.15	Recálculo Estadia	de	R09	Atualizar o valor total da fatura e emitir suplemento após extensão.
RF.16	<i>Upload</i> Documentos	de	R10	Funcionalidade para o tutor carregar PDF/Foto da ficha clínica/boletim.
RF.17	Leitura OCR de Boletins	de	R11	Extração automática de dados das fotos dos boletins de vacinas.
RF.18	Registo de <i>Microchip</i>	de	R12	Campo obrigatório para o identificador eletrónico do animal.
RF.19	Historial Saúde	de	R13	Registo de alergias, condições crónicas e reações passadas.
RF.20	Horários de Medicação	de	R14	Sistema de alertas visuais para as horas exatas dos remédios.

RF.21	Registo de Dosagem	R14	Campo para confirmar a dose exata administrada.
RF.22	Log de Administração	R15	Registo imutável (<i>timestamp</i>) de QUEM deu a medicação.
RF.23	Sinalética de Quarentena	R16	Alerta visual permanente no sistema para animais em isolamento.
RF.24	Registo de Alimentação	R17	Marcar se o animal comeu a dose total, parcial ou se houve recusa.
RF.25	Alerta de Inapetência	R18	Notificação automática se o animal falhar 2 refeições seguidas.
RF.26	Inventário Clínico	R19	Contagem e alerta de <i>stock</i> baixo de medicamentos, curativos e rações terapêuticas.
RF.27	Relatórios para Seguradoras	R20	Exportação de histórico de incidentes em formato PDF oficial.
RF.28	Checklist Móvel de Tarefas	R21	Lista diária para funcionários marcarem tarefas concluídas via tablet.
RF.29	Agendamento de Serviços Extra	R23	Marcação de banhos, tosas ou treinos durante a estadia.
RF.30	Registo de Incidentes com Foto	R24	Documentar ferimentos ou comportamentos anormais com evidência visual.
RF.31	Estado de Limpeza de Boxes	R25	Atualização do estado da box: Livre, Suja ou Higienizada.
RF.32	Mapa de Alocação Visual	R26	Planta interativa com a localização e foto de cada animal no hotel.
RF.33	Impressão de Etiquetas	R27	Gerar etiquetas para taças de comida com nome e restrições alimentares.
RF.34	Galeria de Média Diária	R28	<i>Upload</i> de fotos e vídeos do "dia-a-dia" para o dono.
RF.35	Assinatura Digital	R29	Colha prévia da assinatura do tutor no ecrã para termos de responsabilidade.
RF.36	Faturação Certificada (AT)	R31	Integração com software certificado para emissão de faturas válidas.
RF.37	Timeline do Tutor	R32	Feed estilo "rede social" para o dono acompanhar o progresso do animal.
RF.38	Portal Financeiro Cliente	R33	Área para o tutor descarregar faturas e consultar histórico de pagamentos.

RF.39	Notificações <i>Push</i>	R34	Envio de mensagens automáticas para o telemóvel do tutor (APP).
RF.40	Alerta de Emergência Crítica	R35	Notificação de alta prioridade para incidentes de saúde graves.
RF.41	<i>Dashboard</i> de Gestão	R36	Gráficos de ocupação e rentabilidade mensal para a administração.
RF.42	Página Pública	R57	Necessidade de Página Pública com descrição de serviço, galeria de fotos das instalações e “Sobre Nós”.
RF.43	Partilha de Experiência	R58	Espaço com comentários positivos deixados por outros donos.

Tabela 3: Tabela de requisitos funcionais.

3.6.2. Requisitos Não Funcionais

Novo ID	Nome do Requisito	ID Original	Descrição da Função Específica
RNF.01	Encriptação de Dados	R37	Proteção de dados sensíveis (clínicos e tutores) via algoritmo AES-256.
RNF.02	Controlo de Acessos (RBAC)	R38	Segregação de permissões entre staff operacional e área financeira/gestão.
RNF.03	Autenticação Two Factor	R39	Obrigatoriedade de 2FA (ex: código SMS/Email) para aceder a dados de saúde.
RNF.04	Retenção de Auditoria	R40	Manutenção de logs de atividade do sistema por um período mínimo de 3 anos.
RNF.05	Conformidade Fiscal	R41	Garantia de que todos os documentos emitidos cumprem as normas da AT (NIF/NIPC).
RNF.06	Latência de Operações	R42	Tempo de resposta inferior a 2 segundos em operações críticas (ex: check-in).
RNF.07	Performance do Mapa	R43	Atualização visual da planta interativa em menos de 1 segundo após alteração.
RNF.08	Escalabilidade de Pico	R44	Suporte para mais de 300 acessos simultâneos em épocas de abertura de reservas.
RNF.09	Disponibilidade (<i>Uptime</i>)	R45	Garantia de funcionamento do sistema de 99.9% do tempo anual.

RNF.10	Recuperação de Dados	R46	Política de backups com perda máxima de 1h (RPO) e recuperação em 4h (RTO).
RNF.11	Sincronização <i>Offline</i>	R47	Permitir registo de tarefas em zonas sem rede com sincronização automática posterior.
RNF.12	Ergonomia <i>One-Hand</i>	R49	Interface otimizada para ser operada com apenas um polegar (<i>staff</i> em campo).
RNF.13	Adaptabilidade Visual	R50	Ícones e botões de grandes dimensões para facilitar o uso em ambientes de canil.
RNF.14	Sinalética Cognitiva	R51	Uso de cores e símbolos para reduzir o esforço mental na leitura de estados.
RNF.15	Otimização <i>Mobile</i>	R52	Interface responsiva e fluida para os tutores acederem via <i>smartphone</i> .
RNF.16	Exportação de <i>Logs</i>	R53	Capacidade de exportar dados de auditoria em formatos CSV ou PDF para reuniões.
RNF.17	Localização Regional	R54	Suporte integral à norma PT-PT, incluindo fuso horário, moeda EUR e datas.
RNF.18	Facilidade de Aprendizagem	R55	Design intuitivo que dispense formação técnica avançada para a proprietária.
RNF.19	<i>Standards</i> de Segurança	R56	Aplicação de normas de segurança padrão TLS 1.2 (pois é o usado normalmente) para tráfego <i>web</i> .

Tabela 4: Tabela de requisitos não funcionais.

3.7. Geração de user stories

A técnica de *User Stories* foi adotada para transpor as necessidades qualitativas dos *stakeholders* para especificações técnicas claras. As histórias foram estruturadas para garantir a transição de um modelo de gestão manual para uma infraestrutura tecnológica baseada em dados, segurança clínica e transparência.

As *User Stories* encontram-se organizadas segundo o perfil de cada *stakeholder* central do ecossistema do hotel "Au Au... Sou um Cão!".

3.7.1. Diana Gonçalves Silva (Veterinária e Gestora)

- **US01:** Enquanto **Gestora**, quero que o sistema sugira a alocação de quartos com base no perfil comportamento, para eliminar o risco de incidentes e o esforço mental associado ao planeamento manual.

- O sistema deve sugerir os quartos denominados para os cães mais reativos ;
- **US02:** Enquanto **Gestora**, quero definir regras de "overbooking controlado" e utilização de camas extra para épocas altas, para maximizar a rentabilidade do espaço sem comprometer as normas de segurança.
 - O sistema deve permitir reservas até ao limite de quartos mais o limite de camas extra.
- **US03:** Enquanto **Veterinária**, quero prescrever planos de medicação digitais, para que o sistema emita alertas sonoros e visuais obrigatórios para o *staff*, eliminando o risco de falhas ao tomar as medicações.
 - *Login* com sucesso através de um perfil com permissões clínicas.
 - O sistema deve permitir inserir o nome do medicamento, a dose e o horário exato.
 - O sistema deve disparar um alerta visual nos dispositivos do *staff* 15 minutos antes da hora estipulada.
 - O alerta só pode ser desativado após o registo de administração por parte do cuidador.
- **US04:** Enquanto **Veterinária**, quero sinalizar no sistema animais com sintomas suspeitos, para ativar um protocolo de quarentena que restrinja automaticamente o seu acesso a zonas comuns.
 - O perfil de veterinário pode alterar o estado de saúde do animal para "Quarentena".
 - Ao ativar a quarentena, o sistema deve remover automaticamente o animal da lista dos animais saudáveis.
 - O painel do *staff* deve mostrar um alerta visível na ficha do animal em isolamento.

3.7.2. Clientes (Tutores de Animais)

- **US05:** Enquanto **Tutor**, quero consultar a disponibilidade real do hotel num calendário dinâmico e efetuar reservas de forma *self-service*, para não depender de tempos de resposta via mensagem ou telefone.
 - O calendário deve bloquear datas em que a lotação máxima (quartos + camas extra) foi atingida.
 - O utilizador deve conseguir selecionar datas de entrada e saída, recebendo uma confirmação instantânea no ecrã.

- **US06:** Enquanto **Tutor**, quero preencher a ficha clínica online e realizar o *upload* antecipado de boletins de vacinas, para que o processo de *check-in* presencial seja isento de erros e concluído rapidamente.
 - O formulário online deve aceitar ficheiros em formato PDF ou imagem (JPG/PNG) para as vacinas.
 - O formulário deve ter campos obrigatórios preenchidos (alergias, contactos de emergência) antes da submissão.
 - Os dados submetidos devem ficar imediatamente associados ao perfil do animal na base de dados.

- **US07:** Enquanto **Tutor** de um cão com necessidades médicas, quero receber notificações com *timestamp* exato da administração da medicação, para mitigar a minha ansiedade em relação ao cumprimento do plano de saúde.
 - Assim que o cuidador regista que deu a medicação no sistema, deve ser gerado um registo inalterável de data e hora (*timestamp*).
 - O sistema deve enviar imediatamente uma notificação para a aplicação/perfil do tutor indicando o sucesso da ação.

- **US08:** Enquanto **Tutor**, quero seleccionar serviços extra (como banhos, passeios ou consultas) através de um menu visual durante a reserva, para personalizar a estadia e conhecer o orçamento total antecipadamente.
 - O processo de reserva deve incluir uma etapa de "Carrinho de Serviços Extras".
 - O valor total da reserva deve ser atualizado em tempo real na interface sempre que um extra é adicionado ou removido.

3.7.3. Staff (Cuidadores e Operacionais)

- **US09:** Enquanto **Cuidador**, quero aceder a uma *checklist* diária num dispositivo móvel para registar a conclusão de tarefas (passeios, alimentação) de forma rápida num ambiente agitado.
 - A interface deve ser responsiva e otimizada para uso móvel.
 - O estado da tarefa deve ser atualizado instantaneamente no painel geral de operações.

3.7.4. Recepção e Administrativo

- **US10:** Enquanto **Rececionista**, quero que o sistema bloqueie automaticamente a entrada de animais cujo boletim de vacinas não esteja validado ou atualizado, para assegurar a estrita conformidade sanitária das instalações.
 - Ao iniciar o processo de *check-in*, o sistema verifica a validade das vacinas no perfil do animal.
 - Se a vacina estiver caducada, o botão de "Confirmar Check-in" deve ser desativado.
 - O sistema deve exibir uma mensagem clara de erro exigindo a regularização do documento.
- **US11:** Enquanto **Rececionista**, quero que o sistema compile automaticamente todos os custos associados a uma reserva (estadias e extras) numa fatura certificada, para não ter de usar múltiplas plataformas ou calculadoras no momento do *check-out* do animal.
 - O sistema deve calcular automaticamente o valor total, distinguindo tarifas de época baixa/alta e somando os serviços extra consumidos.
 - O documento gerado tem de incluir os dados fiscais válidos para certificação da Autoridade Tributária.
 - A fatura final deve ser exportável em PDF e enviada para o e-mail do cliente a partir do ecrã de *check-out*.

3.8. Casos de Uso

Com base nos requisitos eliciados e no objetivo de manter o projeto "direto ao ponto" (focado no *core business* do hotel), procedeu-se a uma simplificação do Diagrama de Casos de Uso. Esta decisão estratégica de engenharia visa garantir que o sistema se foca nas operações críticas de segurança clínica e gestão de ocupação, omitindo processos secundários ou meramente administrativos que seriam redundantes na arquitetura global.

Descrição dos Casos de Uso Implementados

1. **Efetuar *Login*:** Ponto de acesso centralizado que garante a autenticação e segurança de todos os utilizadores, segmentando as permissões conforme o perfil (Diana, *Staff*, Recepção ou Cliente).

2. **Efetuar Reserva e Alocação:** Permite ao cliente reservar estadias, ativando o motor de alocação que gere o "puzzle" de vizinhança entre cães reativos e sociáveis para otimizar os 30 quartos disponíveis.
3. **Efetuar Check-In:** Registo formal da entrada do animal no hotel pela receção, desencadeando a mudança de estado da reserva para "Hospedado".
4. **Validar Plano Vacinal (<<include>>):** Verificação obrigatória e automática (via OCR ou manual) da conformidade sanitária do animal no momento da entrada, bloqueando o *check-in* se as vacinas estiverem caducadas.
5. **Efetuar Check-Out:** Processo de saída do animal gerido pela receção, que liberta fisicamente o espaço no mapa de ocupação e encerra a estadia.
6. **Administrar Medicação e Log:** Registo clínico imutável efetuado pela veterinária ou *staff*, onde se confirma a dose e a hora da medicação, gerando um histórico auditável e seguro.
7. **Executar Checklist Diário:** Gestão móvel das rotinas operacionais do *staff* (passeios, alimentação e limpezas), garantindo que nenhum serviço contratado seja esquecido no canil.
8. **Atualizar Diário de Bordo:** Funcionalidade para o *staff* carregar fotografias e estados de espírito do animal, reduzindo a fadiga de comunicação manual via *WhatsApp*.
9. **Visualizar Diário de Bordo:** Permite ao cliente acompanhar a rotina do seu animal em tempo real através de uma *timeline* digital, mitigando a ansiedade da separação.
10. **Gerir Diário de Bordo:** Área onde o cliente pode configurar preferências de partilha ou interagir com as atualizações enviadas pelo hotel.
11. **Ativar modo de quarentena:** Protocolo de isolamento imediato ativado pela gestora/veterinária em caso de suspeita de doença, restringindo automaticamente o acesso do animal a áreas comuns no sistema.
12. **Gerir Stock:** Controlo centralizado do inventário de medicamentos críticos e rações terapêuticas, com alertas de rutura para garantir a continuidade dos tratamentos.
13. **Gerir Faturação:** Consolidação automática de diárias (incluindo *peak pricing*) e serviços extra para a emissão de faturas certificadas em conformidade com a Autoridade Tributária
14. **Criar Conta:** Criação de conta obrigatório para fazer qualquer ação enquanto utilizador.

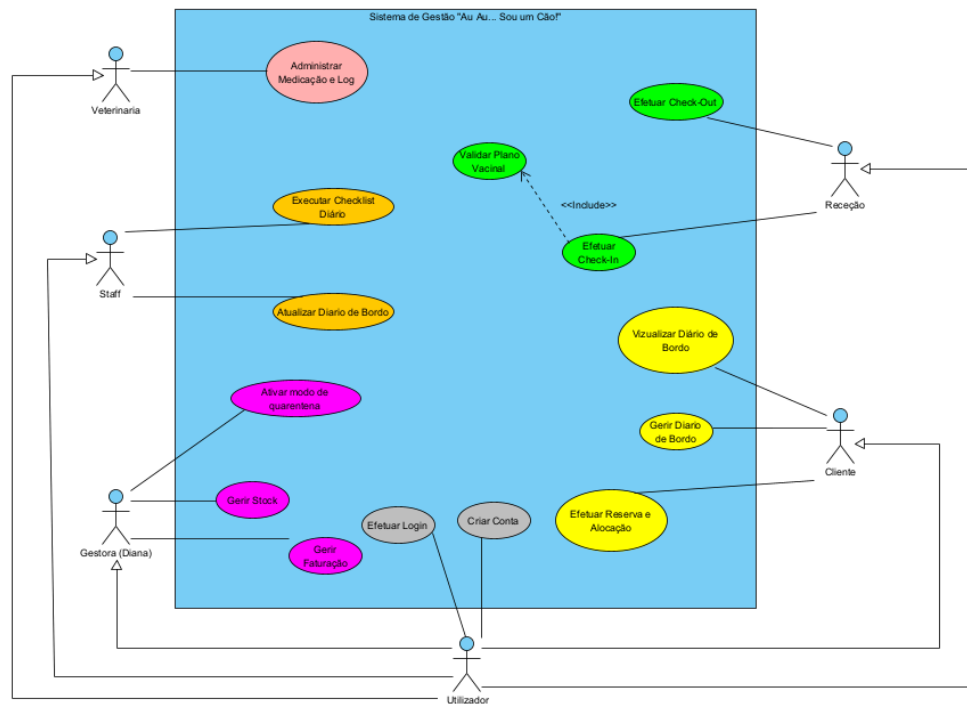


Figura 14: Diagrama de Use Cases.

Para manter a fluidez de leitura do relatório, a especificação exhaustiva dos Casos de Uso (atores, condições e fluxos) foi centralizada num documento anexo. Pode ser consultada na íntegra na folha de cálculo do projeto: “  UC's -LI4 ”.

3.9. Diagramas de Atividades

Para complementar a especificação estrutural dos Casos de Uso, modelou-se o comportamento do sistema através de Diagramas de Atividade. Estes diagramas ilustram os processos de negócio do hotel canino, mapeando o fluxo de controlo e as sequências de ações.

Através da utilização de partições (*swimlanes*), delimitou-se de forma clara as responsabilidades entre os intervenientes (Atores e Sistema) ao longo de fluxos normais, alternativos e de exceção. Para demonstrar esta dinâmica, foram detalhadas três operações críticas:

- Efetuar Reserva e Alocação
- Executar *Checklist*
- Visualizar Diário de Bordo

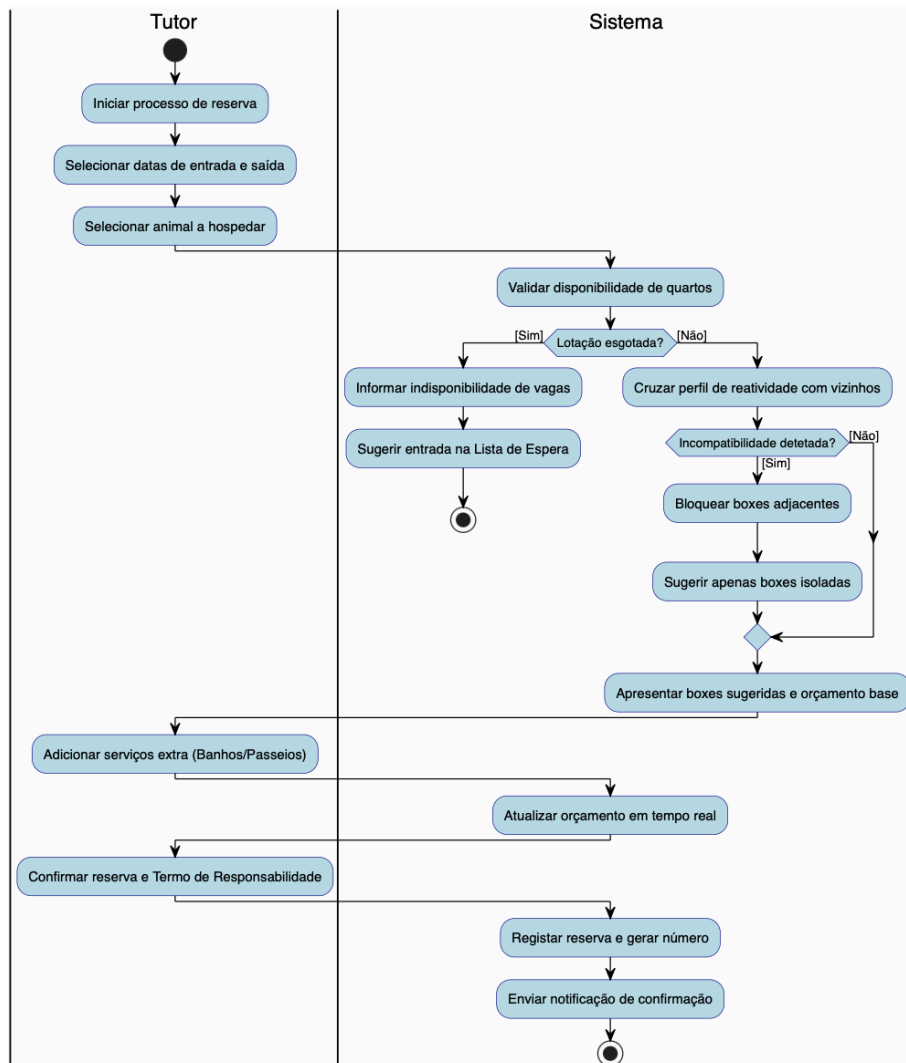


Figura 15: Diagrama de Atividade - Efetuar Reserva e Alocação.

Este diagrama ilustra o processo de negócio associado à reserva de alojamento e alocação espacial. O fluxo evidencia a interação contínua entre o planejamento do cliente e a validação em tempo real, utilizando partições para separar a ação humana do Tutor do processamento automático do Sistema. Destacam-se dois nós de decisão cruciais: o primeiro valida a disponibilidade da infraestrutura, despoletando um fluxo excecional de sugestão para a Lista de Espera em caso de lotação esgotada; o segundo atua como um mecanismo de segurança comportamental, avaliando o perfil de reatividade do animal para garantir o bloqueio dinâmico de boxes adjacentes (fluxo alternativo), assegurando o bem-estar dos hóspedes de forma preventiva antes da apresentação do orçamento base e do fecho da reserva.

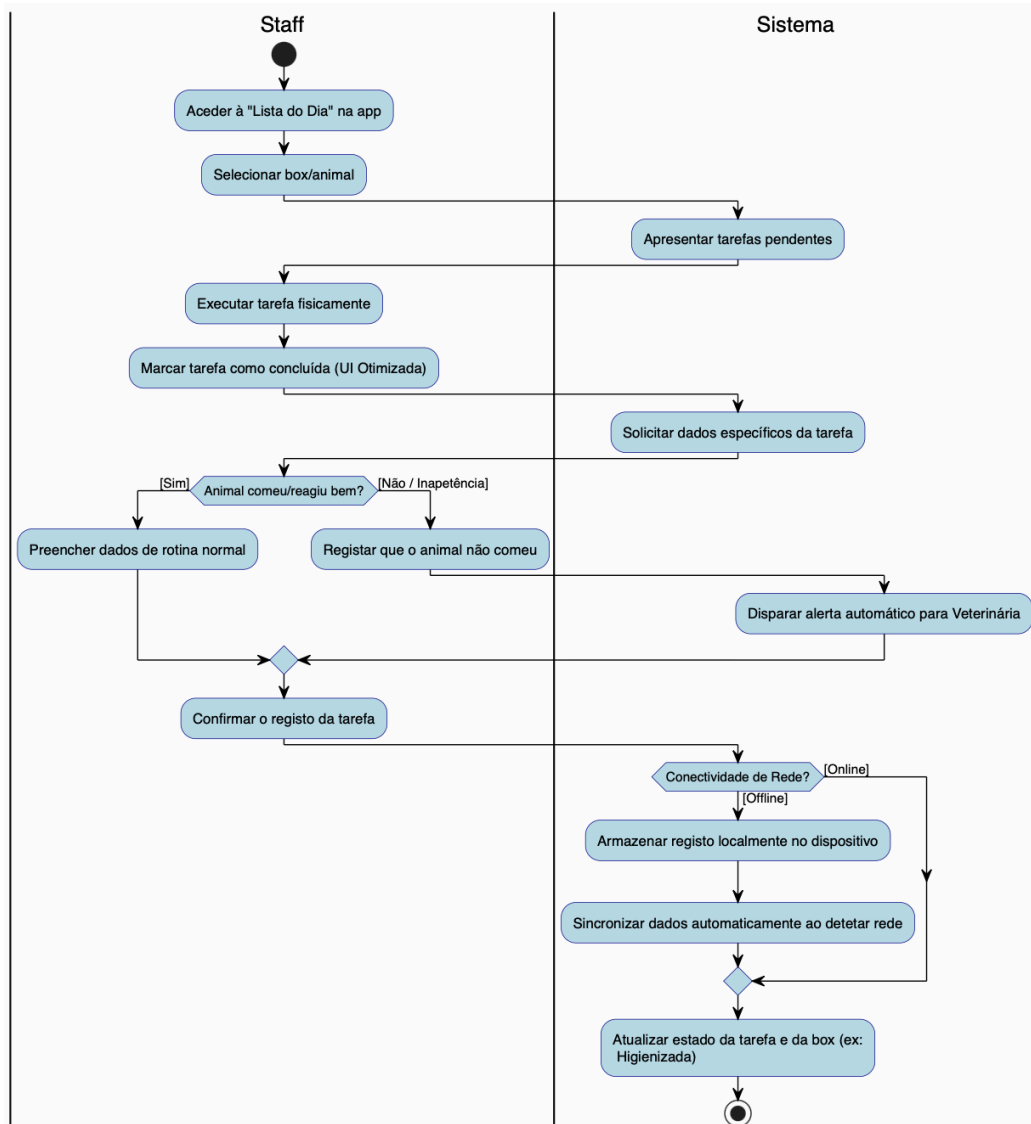


Figura 16: Diagrama de Atividade - Executar Checklist Diário.

Este diagrama ilustra a rotina operacional do Staff no terreno. O fluxo evidencia a interação contínua entre a execução física e o registo digital, utilizando partições para separar a ação humana do processamento do Sistema. Destacam-se dois nós de decisão cruciais: o primeiro valida o bem-estar do animal, despoletando um fluxo alternativo de alerta veterinário em caso de inapetência; o segundo atua como um mecanismo de tolerância a falhas, garantindo que a submissão dos dados não se perde em cenários de quebra de conectividade no canil (fluxo excecional *offline*), armazenando a informação localmente até à sincronização final do estado da tarefa e da box.

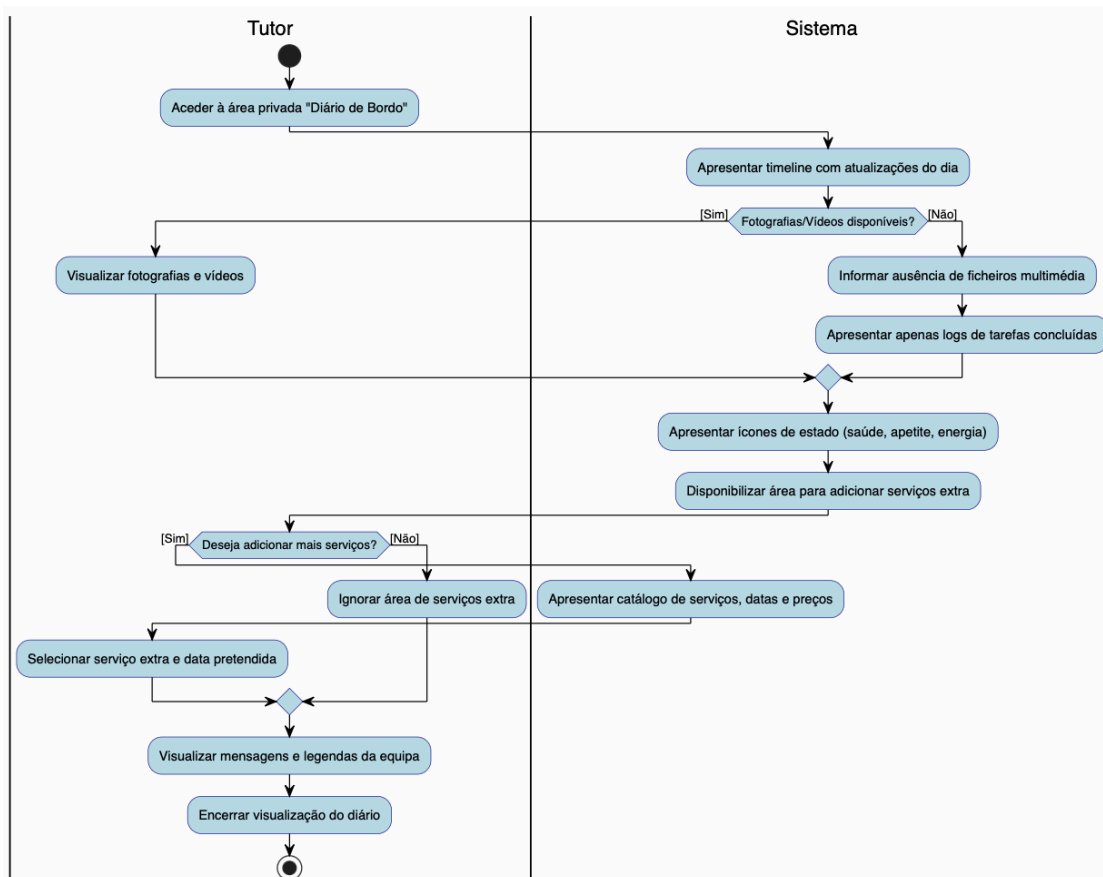


Figura 17: Diagrama de Atividade - Visualizar Diário de Bordo.

Este diagrama ilustra o processo de negócio associado ao acompanhamento da estadia e consulta do bem-estar do animal. O fluxo evidencia a interação entre a navegação do Tutor e a apresentação dinâmica de dados pelo Sistema, recorrendo a partições para delimitar as respetivas responsabilidades operacionais. Destacam-se dois nós de decisão que conferem flexibilidade ao processo: o primeiro adapta a interface gráfica consoante a disponibilidade de ficheiros multimédia (despoletando um fluxo alternativo de contingência textual baseada em *logs*); o segundo atua de forma estratégica, permitindo ao utilizador divergir do fluxo normal de leitura para explorar e adicionar novos serviços à estadia antes de concluir a sua consulta diária.

3.10. Prompts e geração dos resultados

3.10.1. Simulação de entrevista

3.10.1.1. Prompts Iniciais

"Age como Diana Gonçalves Silva, fundadora do hotel 'Au Au... Sou um Cão!'. És estudante de Veterinária, pragmática e focada no bem-estar animal. O teu negócio cresceu muito e sentes-te sobrecarregada com processos manuais. Responde às perguntas do guião de entrevista (abaixo) fornecendo detalhes realistas sobre: a gestão de 20-30 quartos , a equipa (namorado e 2 funcionários) , e a dificuldade em gerir picos de 120 animais. Sê específica sobre medicação, alimentação e faturação."

Bloco A: Génese, Motivação e Objetivos (O Passado e a Visão)

O Insight Inicial: Recuando ao momento da criação, qual foi o "clique" para transformar a dificuldade de emprego na pandemia num hotel canino? Foi uma necessidade pessoal com os teus próprios cães ou algo que observaste no mercado de Braga?

Missão e Valores: Quando abriste as portas, qual era o objetivo principal? Era apenas um negócio de sobrevivência ou já existia a ambição de criar um padrão de cuidados diferenciado pela tua formação em Veterinária?

Evolução do Conceito: O hotel começou com a estrutura que tem hoje ou foi crescendo organicamente? Como é que o teu percurso académico em Medicina Veterinária moldou os serviços que decidiste oferecer desde o primeiro dia?

Superação de Expectativas: Alguma vez imaginaste, no início, que passarias de 10 para 40 animais em épocas altas? Como é que essa escala alterou os teus objetivos iniciais para o negócio?

Bloco B: Engenharia de Requisitos (Operacional e Técnico)

Gestão de Inventário e Saúde: Como controlas atualmente o *stock* de medicamentos e rações específicas para cães com patologias? O sistema deve emitir alertas de *stock* baixo ou de validade de medicamentos?

Rastreabilidade de Incidentes: Se um cão se aleija ou apresenta sintomas durante a estadia, como é feito o registo hoje? Precisas que o *software* gere um "histórico de incidentes" que possa ser exportado para o dono ou para a seguradora?

Regras de Alocação (Quartos): Tens regras de "vizinhança" (ex: não colocar cães reativos ao lado de outros, separar por porte ou idade)? O sistema deve sugerir o quarto ideal com base no perfil do animal?

Faturação Complexa: Como lidas com cancelamentos de última hora ou extensões de estadia não planeadas? O sistema deve calcular automaticamente multas ou descontos por períodos longos?

Comunicação com o Dono: Atualmente perdes muito tempo a enviar fotos/*updates* por *WhatsApp*? Gostarias que o sistema tivesse uma funcionalidade de "Diário de Bordo" onde o *staff* faz upload de uma foto e o dono recebe uma notificação?

Segurança e Permissões: O que é que o *Staff* pode ver/editar em comparação com a Receção? (Ex: O *staff* deve ver dados financeiros ou apenas a ficha clínica e de alimentação?)

3.10.1.2. Avaliação Crítica do Resultado Inicial e Refinamento

Avaliação Crítica do Resultado	Refinamento Aplicado / Ação Tomada
Qualidade e Coerência: A qualidade técnica e a coerência narrativa das entrevistas simuladas revelaram-se altamente satisfatórias e consistentes com o domínio do negócio.	Sumarização: Como o resultado foi muito positivo, o único processamento adicional solicitado à IA consistiu na sumarização de cada interação simulada para integração no Capítulo 3 do relatório.
Validade Estrutural: O modelo de instrução original demonstrou ser uma base estrutural (template) sólida para simular outras entrevistas.	Reutilização com Adaptações Pontuais: A instrução base foi reutilizada com adaptações pontuais no contexto e nas questões específicas para simular os restantes intervenientes (staff, tutores), recorrendo a uma estratégia de meta-prompting

Tabela 5: *Prompts da simulação de entrevistas.*

3.10.2. Simulação de eliciação

3.10.2.1. Prompt Inicial

"Eliciação de Requisitos: Nesta eliciação usamos 2 métodos muito comuns na área: a observação do funcionamento do hotel e análise de um contrato feito pelo hotel com um dono"

Agora eu quero que tu uses teor acadêmico e que me ajudes a executar esses 2 passos eu quero que tires requisitos funcionais e não funcionais tendo em conta a lógica usada no livro "*Software Engineering, Tenth Edition*", no que toca ao contrato faz a simulação de um contrato em pdf do hotel e do cliente para uma estadia de 5 dias com tosquia e banho, tendo em conta algum exemplo que encontres online fiável. Eu quero que tu uses os teus poderes de imaginação para fingires que estavas lá a ver a equipa a trabalhar e entenderes o que que cada um faz, o contrato tá mais ou menos só queria que fosses mais direto e pusesses dinheiro nomes, NIFS e outros."

3.10.2.2. Avaliação Crítica do Resultado Inicial e Refinamento

Avaliação Crítica do Resultado	Refinamento Aplicado / Ação Tomada
Respostas Iniciais Desajustadas: As primeiras iterações do prompt geraram respostas fora do âmbito operacional pretendido.	Abandono do Meta-prompting: A equipa alterou a estratégia, abandonando o meta-prompting e elaborando uma instrução de forma direta (o prompt final).
Elevada Precisão e Realismo: O prompt final formulado diretamente conseguiu captar com elevado grau de precisão a simulação de um dia de trabalho no hotel canino.	Manutenção do Resultado Inicial: Face à pertinência do conteúdo gerado por esta nova abordagem, não houve necessidade de refinar mais as instruções para a IA.
Qualidade Técnica: O conteúdo apresentou-se totalmente dentro do âmbito pretendido e com a pertinência técnica exigida.	Ajustes de Formatação: A intervenção manual limitou-se a correções estéticas e de apresentação visual dos dados, garantindo o alinhamento com as normas do relatório.

Tabela 5: Prompts da simulação de eliciações.

3.10.3. Levantamento de Requisitos

3.10.3.1. Prompt Inicial

“Requisitos - Tendo em conta as entrevistas os requisitos que falamos até agora as eliciações faz me uma tabela com todos os requisitos por ordem organizados da seguinte maneira:

Número do requisito(não faças discriminacao entre funcional e não funcional)

Descrição

Data

Fonte (Diana, Funcionários, etc)

Analista(Diogo Pais para entrevista e Hugo Cunha para os de eliciação)

Tenta manter o rigor que tiveste até agora. “

3.10.3.2. Avaliação Crítica do Resultado Inicial e Refinamento

Avaliação Crítica do Resultado	Refinamento Aplicado / Ação Tomada
Descontextualização de Requisitos: O modelo gerou propostas de requisitos que se revelaram incongruentes ou desajustadas face à lógica de negócio e ao trabalho já consolidado pela equipa até àquela fase do projeto.	Fornecimento de mais contexto: Para evitar requisitos genéricos ou sem sentido, a equipa foi muito mais explícita nas restrições. Obrigou-se o modelo a basear-se apenas nas informações previamente recolhidas (entrevistas, observação e contrato), com indicação expressa para não inventar funcionalidades que não resolvessem os problemas reais do hotel.
Geração Parcial de Dados: Verificou-se uma tendência do modelo para produzir respostas incompletas, limitando-se a apresentar exemplos ilustrativos em vez de realizar o levantamento exaustivo de todos os requisitos solicitados. v	Exigência de todos os dados: A instrução foi alterada para dar uma ordem mais clara, exigindo que a IA extraísse a totalidade dos requisitos. Utilizaram-se comandos diretos e imperativos como "não resumas a resposta" e "não deixes nenhuma informação de fora".

Estruturação Inadequada da Informação: O output apresentou uma segregação não solicitada e indesejada entre requisitos funcionais e não funcionais, impondo uma formatação errada e não alinhada com as diretrizes.	Controlo rígido de formatação: Forneceu-se a estrutura exata no prompt com o formato da tabela em bruto (colunas: ID, Descrição, Data, Fonte, Analista). Exigiu-se que o modelo não fizesse qualquer classificação prévia, deixando a tarefa de separar e organizar os requisitos para a equipa.
Comportamento Livre vs. Controlado: A avaliação geral mostrou que a IA teve dificuldades quando deixada a gerar texto de forma totalmente livre, necessitando de ajustes iterativos.	Imposição de regras e limites claros: Concluiu-se que os LLMs funcionam muito melhor com regras bem definidas. Graças a este refinamento das instruções, foi possível obter a tabela correta e completa apresentada na secção de levantamento preliminar.

Tabela 6: Prompts de levantamento de entrevistas.

3.10.4. Refinamento de Requisitos

3.10.4.1. Prompt Inicial

“Na próxima fase que é o 2.7 Refinamento de requisitos ambíguos nós pretendemos refinar este requisitos retirando os que são ambíguos e ou duplicados eu preciso que analises o texto anterior e me digas quais são os requisitos finais. Lembra-te de tudo o que falámos até agora se me puderes dar uma tabela com o número do requisito definição e onde foi retirado.”

3.10.4.2. Avaliação Crítica do Resultado Inicial e Refinamento

Avaliação Crítica do Resultado	Refinamento Aplicado / Ação Tomada
Resultados Globalmente Satisfatórios: O modelo fez um excelente trabalho ao analisar a lista bruta, conseguindo identificar e remover com sucesso os requisitos repetidos e realizando corretamente a separação entre requisitos funcionais e não funcionais.	Manutenção da Estrutura Lógica: Nenhuma ação corretiva foi necessária nesta vertente, visto que os resultados foram exatamente ao encontro do pretendido pela equipa.

Descrições Complexas e Extensas: Surgiu o contratempo de a IA gerar descrições demasiado longas e complexas. Isto ia contra o objetivo principal de ter uma lista de requisitos simples, direta e fácil de consultar pela equipa e pelos stakeholders.	Divisão e Simplificação: Para resolver o problema encontrado, a equipa ajustou o prompt pedindo à IA para aumentar o número de requisitos, dividindo os requisitos complexos em vários requisitos mais simples e granulares.
---	---

Tabela 7: Prompts de refinamento de requisitos.

A execução do capítulo 3 foi a que apresentou os desvios mais expressivos dentro da Etapa 1, ainda que todos controlados e sem comprometer a data de entrega global. As entrevistas com o membro do *staff* e as respetivas conclusões do levantamento de informação registaram um atraso de um dia, resultante da dificuldade em compatibilizar a disponibilidade dos intervenientes. A elaboração dos requisitos funcionais e não funcionais prolongou-se um dia além do previsto, reflexo da elevada granularidade da lista de requisitos eliciados. O refinamento de requisitos ambíguos e a geração de user stories acusaram igualmente desvios de um dia, explicados pela complexidade inerente à tradução de necessidades qualitativas em especificações técnicas mensuráveis. As secções de casos de uso e diagramas de atividades foram, ainda assim, concluídas dentro do prazo, compensando parcialmente os atrasos acumulados. O capítulo 3 ficou concluído a 02/03, respeitando a data limite da Etapa 1 definida pelo docente.

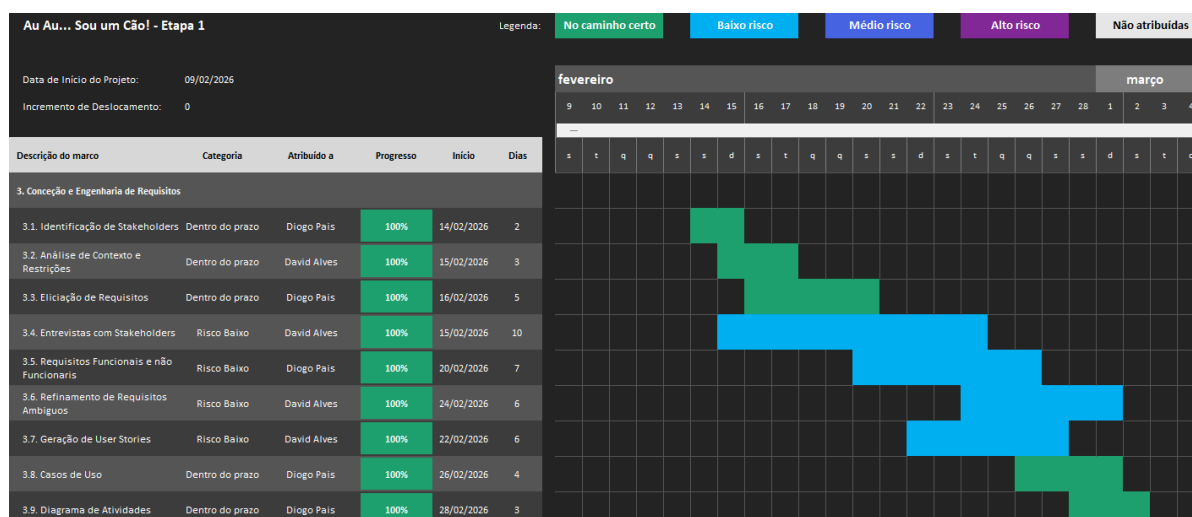


Figura 18: Diagrama de Gantt final – Capítulo 3.

4. Arquitetura e *Design* do Software

Este capítulo descreve a solução técnica concebida para suportar as necessidades operacionais do hotel "Au Au... Sou um Cão!". A arquitetura foi desenhada para garantir alta disponibilidade (99,9%), segurança rigorosa dos dados clínicos e uma experiência de utilização fluida, tanto para os tutores como para o *staff* em campo.

O capítulo 4 foi planeado para decorrer entre os dias 02/03 e 23/03, correspondendo integralmente à Etapa 2. Para além da arquitetura e design do sistema, este capítulo incorpora uma análise de alternativas arquiteturais, a justificação das escolhas tecnológicas e o registo dos prompts e avaliações LLM utilizados ao longo do processo. A figura seguinte apresenta o diagrama de Gantt objetivo para este capítulo.

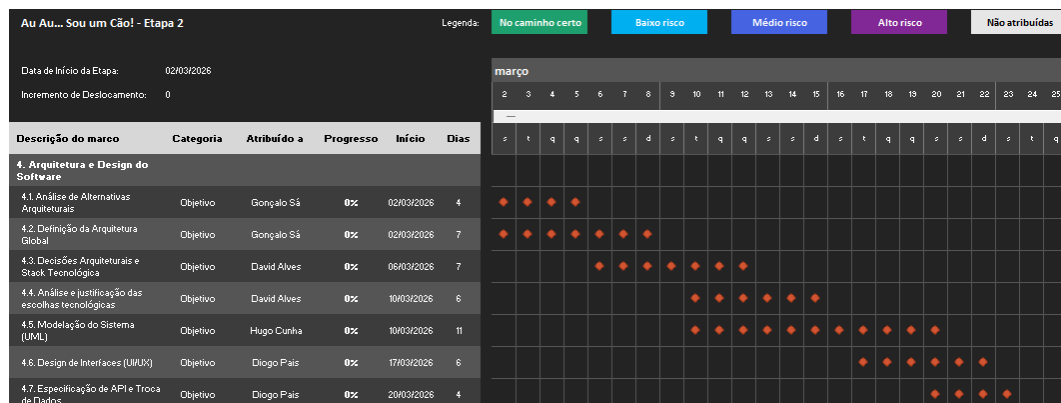


Figura 19: Diagrama de Gantt objetivo – Capítulo 4.

4.1 Análise de Alternativas Arquiteturais

Para o desenvolvimento do sistema "Au Au... Sou um Cão!", a equipa ponderou três abordagens arquiteturais distintas. O objetivo foi encontrar o equilíbrio ideal entre a complexidade de desenvolvimento, a facilidade de manutenção e a capacidade de resposta aos picos sazonais de ocupação (até 40 animais e 500 acessos simultâneos)

Alternativa A: Arquitetura Monolítica Tradicional (*Server-Side Rendering*)

Neste modelo, a interface de utilizador, a lógica de negócio e o acesso aos dados estariam todos acoplados numa única aplicação (por exemplo, usando templates renderizados no servidor).

- **Pontos Fortes (Pros):** Simplicidade inicial de desenvolvimento, facilidade de *deployment* (apenas um bloco para colocar no servidor) e facilidade em testes *end-to-end*.

- **Pontos Fracos (Contras):** Forte acoplamento entre o *frontend* e o *backend*. Se os tutores gerassem muito tráfego no portal *web* durante a abertura de vagas de verão, teríamos de escalar toda a aplicação (o que consome mais recursos) em vez de escalar apenas a interface. Adicionalmente, dificultaria a criação de uma interface *Mobile-First* fluida para o *staff*.

Alternativa B: Arquitetura Baseada em Microsserviços

Esta abordagem dividiria o sistema em pequenos serviços totalmente independentes (ex: Serviço de Reservas, Serviço Clínico, Serviço de Faturação), cada um com a sua própria base de dados.

- **Pontos Fortes (Pros):** Escalabilidade extrema e isolamento de falhas (se o serviço de faturação falhasse, as reservas continuariam a funcionar).
- **Pontos Fracos (Contras):** Alta complexidade operacional e de infraestrutura. Sendo um projeto académico desenvolvido por 4 pessoas, a sobrecarga de gerir a comunicação entre serviços, consistência de dados distribuídos e múltiplos *deployments* na AWS seria um enorme *overkill* para a dimensão do negócio (um único hotel físico).

Alternativa C: Arquitetura Cliente-Servidor em Três Camadas

Esta é a solução que decidimos adotar, separando o Portal *Web* (React) da API de Lógica de Negócio (Node.js) e da Base de Dados (PostgreSQL).

Pontos Fortes (Pros): Permite escalar o *frontend* e o *backend* de forma independente. A API RESTful serve perfeitamente os três módulos de interface (Cliente, Gestão, Staff) a partir de uma única fonte de verdade. Garante um bom nível de modularidade (através da Arquitetura Hexagonal no *backend*) sem a complexidade exagerada dos microsserviços.

Pontos Fracos (Contras): Exige a gestão de dois repositórios/ambientes de *deployment* distintos (um para o React, outro para o Node.js), aumentando ligeiramente a complexidade face ao monólito puro.

4.1.1. Justificação da Escolha

Decisão: A equipa optou pela **Arquitetura Cliente-Servidor em Três Camadas**. A justificação prende-se com a natureza do caso de estudo. O hotel necessita de uma interface ágil e *mobile-first* para o *staff* nos canis e de um portal intuitivo para os tutores. Um monólito tornaria a interface pesada e pouco reativa, enquanto os microsserviços

trariam uma complexidade desnecessária e custos de *cloud* potencialmente superiores ao *AWS Free Tier*. A separação em três camadas oferece a flexibilidade exata que o sistema precisa para aguentar os picos de verão (escalando apenas a Lógica de Negócio ou a Base de Dados, se necessário) , mantendo o código organizado em subsistemas coesos.

4.2. Definição da Arquitetura Global

Para dar resposta à multiplicidade de utilizadores e à necessidade de processamento intensivo durante as épocas altas, o sistema foi desenhado com base numa **Arquitetura Cliente-Servidor distribuída em Três Camadas (3-Tier Architecture)**. Esta escolha arquitetural separa a interface de utilização, as regras de negócio e o armazenamento de dados, o que é fundamental para um ecossistema que cruza operações de campo (canis) com acessos externos (tutores em casa) e gestão de escritório. A infraestrutura do sistema divide-se da seguinte forma:

4.2.1. Camada de Apresentação (Frontend / Clientes)

Esta camada representa os pontos de contacto com o sistema. Seguindo as melhores práticas de otimização de recursos de desenvolvimento, optou-se por consolidar a Camada de Apresentação num **Portal Web Unificado**. O acesso às diferentes funcionalidades é gerido dinamicamente através de um sistema de rotas protegido por perfis (RBAC), subdividindo a interface em três ambientes distintos consoante o utilizador:

- **Módulo Web do Cliente (Tutor):** Rota responsiva acedida via *browser*, desenhada para que os donos possam consultar a disponibilidade, efetuar reservas de forma autónoma, submeter documentação (boletins de vacinas) e acompanhar o "Diário de Bordo" do animal.
- **Dashboard de Gestão e Receção (Diana e Administrativos):** Rota otimizada para *desktop*, utilizada na receção e no *backoffice*. Permite uma visão global do hotel, gestão do calendário de reservas, processamento de *check-ins* e *check-outs*, ativação de estados de quarentena, e consulta de relatórios de faturação e ocupação.
- **Módulo Operacional Mobile-First (Staff):** Rota desenhada especificamente para ser acedida através do *browser* dos dispositivos móveis do *staff* nos canis. Mantém o conceito *stress-proof* e a operação com uma só mão. Foca-se exclusivamente em tarefas operacionais: validação de *checklists* de limpeza, passeios, registo de alimentação e administração de medicação.

4.2.2. Camada de Lógica de Negócio (Application Server / Backend)

Atua como o "cérebro" centralizado do sistema. Recebe e processa todos os pedidos vindos das diferentes rotas da Camada de Apresentação. As suas principais responsabilidades incluem:

- **Motor de Regras de Negócio:** Executa o algoritmo de sugestão de alocação de quartos (cruzando o porte e a reatividade do animal), calcula o preço dinâmico (*peak pricing*) e as taxas de cancelamento.
- **Orquestração de Dados e Comunicação:** Gere o envio automático de notificações (ex: alertas de medicação para o tutor) e processa os pedidos de leitura e escrita antes de os enviar para a base de dados.

4.2.3. Camada de Dados (Database Tier)

A camada mais profunda e protegida do sistema, isolada de acessos diretos por parte dos clientes. É constituída por um repositório centralizado que assegura a persistência, integridade e auditabilidade inalterável de todos os registos (historial clínico, logs de medicação e dados de faturação).

- **Suporte a Picos de Lotação (40 animais):** A principal vantagem estrutural desta arquitetura em três camadas é a sua capacidade de escalabilidade independente. Durante épocas festivas (Verão e Natal), quando a lotação do hotel atinge o pico de 40 animais e o portal pode receber mais de 500 acessos simultâneos na abertura de vagas, a Camada de Lógica de Negócio e a Base de Dados podem ser reforçadas (escaladas) de forma transparente. Isto garante que o sistema não sofre degradação de performance, mantendo tempos de resposta inferiores a 2 segundos nas operações críticas.

4.3. Decisões Arquiteturais e *Stack* Tecnológica

Para concretizar a arquitetura em três camadas definida e suportar a necessidade crítica de fornecer acessos distintos consoante o perfil do utilizador, a *stack* tecnológica e os padrões de desenho de *software* foram selecionados privilegiando a escalabilidade, a manutenibilidade do código e a centralização do ecossistema *web*.

4.3.1. Padrão de Desenho Interno do Servidor (Arquitetura Hexagonal)

No que diz respeito à organização interna do código da Camada de Lógica de Negócio (Backend), adotou-se o padrão de Arquitetura Hexagonal (*Ports and Adapters*). O desenvolvimento será feito em **TypeScript / Node.js**, estruturado da seguinte forma:

- **Core (Domínio Puro):** Onde residem as regras vitais do hotel, como o cálculo de capacidade de boxes/quartos clínicos, validação de compatibilidade de isolamento entre pacientes e as validações de dosagem de fármacos. Este núcleo não possui dependências de bases de dados ou bibliotecas externas.
- **Ports (Interfaces):** Contratos que definem como o *Core* comunica com o mundo exterior.
- **Adapters (Implementações):** Os módulos que ligam o sistema às tecnologias externas. Destaca-se o isolamento da integração com a Autoridade Tributária, garantindo a conformidade fiscal, e o módulo de auditoria, que assegura a retenção de *logs* imutáveis por 3 anos.

4.3.2. Tecnologias de Frontend (Camada de Apresentação)

Para garantir a coesão do sistema e evitar a redundância de manter uma aplicação nativa separada, todo o *Frontend* será consolidado numa única *framework*:

- **Portal Web Unificado (React.js):** O sistema funcionará como uma *Single Page Application* (SPA) desenvolvida em **React.js**. A separação dos ambientes de trabalho (Cliente, Gestão e Staff) será estritamente controlada por um sistema de rotas seguro (*React Router*).
- **Adaptação de UI e Modo Offline:** O Portal do Cliente e o Módulo do Staff seguirão uma abordagem *Mobile-First*. O acesso do *staff* fará uso de APIs web padrão (HTML5) para invocar a câmara do dispositivo (para o registo fotográfico do Diário de Bordo) e utilizará tecnologias nativas dos *browsers* (como *Service Workers* e *Local Storage*) para garantir o funcionamento em modo *offline* com sincronização posterior, simulando a experiência de uma aplicação móvel diretamente no navegador. O Dashboard da Gestão renderizará vistas complexas de alocação de boxes otimizadas para *desktop*.

4.3.3. Controlo de Acessos e Segurança (RBAC)

Como os três perfis de utilizadores acedem ao mesmo portal web *root*, o sistema implementa uma rigorosa matriz de permissões baseada no modelo RBAC (*Role-Based Access Control*), gerida através de **JSON Web Tokens (JWT)**:

- **Perfil Receção:** Possui permissões de leitura e escrita exclusivas para o fluxo de gestão clínica (*admissão*, *check-in*, protocolo clínico, *checkout*, faturação).
- **Perfil Gestão/Veterinária (Diana):** Assume privilégios de "*Super-User*". Herda as permissões da Receção e Veterinária, mas detém acesso exclusivo a operações críticas: ativação de protocolos de Quarentena, prescrições médicas e visualização de dados financeiros.
- **Perfil Staff (Cuidadores):** O *login* redireciona e restringe o utilizador à vista *mobile-first* operacional. O *backend* rejeita ativamente qualquer pedido deste perfil dirigido a rotas ou *endpoints* financeiros, segregando totalmente o acesso a faturas e alterações de reservas.
- **Encriptação:** Todos os dados sensíveis são protegidos utilizando o algoritmo AES-256 e transmitidos via canais seguros (SSL/TLS).

4.3.4. Base de Dados e Infraestrutura Cloud (AWS)

Para garantir os rigorosos Requisitos Não Funcionais de escalabilidade e disponibilidade, optou-se por alojar a infraestrutura na nuvem da AWS (Amazon Web Services):

- **Alojamento Aplicacional:** O servidor de Lógica de Negócio (Node.js/Express) e a distribuição da aplicação *Frontend* (React.js) serão alojados no serviço **AWS Elastic Beanstalk** (ou em instâncias **AWS EC2**). Esta escolha permite manter a API na mesma Rede Virtual Privada (VPC) da base de dados RDS, garantindo latências de comunicação mínimas.
- **Base de Dados Relacional:** Adoção do PostgreSQL alojado no serviço gerido **AWS RDS**. O serviço assegura um uptime de 99.9% e gere backups automáticos (RPO < 1h e RTO < 4h).
- **Armazenamento Multimédia:** Os ficheiros pesados (boletins de vacinas e média do Diário de Bordo) serão armazenados no **AWS S3 (Simple Storage Service)**, garantindo uma latência inferior a 2 segundos nas operações críticas da aplicação web.

Otimização de Custos: De forma a adequar esta arquitetura ao contexto académico, toda a infraestrutura foi dimensionada para operar dentro do *AWS Free Tier* (12 meses).

A utilização de instâncias de baixo consumo computacional (t3.micro para a API e db.t3.micro para a base de dados) em configurações de instância única garante custo zero durante a atual fase de desenvolvimento, teste e demonstração do protótipo.

4.4. Análise e justificação das escolhas tecnológicas

4.4.1. Escolha da infraestrutura Cloud

A decisão de onde alojar o sistema "Au Au... Sou um Cão!" foi crucial para garantir que o hotel pudesse operar sem interrupções, especialmente nos picos de Verão e Natal. Escolhemos a AWS após compararmos os principais fornecedores do mercado.

Fornecedor	Prós	Contras
AWS (Escolhida)	Líder de mercado com a infraestrutura mais robusta; Enorme variedade de serviços; Programa <i>Free Tier</i> muito generoso para estudantes.	A consola de gestão pode ser complexa devido à enorme quantidade de opções.
Microsoft Azure	Excelente integração com ferramentas Microsoft; Forte em ambientes empresariais	Menos intuitiva para projetos baseados em Open Source (como Node.js e PostgreSQL) comparado com a AWS.
Google Cloud (GCP)	Interface simples; Excelente para análise de dados e IA.	Ecossistema de serviços menos maduro em algumas áreas de infraestrutura clássica.

Tabela 8: Prós e Contras de serviços Cloud.

Justificação: A AWS foi a escolha óbvia não só pela sua fiabilidade global, mas também pela facilidade com que conseguimos configurar o projeto dentro do *Free Tier*. Isto permitiu-nos ter um ambiente de produção real, com segurança profissional, a custo zero durante a fase de desenvolvimento académico.

4.4.2. Análise Comparativa e Justificação da Stack Tecnológica

A seleção das tecnologias para o desenvolvimento do sistema "Au Au... Sou um Cão!" não foi arbitrária. Resultou de uma ponderação entre os requisitos operacionais do negócio (escalabilidade para lidar com picos de lotação) e as restrições académicas do projeto (curva de aprendizagem e limitação de custos).

Abaixo apresenta-se a avaliação das alternativas para as três dimensões principais da arquitetura tecnológica: *Frontend*, *Backend* e Base de Dados.

4.4.2.1. Camada de Apresentação (*Frontend*)

Para o desenvolvimento do portal web unificado, foram ponderadas três abordagens principais para o desenvolvimento da SPA.

Tecnologia	Prós	Contras
React.js (Escolhida)	Vasto ecossistema e suporte comunitário; Reutilização de componentes; Facilidade de integração com <i>Service Workers</i> para modo offline	Curva de aprendizagem inicial; Exige a configuração de bibliotecas de terceiros para roteamento (<i>React Router</i>) e gestão de estado.
Angular	<i>Framework</i> completa (inclui roteamento e http client nativos); Forte tipagem (TypeScript nativo); Excelente para aplicações empresariais densas.	Curva de aprendizagem muito acentuada; Excesso de complexidade (<i>boilerplate</i>) para um protótipo ágil; Maior peso do bundle final.
Vue.js	Curva de aprendizagem suave; Sintaxe limpa e intuitiva; Excelente documentação.	Ecossistema corporativo menor quando comparado ao React; Menor oferta de bibliotecas específicas para manipulação de fluxos complexos offline.

Tabela 9: Prós e Contras das opções frontend.

Justificação: O React.js foi escolhido devido à sua flexibilidade e à vasta gama de bibliotecas disponíveis que aceleram o desenvolvimento iterativo. A capacidade de manipular de forma eficiente o *Document Object Model* (DOM) virtual garante a fluidez necessária (latência inferior a 1 segundo) para os mapas de alocação. Além disso, a facilidade em implementar estratégias *Mobile-First* e capacidades *offline* via *Service Workers* adequa-se perfeitamente à realidade do staff nos canis.

4.4.2.2. Camada de Lógica de Negócio (*Backend*)

O "motor" central do sistema precisava de suportar processamento assíncrono rápido, integrando as regras de negócio clínicas e financeiras.

Tecnologia	Prós	Contras
Node.js com TypeScript (Escolhida)	Uniformidade de linguagem (JavaScript/TypeScript no <i>Frontend</i> e <i>Backend</i>); Elevada performance em operações I/O (assíncronas)	Sendo <i>single-threaded</i> , pode sofrer bloqueios se submetido a processamento de CPU muito intensivo; Tipagem exige configuração rigorosa no arranque
Java (Spring Boot)	Altamente tipado e seguro; Padrões de desenho empresariais consolidados; Excelente para concorrência pesada.	Consumo de memória RAM elevado (incompatível com a eficiência desejada para o <i>AWS Free Tier</i>); Verbosidade do código atrasa o <i>time-to-market</i> .
Python (Django/FastAPI)	Sintaxe rápida e clara; Excelente para integração futura com algoritmos de Inteligência Artificial ou <i>Machine Learning</i> .	Performance I/O nativa geralmente inferior ao Node.js; Fragmentação do ecossistema de bibliotecas assíncronas.

Tabela 10: Prós e Contras das opções backend.

Justificação: Optou-se por Node.js com TypeScript primariamente pelo paradigma *Full-Stack* JavaScript. A utilização de TypeScript no *Backend* garante a tipagem forte e a segurança necessárias para as regras de negócio puras (Arquitetura

Hexagonal), reduzindo erros em tempo de execução. Paralelamente, o motor assíncrono do Node.js é ideal para servir as múltiplas chamadas concorrentes da API esperadas nos picos de época alta, consumindo poucos recursos computacionais das instâncias AWS EC2/Elastic Beanstalk (t3.micro).

4.4.2.3. Camada de Base de Dados

A persistência de dados sensíveis (faturação e histórico clínico) exigia uma avaliação rigorosa SGBD.

Tecnologia	Prós	Contras
PostgreSQL (Escolhida)	Estrita conformidade ACID; Forte suporte a chaves estrangeiras e integridade referencial; Excelente integração com serviços geridos (AWS RDS).	Configuração de replicação e escalabilidade horizontal mais complexa do que em bases de dados NoSQL.
MongoDB (NoSQL)	Esquema flexível (documentos JSON); Excelente para escalabilidade horizontal e iterações rápidas sem migrações complexas.	Falta de integridade referencial estrita nativa; Complexidade acrescida para gerar relatórios financeiros e cruzamentos de dados complexos (JOINS). atrasa o <i>time-to-market</i> .
MySQL	Rápido, leve e com vasta literatura; Cumpre requisitos ACID.	Menor suporte nativo para tipos de dados complexos (como arrays ou JSON nativos) face ao PostgreSQL.

Tabela 11: Prós e Contras das opções da base de dados.

Justificação: A escolha recaiu sobre o PostgreSQL alojado na AWS RDS. O modelo de negócio do hotel é intrinsecamente relacional: um tutor possui animais, que efetuam reservas, que geram faturas. O cumprimento estrito das propriedades ACID do PostgreSQL é inegociável para garantir que não existem sobreposições de alocações (*overbooking* descontrolado) ou perdas de registos de medicação clínica crítica. Além disso, a gestão automatizada de *backups* pelo AWS RDS apoia o cumprimento dos requisitos não funcionais de recuperação de dados.

4.5. Modelação do Sistema (UML)

Para suportar a lógica de negócio descrita na Arquitetura Hexagonal e garantir a integridade dos dados no SGBD (PostgreSQL), procedeu-se à modelação estrutural do domínio. O modelo de dados foi desenhado para normalizar a informação clínica, financeira e logística, evitando redundâncias e garantindo a rastreabilidade exigida pelos requisitos operacionais.

4.5.1. Diagrama de Componentes

Sendo a primeira representação estrutural da arquitetura, o Diagrama de Componentes (Figura 5) ilustra a organização física, as dependências do código fonte e as interfaces estabelecidas entre as diferentes camadas do sistema "Au Au... Sou um Cão!". A aplicação estrutura-se em três grandes componentes principais, garantindo um baixo nível de acoplamento:

1. **PortalWebUI:** O componente de fronteira (Camada de Apresentação), que engloba a SPA em React. Este componente atua como "cliente", dependendo exclusivamente da interface agregadora `IHotelCaninoLN`, desconhecendo por completo a implementação concreta e as regras de negócio subjacentes.
2. **HotelCaninoLN:** O núcleo central do sistema (Camada de Lógica de Negócio). Este componente expõe os seus serviços ao exterior através da interface `IHotelCaninoLN` e atua como orquestrador dos seus subcomponentes internos. A sua responsabilidade é processar todas as regras clínicas, de reservas e financeiras.
3. **HotelCaninoDL:** O componente de infraestrutura (Camada de Dados), responsável pela comunicação com o PostgreSQL e com a *cloud* (AWS S3). A camada lógica (`HotelCaninoLN`) necessita de materializar os dados, mas fá-lo de forma abstraída exigindo a interface `IHotelCaninoDL`. Esta separação garante que futuras alterações na base de dados tenham impacto nulo na lógica de negócio central.

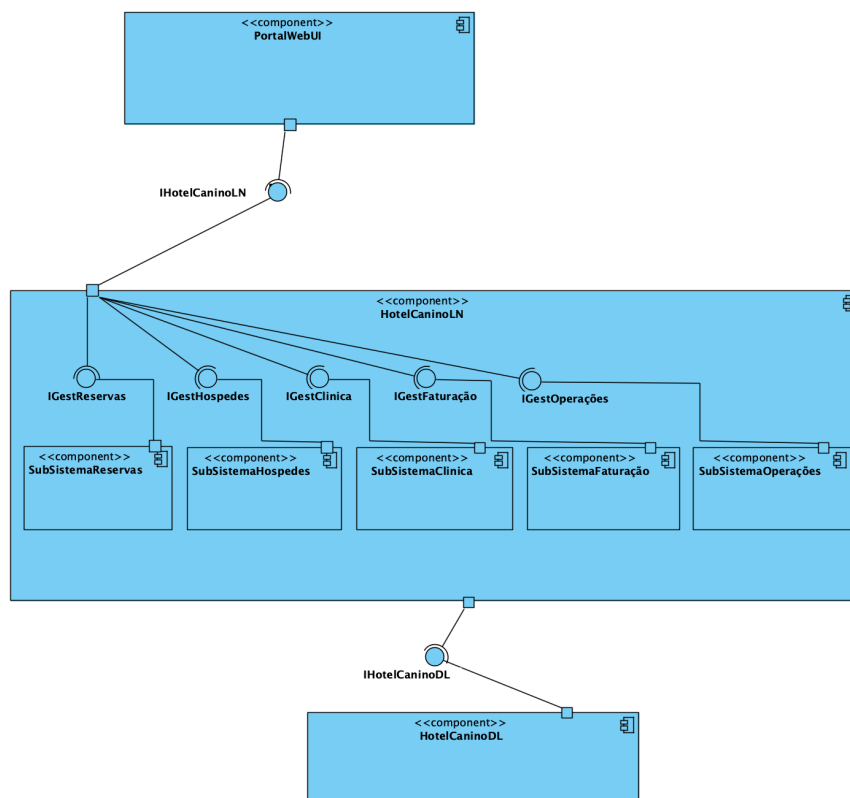


Figura 20: Diagrama de Componentes.

Para organizar o interior do componente HotelCaninoLN, procedeu-se à identificação de todas as responsabilidades do sistema a partir da análise dos Casos de Uso. Seguindo as boas práticas de engenharia de software, estas operações foram agrupadas por afinidade conceitual e domínio de dados, resultando em cinco subsistemas distintos, cada um expondo a sua própria API (Interface):

- **SubSistemaReservas (IGestReservas):** Encapsula a lógica de gestão de alojamentos, validação de disponibilidade por datas, motor de *overbooking* controlado e gestão da lista de espera (*Waitlist*).
- **SubSistemaHospedes (IGestHospedes):** Gere as entidades principais do negócio, processando o registo de Tutores e Animais, bem como a validação obrigatória dos boletins de vacinação e análise de perfis de reatividade.
- **SubSistemaClinica (IGestClinica):** O módulo mais crítico para a segurança do hotel. Processa a prescrição de medicações, a geração de *logs* imutáveis de administração de fármacos e o bloqueio de animais em estado de quarentena.
- **SubSistemaFaturacao (IGestFaturacao):** Centraliza os cálculos financeiros, incluindo a aplicação de *peak pricing*, taxas de cancelamento contratuais e a integração segura para a emissão de faturas certificadas pela Autoridade Tributária.

- **SubSistemaOperacoes (IGestOperacoes):** Focado na operação diária e no *Staff*, gere a atribuição de tarefas, a atualização do estado de limpeza das boxes, os serviços extra e o *upload* de fotografias para o Diário de Bordo do tutor.

4.5.2. Diagrama de Classes

Este diagrama (Figura 2) modela a estrutura estática do sistema, ilustrando de forma detalhada as entidades principais, as suas relações e a integração entre a Lógica de Negócio (Camada LN) e o acesso aos dados (Camada DL).

O elemento de fronteira central é a classe *GestorHotelFacade*, que aplica o padrão *Facade* para disponibilizar uma interface unificada (*IGestorHotel*) à camada de apresentação (*Frontend*). A partir deste ponto, a arquitetura ramifica-se em cinco subsistemas lógicos altamente coesos. Cada subsistema possui a sua própria fachada que implementa o respetivo contrato/interface, encapsulando e isolando a complexidade das regras de negócio de cada área operacional do hotel.

No núcleo do sistema residem as Entidades de Domínio. Estes são os objetos fundamentais que representam os conceitos reais do negócio e que circulam entre os subsistemas. O modelo tira ainda partido de relações de herança (ex: Tutor e Funcionario herdam da classe base Utilizadores) e enumerações (ex: EstadoReserva, TipoPerfil) para garantir a integridade dos tipos de dados.

Para garantir a persistência da informação de forma estruturada e segura, o diagrama reflete a implementação transversal do padrão **DAO (Data Access Object)**. Em vez de manterem dados em memória, as *Facades* de negócio delegam todas as operações de leitura e escrita na base de dados (PostgreSQL) a classes dedicadas. Esta decisão arquitetural isola totalmente a lógica de negócio da infraestrutura de armazenamento físico, garantindo um baixo acoplamento e facilitando futuras manutenções ou substituições tecnológicas.

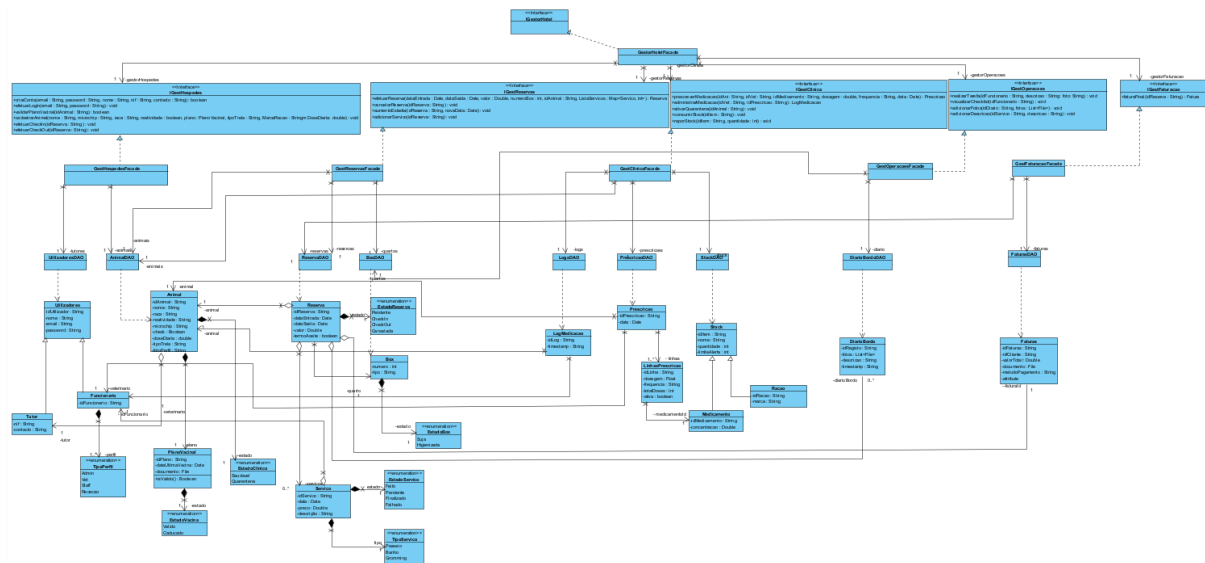


Figura 21: Diagrama de Classes.

4.5.3. Diagramas de Sequência

Enquanto o Diagrama de Classes modela a estrutura estática do sistema, os Diagramas de Sequência detalham o seu comportamento dinâmico e temporal. Estes diagramas ilustram o fluxo sequencial de mensagens e a cooperação entre os diferentes objetos e camadas da arquitetura para a concretização dos Casos de Uso mais críticos do hotel canino. A modelação destes fluxos operacionais teve como prioridade o cumprimento estrito das boas práticas de Desenho Orientado a Objetos (OOD) e dos padrões definidos para a arquitetura do sistema. Em todos os diagramas apresentados, destacam-se três princípios fundamentais:

- **Baixo Acoplamento e Orquestração Central:** A comunicação flui sempre a partir do controlador principal — a interface raiz IGestorHotel. Para proteger as fronteiras lógicas, as fachadas dos subsistemas não comunicam diretamente entre si. É o Gestor Central que orquestra a recolha e entrega de dados (ex: o Gestor obtém o NIF na GestHospedes e entrega-o à GestFaturacao).
- **Princípio do Especialista da Informação:** A manipulação de dados e a instanciação de objetos ocorrem estritamente nos subsistemas onde esses dados residem ou são geridos. As operações não quebram o encapsulamento, delegando as responsabilidades às respetivas Fachadas e Entidades.
- **Persistência Abstráida (Padrão DAO):** O fecho de cada transação é sempre delegado aos objetos DAO, garantindo que a persistência do estado do sistema na base de dados fica totalmente separada da lógica de negócio principal.

Para demonstrar a fiabilidade da arquitetura, procedeu-se à modelação de oito operações estruturantes, garantindo uma cobertura transversal de todos os subsistemas do hotel:

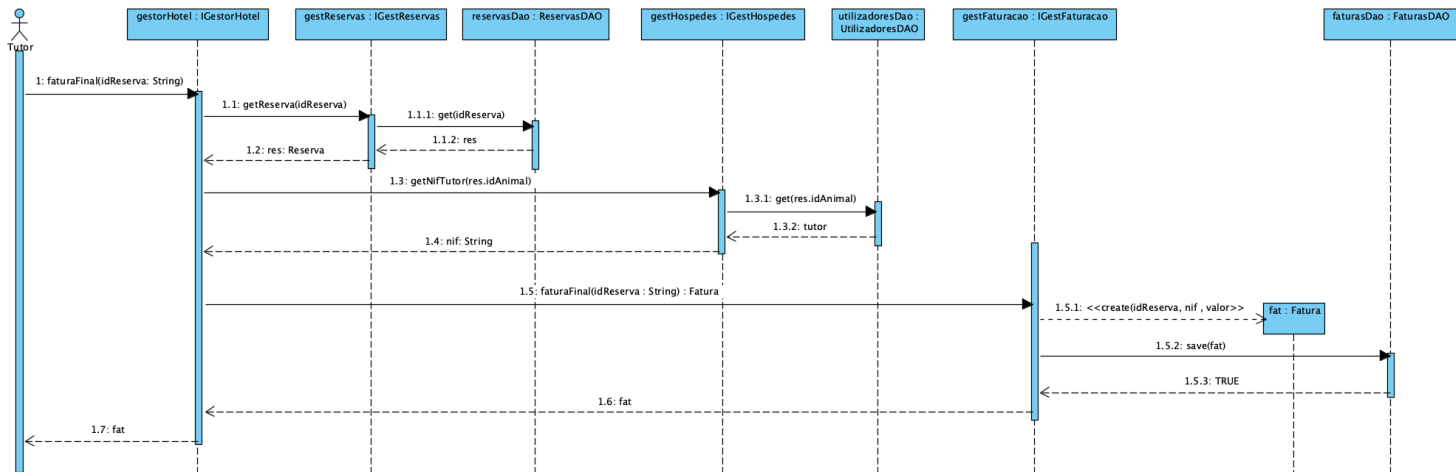


Figura 22: Diagrama de Sequência - faturaFinal.

Neste fluxo, a GestorHotel atua como orquestradora central para garantir o baixo acoplamento do sistema. Ao receber o pedido do Tutor, o gestor interroga a GestReservas para obter a instância da reserva (e o seu valor) e, de seguida, a GestHospedes para recuperar o NIF do cliente através do respetivo DAO. Com as informações reunidas, o gestor delega a criação (<<create>>) e a persistência da entidade Fatura à GestFaturacao, que submete o objeto ao FaturasDAO antes de o documento ser retornado ao utilizador.

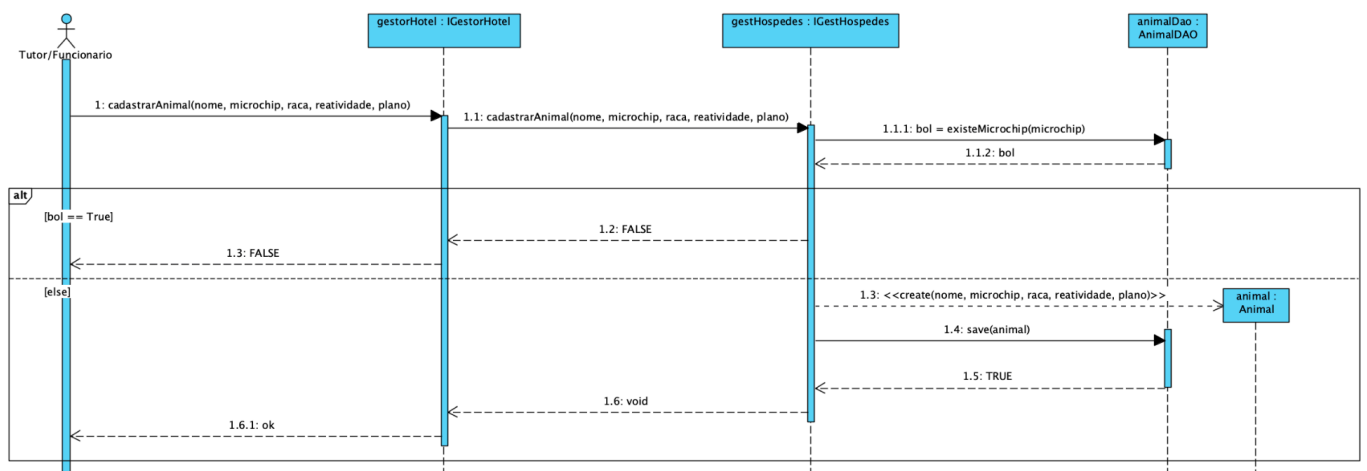


Figura 23: Diagrama de Sequência - cadastrarAnimal.

Nesta operação, o controlador GestorHotel delega o pedido de registo à GestHospedes. Para garantir a integridade dos dados, a fachada verifica a unicidade do registo consultando previamente o AnimalDAO pelo *microchip*. Através de um fragmento condicional (alt), o fluxo é interrompido devolvendo erro caso o animal já exista; caso contrário (*e/se*), a GestHospedes assume a responsabilidade de instanciar a nova entidade Animal e persistir a mesma na base de dados, fechando o ciclo com uma confirmação de sucesso ao utilizador.

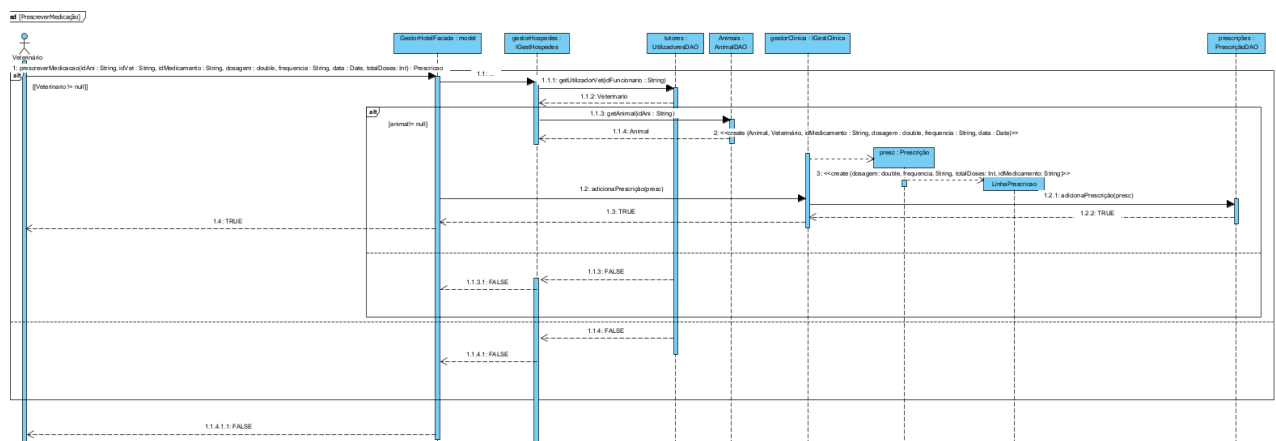


Figura 24: Diagrama de Sequência - *prescreverMedicação*.

Nesta operação, o controlador GestorHotelFacade recebe e processa o pedido de prescrever uma medicação primeiro verifica a existencia do animal e veterinário em seguida criar presc que é uma Prescrição com os argumentos dados e com que por sua vez itera sobre a map de linhas e adiciona cada uma das linhas da prescrição a mesma, por fim adicionamos a prescrição no PrescriçãoDAO.

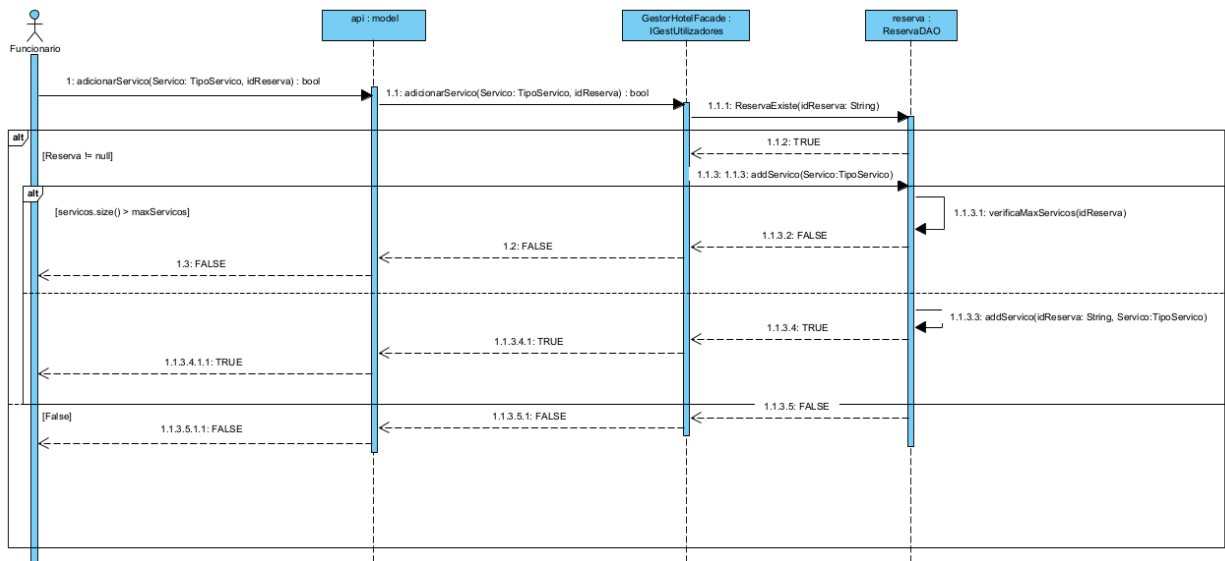


Figura 25: Diagrama de Sequência - adicionarTarefa.

Nesta operação, o controlador HotelGestorFacade gere o pedido de adição de uma nova tarefa por alguém que tem uma reserva, mas primeiro verificamos a reserva após isso se existir uma reserva pedimos para que ele adicione a reserva o serviço, se a reserva já tiver o máximo de serviços não adicionamos se ainda não tivermos o número máximo deixamos adicionar.

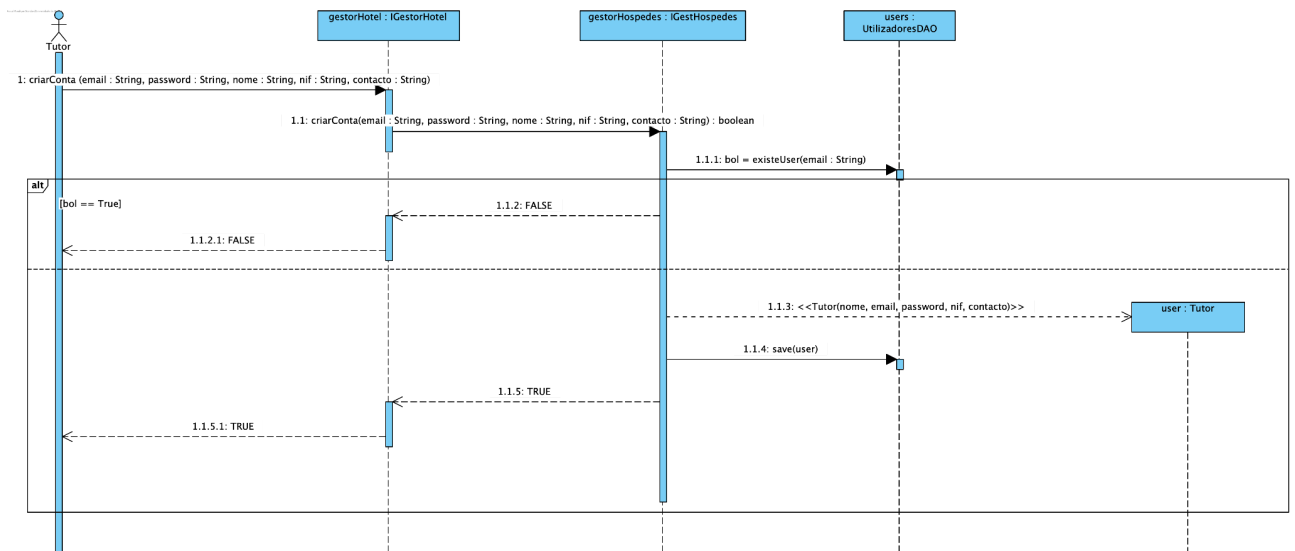


Figura 26: Diagrama de Sequência - criarConta.

Neste fluxo, o gestorHotel recebe o pedido de criação de conta e delega a operação ao gestorHospedes, que retorna um booleano de sucesso. Para garantir a integridade dos dados, a fachada consulta previamente o UtilizadoresDAO para verificar se o email já se encontra registado no sistema (bol = existeUser(email : String)). Através de um fragmento condicional (alt), caso o utilizador já exista (bol == True), o fluxo é interrompido e é devolvido FALSE ao gestor e subseqüentemente ao utilizador. Caso contrário, o subsistema assume a responsabilidade de instanciar a nova entidade Tutor (<<Tutor(nome, email, password, nif, contacto)>>), persisti-la na base de dados (save(user)) e retornar TRUE como confirmação de sucesso ao longo da cadeia de chamadas.

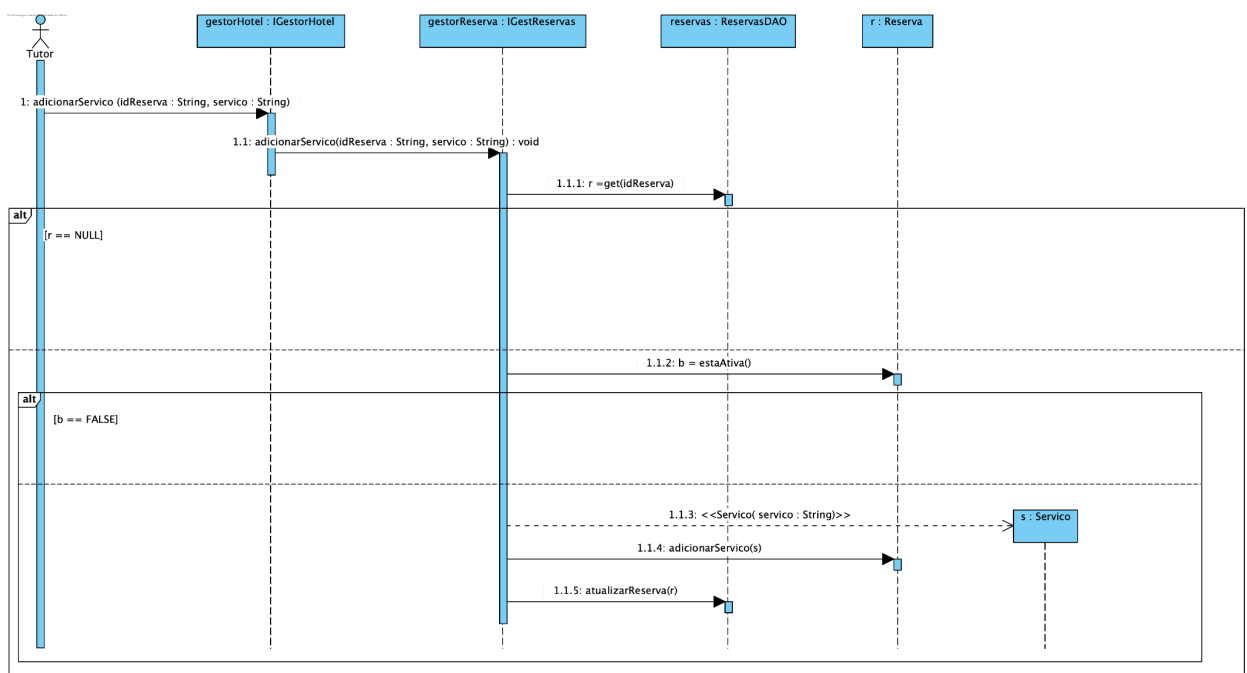


Figura 27: Diagrama de Sequência - adicionarServico.

Neste fluxo, o gestorHotel atua como orquestrador central, recebendo o pedido de adição de um serviço a uma reserva existente. O gestor delega a operação à gestorReserva, que por sua vez consulta o reservasDAO para obter a instância da reserva através do identificador fornecido (r = get(idReserva)). Através de um fragmento condicional (alt), o fluxo é interrompido caso a reserva não exista (r == NULL). Verificada a sua existência, a fachada valida se a reserva se encontra ativa (b = estaAtiva()), sendo novamente interrompida caso contrário (b == FALSE). Cumpridas ambas as validações, o subsistema procede à instanciação do novo objeto Servico (<<Servico(servico : String)>>), adiciona-o à reserva (adicionarServico(s)) e persiste o estado atualizado na base de dados (atualizarReserva(r)), garantindo a rastreabilidade do serviço contratado.

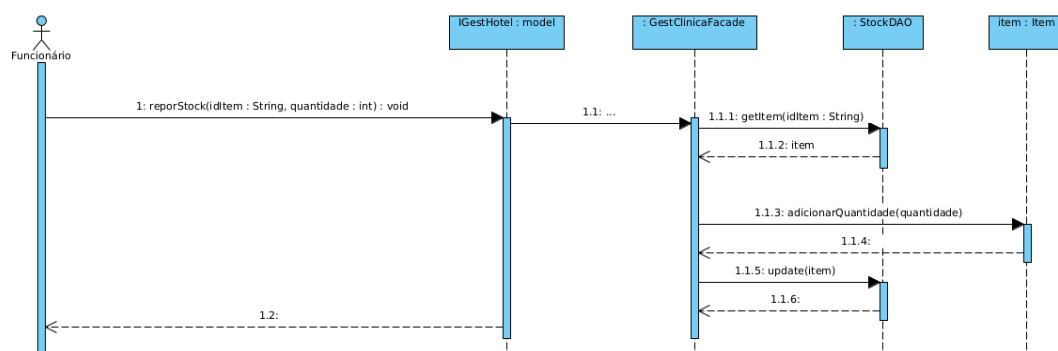


Figura 28: Diagrama de Sequência - reporStock.

Neste fluxo, o Funcionário inicia a operação de reposição de stock, com o IGestHotel a delegar a execução diretamente à GestClinicaFacade. Cumprindo o Princípio SLAP e o padrão de Especialista da Informação, a fachada recupera a instância do Item através do StockDAO e delega à própria entidade a responsabilidade algorítmica de alterar o seu estado (adicionarQuantidade). Após a atualização dos dados em memória, a fachada assegura a persistência invocando o método update no respetivo DAO, concluindo a transação e devolvendo o controlo ao utilizador.

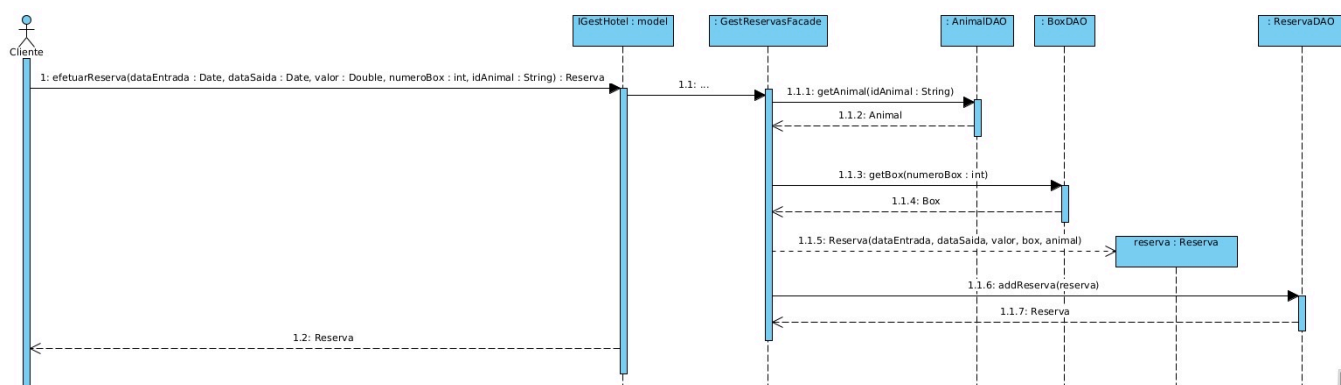


Figura 29: Diagrama de Sequência - efetuarReserva.

Nesta operação, o IGestHotel atua como controlador central, delegando o pedido inicial do Cliente diretamente à GestReservasFacade. Para garantir a integridade referencial dos dados, a fachada consulta previamente o AnimalDAO e o BoxDAO para recuperar as instâncias necessárias. Reunidos os objetos de domínio subjacentes, a fachada procede à instanciação da nova entidade Reserva e assegura a sua persistência imediata através do ReservaDAO, concluindo o fluxo com a devolução do registo validado ao utilizador.

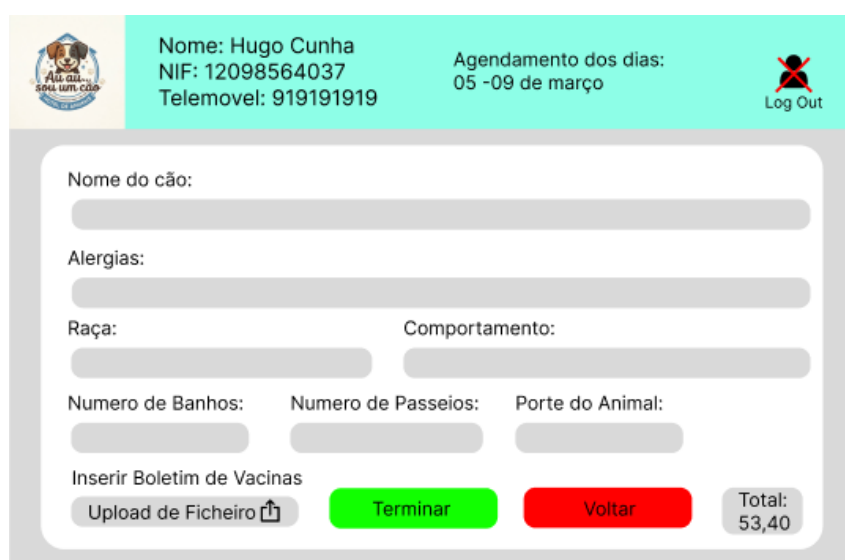
4.6. Design de Interfaces (UI/UX)

Nesta secção, o foco transita da infraestrutura de *backend* para a interação direta com os utilizadores. O design das interfaces (UI) e a experiência de utilizador (UX) foram concebidos em estrito alinhamento com a arquitetura definida (Portal Web Unificado em React.js), garantindo que o *software* se adapta responsivamente à realidade física e ao contexto de uso de cada perfil no hotel canino.

A abordagem de design no Figma dividiu-se em três vistas perfeitamente segregadas pelo sistema de rotas RBAC:

1. O **Módulo Web do Cliente (Tutor)**, com um design limpo e responsivo focado em transmitir confiança e autonomia na gestão de reservas;
2. O **Dashboard de Gestão e Receção**, otimizado para ecrãs *desktop*, privilegiando a densidade de informação (calendários e mapas de ocupação);
3. O **Módulo Operacional (Staff)**, desenhado sob os princípios de *Mobile-First*, *Stress-Proof* e *One-Hand Operation*, garantindo que os cuidadores conseguem registar informações críticas no browser dos seus telemóveis de forma rápida e isenta de erros, mesmo num ambiente de canil ruidoso e movimentado.

4.6.1. Módulo Web do Cliente (Tutor)



A interface do Módulo Web do Cliente (Tutor) apresenta uma barra de topo com informações do utilizador e uma seção de formulário para a reserva. A barra de topo, com fundo verde, contém o logótipo do hotel canino à esquerda, os dados pessoais do utilizador (Nome: Hugo Cunha, NIF: 12098564037, Telemovel: 919191919) no centro e o agendamento dos dias (05 -09 de março) e o botão 'Log Out' à direita. O formulário principal, com fundo cinza claro, contém campos para 'Nome do cão:', 'Alergias:', 'Raça:', 'Comportamento:', 'Numero de Banhos:', 'Numero de Passeios:', e 'Porte do Animal:'. Abaixo destes campos, há uma seção para 'Inserir Boletim de Vacinas' com um botão 'Upload de Ficheiro' e dois botões de ação: 'Terminar' (verde) e 'Voltar' (vermelho). No canto inferior direito do formulário, um botão cinza indica o 'Total: 53,40'.

Figura 30: Divisão em passos lógicos.

- **Justificação de UI/UX:** A interface privilegia a clareza e a facilidade de uso. Sendo uma página *web* responsiva, os formulários de reserva e submissão de

vacinas foram divididos em passos lógicos (*wizards*) para não sobrecarregar o utilizador com demasiada informação no mesmo ecrã.



Figura 31: *Opções de Utilização.*

- **Justificação de UI/UX:** O design foca-se numa experiência utilitária. O utilizador deste portal quer, muito provavelmente, fazer apenas duas coisas: marcar dias para o seu cão ou ver como o cão se está a portar no hotel naquele dia. A interface reflete perfeitamente essa jornada do utilizador, priorizando o acesso rápido a essas duas tarefas.



Figura 32: *Diário de Bordo.*

- **Justificação de UI/UX:** A interface garante uma navegação consistente e permite o reconhecimento imediato do animal através da sua fotografia e resumo logo no topo. O histórico está claramente organizado em cartões individuais, utilizando um código de cores que facilita a leitura rápida do bem-estar do cão. Por fim, o sistema assegura o controlo do utilizador ao posicionar os botões de ação de forma lógica no final dos dados.

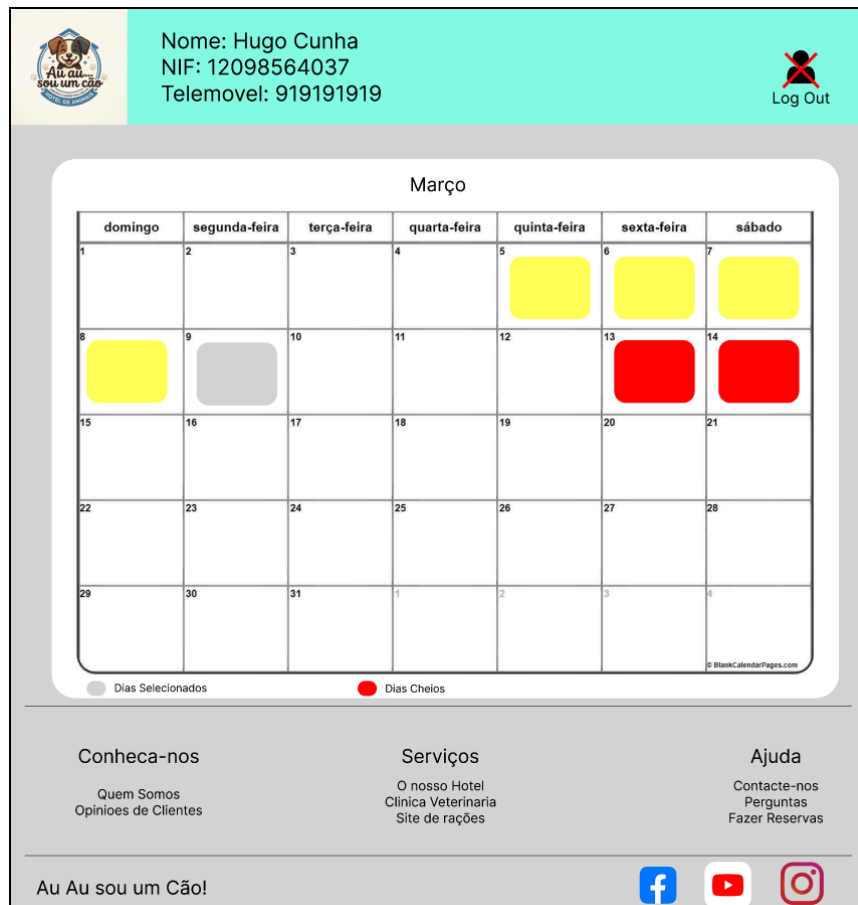


Figura 33: Marcação de Dias.

- Justificação de UI/UX:** A utilização de um calendário tradicional funciona como uma metáfora perfeita do mundo real, tornando a escolha de datas muito natural. O estado do sistema (as reservas) é visível de imediato graças ao código de cores e à legenda de apoio ("Dias Seleccionados" e "Dias Cheios"), o que reduz o esforço de memória.

4.6.2. Dashboard de Gestão (Desktop)



Figura 34: Dashboard com Tarefas.



Figura 35: Foto do Stock com os filtros de lado.

- **Justificação de UI/UX:** Desenhado para maximizar a eficiência do trabalho de secretariado da Diana. O uso de tabelas de dados complexas, filtros rápidos e códigos de cores (ex: vermelho para quarentena, verde para *check-out*) permite uma leitura instantânea do estado geral do hotel num monitor panorâmico.

4.6.3. Módulo Operacional do Staff (Mobile-First Web)

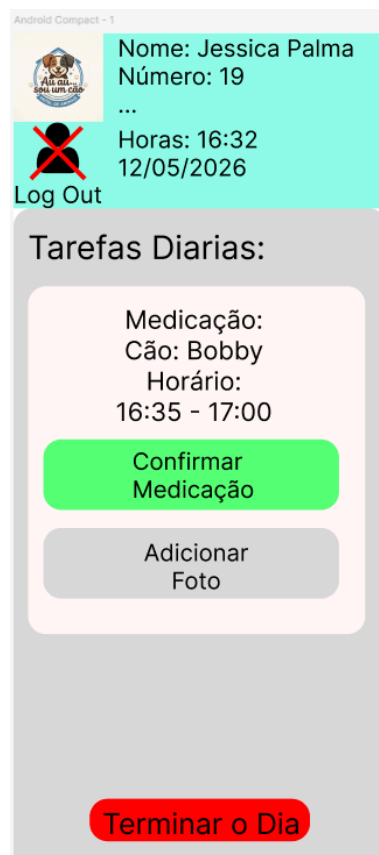


Figura 36: UI para One-Hand Operation.

- **Justificação de UI/UX:** Embora seja acedido via *browser*, a interface simula o comportamento de uma *app* nativa. Os botões de ação (ex: "Confirmar Medicação") foram sobredimensionados e colocados na metade inferior do ecrã para permitir o uso seguro com apenas uma mão (*One-Hand Operation*), assumindo que o cuidador tem a outra mão ocupada com os animais. Cores de alto contraste garantem a legibilidade no exterior.

4.6.4. Módulo do Veterinário

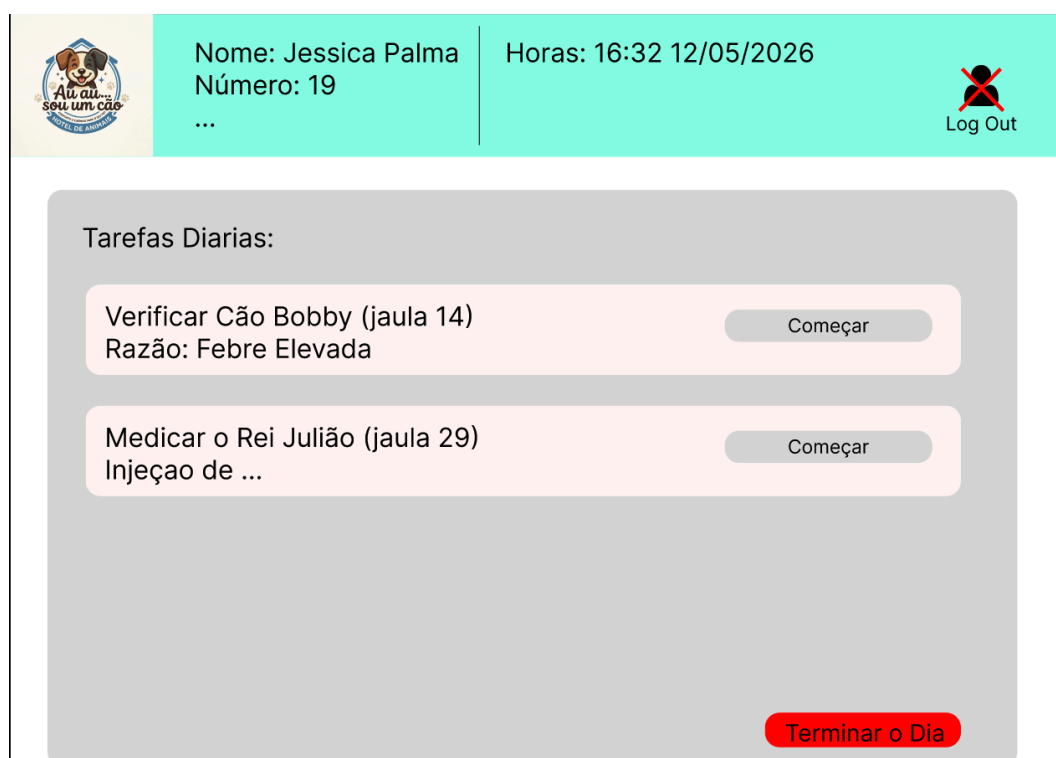


Figura 37: Dashboard do Veterinário.

- **Justificação de UI/UX:** O *design* minimalista organiza as tarefas em cartões individuais bem definidos, fornecendo contexto direto (como o número da jaula) para prevenir erros de identificação. Os botões "Começar" funcionam como chamadas para a ação claras e imediatas, enquanto o botão vermelho "Terminar o Dia", estrategicamente isolado no canto inferior, utiliza a cor para sinalizar de forma intuitiva a conclusão definitiva do turno de trabalho.

Figura 38: Dashboard do Veterinário.

- Justificação de UI/UX:** A interface mantém a navegação consistente e prioriza o reconhecimento imediato ao apresentar os dados vitais do animal e o motivo da avaliação logo no topo, evitando falhas de memória. O campo de texto livre oferece a flexibilidade necessária para registos médicos específicos. Por fim, o *design* utiliza a cor como uma excelente ferramenta de prevenção de erros: destaca a conclusão normal a azul, enquanto reserva o vermelho para ações críticas e de alerta, como o "Pedido de Quarentena", garantindo decisões seguras.

4.7. Especificação de API e Troca de Dados

A comunicação entre a Camada de Apresentação (Portal *Web* Unificado em React.js) e a Camada de Lógica de Negócio (*Backend* em Node.js) será realizada através de uma API RESTful sobre o protocolo seguro HTTPS. Esta arquitetura garante um baixo acoplamento entre o cliente e o servidor, permitindo que a interface de utilizador evolua de forma independente das regras de negócio.

4.7.1. Definição do Protocolo de Troca de Dados

Para garantir a eficiência e a padronização, a troca de informação seguirá as seguintes diretrizes:

- **Formato de Dados:** Todos os *payloads* (dados enviados) e respostas serão formatados em **JSON (JavaScript Object Notation)**.
- **Autenticação e Segurança:** Sendo a API *stateless* (sem estado), cada pedido feito pelas rotas protegidas (Tutor, Gestão ou Staff) deve incluir um token de autenticação JWT no cabeçalho HTTP (*Authorization: Bearer <token>*). O *backend* utiliza este *token* para resolver o RBAC e aceitar ou rejeitar o pedido.
- **Respostas HTTP Padrão:** O sistema fará uso dos códigos de estado HTTP semânticos: 200 OK para sucesso, 201 *Created* para novos registos (ex: novas reservas), 400 *Bad Request* para erros de validação, e 403 *Forbidden* caso um utilizador tenta aceder a uma rota para a qual não tem permissão (ex: *Staff* a tentar aceder a Faturas).

4.7.2. Documentação dos Endpoints Principais

Abaixo detalham-se os *endpoints* críticos que suportam os Casos de Uso centrais do hotel canino, refletindo a arquitetura final implementada e as integrações com infraestruturas *Cloud* (AWS S3):

1. Efetuar uma Nova Reserva

- **Endpoint:** POST */api/reservas*
- **Acesso:** Tutor, Gestão (Diana/Receção)
- **Descrição:** Submete um pedido de reserva para um animal específico. A *Facade* no *backend* cruza a disponibilidade física, valida o estado de limpeza da *box* (ignora *boxes* marcadas como "Sujas") e analisa a reatividade do animal antes de confirmar a alocação.
- **Exemplo de Request (JSON):**

JSON

```
{
  "animalId": "A-789",
  "dataEntrada": "2026-06-10T14:00:00Z",
  "dataSaida": "2026-06-15T10:00:00Z",
  "servicos": ["Grooming", "Passeio"]
}
```

- **Exemplo de Response (201 Created):**

JSON

```
{
```

```

"sucesso": true,
"idReserva": "RES-1045",
"boxAtribuida": 12,
"estado": "Pendente",
"valorEstimado": 150.00
}

```

2. Consultar Historial e Diário de Bordo do Animal

- **Endpoint:** GET /api/animais/:idAnimal/historial
- **Acesso:** Tutor, Gestão, Veterinário
- **Descrição:** Devolve a cronologia de eventos de um animal, cruzando os dados clínicos com a informação operacional. As fotografias capturadas pelo *staff* são devolvidas como URLs seguros provenientes do *bucket* AWS S3.
- **Exemplo de Response (200 OK):**

JSON

```

{
  "idAnimal": "A-789",
  "nome": "Bobi",
  "estadoClinico": "Saudavel",
  "diarioBordo": [
    {
      "idRegisto": "LOG-551",
      "timestamp": "2026-06-11T10:30:00Z",
      "descricao": " Passeio Diário - Concluído com muita energia!",
      "fotos":
        ["https://hotel-animais-assets.s3.eu-west-3.amazonaws.com/fotos/bobi-11-06.jpg"]
    }
  ]
}

```

3. Registrar Execução de Tarefa Diária (Staff)

- **Endpoint:** PATCH /api/tarefas/:idServico/concluir
- **Acesso:** Staff (Cuidadores)
- **Descrição:** Atualiza o estado de uma tarefa diária a partir da interface *Mobile-First*. Uma vez que suporta o envio nativo de fotografias da câmara do telemóvel, o protocolo de envio foi atualizado para multipart/form-data (em vez de JSON estrito).

- **Exemplo de Request (FormData):**

Plaintext

estado: "Finalizado"

nomeStaff: "Diogo Pais"

fotoProva: [Ficheiro Binário da Imagem .jpg]

- **Exemplo de Response (200 OK):**

JSON

```
{
  "sucesso": true,
  "mensagem": "Tarefa concluída com sucesso e registo fotográfico guardado no Diário de Bordo."
}
```

4. Processar Check-Out e Fatura Final

- **Endpoint:** POST /api/faturacao/reservas/:idReserva/checkout
- **Acesso:** Tutor, Recepção
- **Descrição:** Calcula o valor final da estadia e emite a fatura. Executa operações atómicas na base de dados: encerra a reserva, transita a respetiva *box* para o estado "Suja" (despoletando rotinas de manutenção) e atribui "Alta médica" caso o cão estivesse em quarentena.
- **Exemplo de Response (200 OK):**

JSON

```
{
  "sucesso": true,
  "idFatura": "FAT-2026-089",
  "nifCliente": "123456789",
  "valorTotal": 150.00,
  "acoesAutomaticas": [
    "Box 12 marcada como Suja.",
    "Alta médica atribuída ao animal."
  ],
  "linkDocumento":
    "https://hotel-animais-assets.s3.eu-west-3.amazonaws.com/faturas/FAT-2026-089.pdf"
}
```

4.8. Prompts e geração dos resultados

Nesta fase de definição arquitetural e modelação do sistema, os LLMs foram utilizados como agentes de apoio à decisão técnica e à prototipagem rápida de diagramas. Em vez de delegar a conceção total ao modelo, o grupo adotou uma postura de co-criação, onde o LLM gerou propostas base que foram posteriormente validadas e refinadas pela equipa, garantindo o cumprimento rigoroso dos requisitos do hotel canino.

4.8.1. Definição da Stack Tecnológica e Arquitetura Hexagonal

4.8.1.1. Prompt Inicial

"Atua como um Arquiteto de *Software* Sênior. Estamos a desenvolver um sistema de gestão para um hotel de animais (*frontend* em React, *backend* em Node.js/TypeScript e base de dados PostgreSQL). O sistema vai ter picos de tráfego intensos no verão (até 40 animais). Precisamos implementar uma Arquitetura Hexagonal no backend. Podes sugerir a estrutura de pastas e explicar como os '*Ports*' e '*Adapters*' se devem relacionar com a nossa lógica de domínio (Gestão de Reservas, Clínica e Faturação)?"

4.8.1.2. Avaliação Crítica do Resultado Inicial e Refinamento

Avaliação Crítica do Resultado	Refinamento Aplicado / Ação Tomada
Estrutura Base Bem Definida: O modelo forneceu uma árvore de diretórios bem estruturada, separando claramente o Core (domínio puro), os Ports (interfaces) e os Adapters (controladores Express e repositórios Prisma). Sugeriu igualmente e de forma correta a utilização do padrão Facade para a comunicação entre domínios.	Adoção Parcial: A estrutura gerada revelou-se globalmente satisfatória e, após a combinação com as intervenções humanas, serviu de base para a elaboração do Diagrama de Componentes final do sistema.

Falha no Isolamento de Dependências Externas: A análise crítica ao output revelou uma lacuna relevante: o LLM falhou em isolar a comunicação com a Autoridade Tributária num adaptador externo dedicado. Esta falha arquitetural poderia introduzir um acoplamento indesejado entre a lógica clínica do hotel e uma dependência de infraestrutura externa do Estado.	Intervenção Manual (FaturacaoAdapter): Face à limitação identificada, a equipa interveio manualmente na arquitetura e desenhou um <i>FaturacaoAdapter</i> dedicado. Esta ação garantiu que eventuais falhas na API do Estado nunca comprometessem a lógica clínica central do sistema.
---	---

Tabela 12: Prompts da Arquitetura e Stack Tecnológica .

4.8.2. Geração de Esboço Estrutural para o Diagrama de Classes

4.8.2.1. Prompt Inicial

"Com base na Arquitetura Hexagonal que definimos, gera o código PlantUML para o nosso Diagrama de Classes. Inclui as entidades principais (GestorHotelFacade, Reserva, Animal, Tutor, Funcionario, Fatura). Aplica o padrão DAO para a persistência. O GestorHotel deve delegar funções para subsistemas como GestReservasFacade e GestClinicaFacade. Usa os estereótipos adequados."

4.8.2.2. Avaliação Crítica do Resultado Inicial e Refinamento

Avaliação Crítica do Resultado	Refinamento Aplicado / Ação Tomada
Base Conceptual Funcional: O LLM gerou um script PlantUML funcional que mapeou corretamente a herança entre Utilizador, Tutor e Funcionario, e instanciou as classes DAO básicas.	Uso como Auxiliar de Raciocínio: Devido às limitações visuais e técnicas, o código gerado serviu estritamente como base conceptual e auxiliar de raciocínio, não sendo utilizado como artefacto final.

Falhas de Desenho Significativas: A proposta apresentava acoplamento excessivo através de relações bidirecionais potencialmente problemáticas entre as entidades Reserva e Animal. Além disso, apresentava uma representação confusa das partições de interface (IGestorHotel, IGestReservas), o que comprometia a correção técnica e a legibilidade do diagrama.	Modelação Manual (Descarte do Output): A equipa descartou o output do LLM. Procedeu-se à modelação manual e definitiva de todo o Diagrama de Classes no Visual Paradigm, assegurando que a navegação entre classes fluísse exclusivamente no sentido correto e que a segregação de interfaces fosse aplicada com o rigor exigido.
---	---

Tabela 13: Prompts do Diagrama de Classes.

4.8.3. Desenho da Infraestrutura Cloud (AWS Free Tier)

4.8.3.1. Prompt Inicial

"Preciso de desenhar a infraestrutura AWS para este hotel de animais. O sistema precisa de ter 99,9% de uptime e backups frequentes. Como o projeto é académico, a arquitetura deve usar estritamente recursos do AWS Free Tier. Quais os serviços ideais para alojar a app em React, a API em Node.js, a base de dados PostgreSQL e as fotografias do Diário de Bordo?"

4.8.3.2. Avaliação Crítica do Resultado Inicial e Refinamento

Avaliação Crítica do Resultado	Refinamento Aplicado / Ação Tomada
Resposta Altamente Precisa: O modelo apresentou um output perfeitamente alinhado com a visão arquitetural da equipa. A recomendação dos serviços AWS S3, instâncias EC2 (t3.micro) ou Elastic Beanstalk e Amazon RDS (db.t3.micro) validou integralmente as opções que já tinham sido equacionadas antes da submissão do prompt.	<i>Aceitação da Estrutura Base: O conteúdo gerado foi aceite como solução arquitetural válida sem necessidade de aplicar alterações estruturais de relevo nas recomendações dos serviços.</i>

Gestão de Limites do Free Tier: Apesar de a estrutura recomendada estar correta, identificou-se a necessidade de precaver o esgotamento prematuro do limite de 5 GB de armazenamento gratuito disponibilizado pelo AWS Free Tier.

Ajuste de Políticas de Retenção (Intervenção Manual): A única intervenção da equipa consistiu no ajuste das políticas de retenção para os buckets S3. Foi definido que os ficheiros multimédia do "Diário de Bordo" (fotografias e vídeos) seriam purgados automaticamente 30 dias após o check-out do animal.

Tabela 14: Prompts da Infraestrutura AWS.

A execução do capítulo 4 decorreu de forma controlada. A camada de dados na definição da arquitetura global registou um atraso de um dia, resultado do tempo necessário para consolidar as decisões em equipa. A análise comparativa da stack tecnológica prolongou-se igualmente um dia, dado o rigor exigido na justificação das escolhas entre alternativas. O diagrama de classes e os diagramas de sequência atrasaram um dia cada, justificado pelo volume e complexidade dos modelos produzidos. O módulo operacional do staff no design de interfaces precisou de um dia extra para incorporar o feedback da stakeholder Diana Silva. Todas as restantes secções, incluindo a análise de alternativas arquiteturais, a documentação dos endpoints e os três blocos de prompts e avaliações LLM, foram concluídas dentro do prazo. O capítulo ficou concluído a 23/03, respeitando o prazo da Etapa 2.

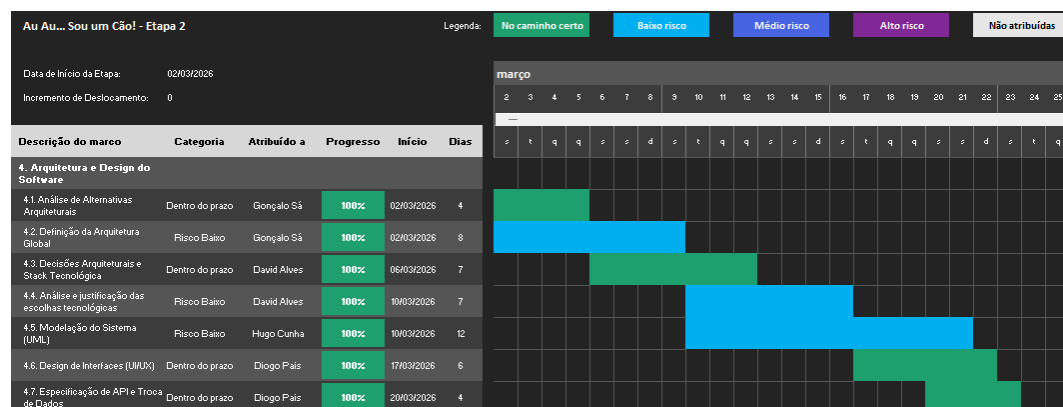


Figura 39: Diagrama de Gantt final – Capítulo 4.

5. Implementação e Desenvolvimento Assistido por LLM

Nesta fase do projeto, o foco transitou da concepção arquitetural para a materialização do software. O desenvolvimento, teste e a subsequente demonstração do protótipo foram realizados diretamente sobre a infraestrutura *cloud* projetada (AWS), garantindo a fidelidade absoluta entre o ambiente de implementação e a arquitetura de produção definida no Capítulo 4.

O desenvolvimento foi pautado por uma abordagem iterativa e incremental, onde a Inteligência Artificial (LLM) foi integrada como um assistente de programação para acelerar a tradução dos modelos conceituais (UML) em código estruturado, validado e otimizado para *deployment* na nuvem.

O capítulo 5 foi planeado para decorrer entre os dias 23/03 e 13/04, correspondendo à Etapa 3 de implementação. A figura seguinte apresenta o diagrama de Gantt objetivo para este capítulo.

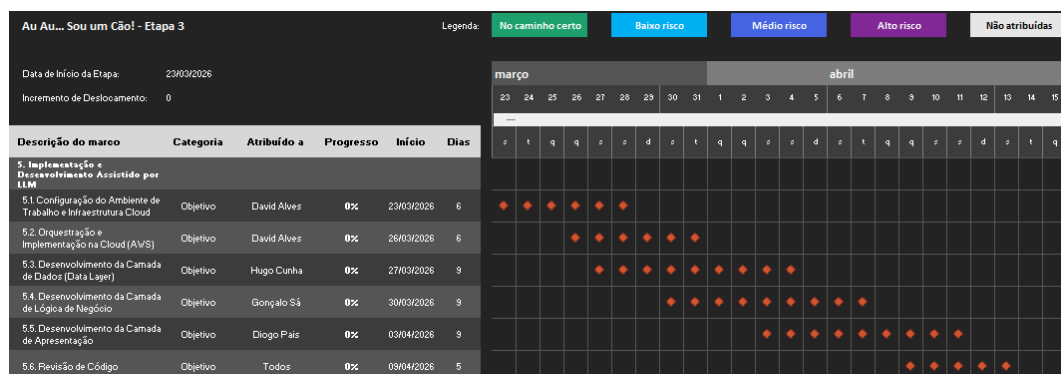


Figura 40: Diagrama de Gantt objetivo– Capítulo 5.

5.1. Configuração do Ambiente de Trabalho e Infraestrutura *Cloud*

Para assegurar a estabilidade do desenvolvimento modular e garantir que o código se mantinha coeso no momento de integração com a AWS, a primeira etapa consistiu na configuração rigorosa do ecossistema de trabalho:

- **Controlo de Versões e Estrutura (*Monorepo*):** O projeto foi inicializado utilizando Git para controlo de versões, adotando-se uma estrutura com pastas independentes para a Camada de Lógica de Negócio (*/backend*) e a Camada de Apresentação (*/frontend*).
- **Inicialização do Backend (Node.js/TypeScript):** A base do servidor foi configurada através do compilador de TypeScript para garantir tipagem estática rigorosa. Foram instaladas as dependências para a API REST (*Express*) e o ORM (*Prisma*), preparando a aplicação para ser alojada em instâncias AWS EC2 / Elastic Beanstalk.
- **Instanciação da Base de Dados (AWS RDS):** Configurou-se uma instância real de PostgreSQL através do serviço AWS RDS (nível *Free Tier*, utilizando instâncias *db.t3.micro*). O esquema da base de dados gerado pelo Prisma foi migrado diretamente para esta instância remota, garantindo a persistência centralizada desde o primeiro dia de desenvolvimento.
- **Integração de Armazenamento (AWS S3):** Para o manuseamento de ficheiros estáticos (fotografias do diário de bordo e PDFs), o ambiente foi ligado a um *bucket* do AWS S3, configurando as credenciais de acesso seguras (*Identity and Access Management (IAM) Roles*) diretamente no código da API.
- **Inicialização do Frontend (React.js):** A interface web foi construída utilizando *Vite*, pré-configurando bibliotecas fundamentais como o *React Router* e o *Axios* (configurado para consumir os *endpoints* públicos da API alojada na AWS).
- **Gestão de Tráfego e Comunicação Segura (AWS CloudFront):** Para garantir a segurança na comunicação entre a interface web (alojada no AWS Amplify) e a API (AWS Elastic Beanstalk), implementou-se o AWS CloudFront. Este serviço atuou como intermediário para gerir os certificados SSL/TLS, efetuando a tradução e o reencaminhamento de todo o tráfego de HTTP para HTTPS, assegurando assim a encriptação e a integridade dos dados em trânsito.
- **Padronização e Segurança:** As chaves de acesso à AWS e os segredos JWT foram isolados no ambiente virtual da AWS. Configuraram-se o ESLint e o Prettier para garantir a análise estática e a formatação automática do código gerado.

5.2. Orquestração e Implementação na *Cloud* (AWS)

A materialização da infraestrutura do sistema "Au Au... Sou um Cão!" foi integralmente realizada na AWS. Tirando partido do programa *Free Tier*, foi possível provisionar um ambiente de produção seguro, escalável e de baixo custo, que reflete fielmente a arquitetura de três camadas definida na Etapa 2. A orquestração dos vários serviços geridos da AWS exigiu um planeamento rigoroso ao nível de redes, acessos e automação de *deployments*.

5.2.1. Amazon RDS e Prisma: Persistência e Segurança de Dados

O armazenamento relacional foi assegurado através do serviço AWS RDS, provisionando-se uma instância PostgreSQL (db.t3.micro) ao abrigo do *Free Tier*. A segurança e a comunicação entre os recursos da *Cloud* foram geridas através de *Security Groups* e do AWS IAM. Em vez de expor credenciais no código, foi criada uma IAM *Role* dedicada, atribuída ao ambiente de alojamento do *backend*, que confere permissões seguras e diretas de acesso tanto à base de dados como aos buckets de armazenamento.

As variáveis de ambiente do sistema, incluindo as *connection strings* geradas para o Prisma ORM, foram configuradas de forma segura diretamente na consola da AWS, sendo ajustadas à medida que os ambientes de *backend* e base de dados eram estabelecidos. Para garantir a fiabilidade estrutural a cada nova versão, o processo de *deploy* foi configurado (através do ficheiro *Procfile*) para executar automaticamente as migrações do Prisma (*npx prisma migrate deploy*), assegurando que o esquema do PostgreSQL se mantém sempre em sincronia com a lógica de negócio atual.

5.2.2. Amazon S3: Gestão Eficiente e Segura de Media

Para evitar a sobrecarga da base de dados e assegurar a performance da aplicação, todos os ficheiros multimédia e documentos PDF foram delegados ao Amazon S3. A estrutura do *bucket* foi dividida em dois diretórios lógicos, com políticas de acesso distintas, refletindo as diferentes necessidades de privacidade do hotel:

- **Diretório de Fotografias (Público):** Destinado exclusivamente às fotografias partilhadas pelo *staff* no "Diário de Bordo". O acesso de leitura público permite aos tutores visualizarem os conteúdos de forma rápida e direta.
- **Diretório de Documentos (Privado):** Utilizado para alojar dados sensíveis, como faturas e os boletins de vacinas dos animais. O acesso é estritamente privado; quando o *staff* necessita de consultar um ficheiro, a API gera automaticamente um *Pre-Signed URL* (link temporário) com uma validade máxima de 15 minutos.

A integração entre o *backend* e o Amazon S3 flui de forma transparente e automática graças à IAM *Role* configurada, que concede ao servidor permissão total sobre os *buckets*. Embora as regras de ciclo de vida do S3 (*Lifecycle Rules*) para a eliminação automática de ficheiros antigos e otimização de custos não estejam ainda ativadas, encontram-se já planeadas para iterações futuras do sistema.

5.2.3. Alojamento, Distribuição e Integração Contínua

A separação física entre o *Frontend* e o *Backend* foi concretizada através de serviços otimizados para cada finalidade:

- **Camada de Apresentação (*Frontend*):** A SPA em React (Vite) foi alojada no AWS Amplify. Esta escolha permitiu estabelecer um fluxo de Integração e Entrega Contínuas em que o portal é reconstruído e atualizado automaticamente na *Cloud* sempre que é efetuado um push de código no repositório GitHub.
- **Camada de Lógica de Negócio (*Backend*):** A API em Node.js foi alojada no AWS *Elastic Beanstalk*, operando, para já, através de um processo de *deploy* manual das novas versões.

Durante a integração final destas duas camadas, surgiu um desafio arquitetural relacionado com o encaminhamento seguro de tráfego. Devido a limitações de configuração e emissão de certificados no contexto do *Free Tier*, verificou-se uma incompatibilidade de protocolos, em que o AWS Amplify operava via HTTPS e o Elastic Beanstalk respondia apenas em HTTP. Para resolver este conflito (que resultaria em erros de *Mixed Content* no *browser*), introduziu-se o serviço AWS CloudFront na arquitetura. Atuando como uma Content Delivery Network (CDN) e Reverse Proxy, o CloudFront ficou responsável por assegurar a terminação SSL e a tradução do tráfego entre HTTPS e HTTP, garantindo que toda a comunicação entre o dispositivo do cliente e os servidores do hotel se mantém segura e funcional.

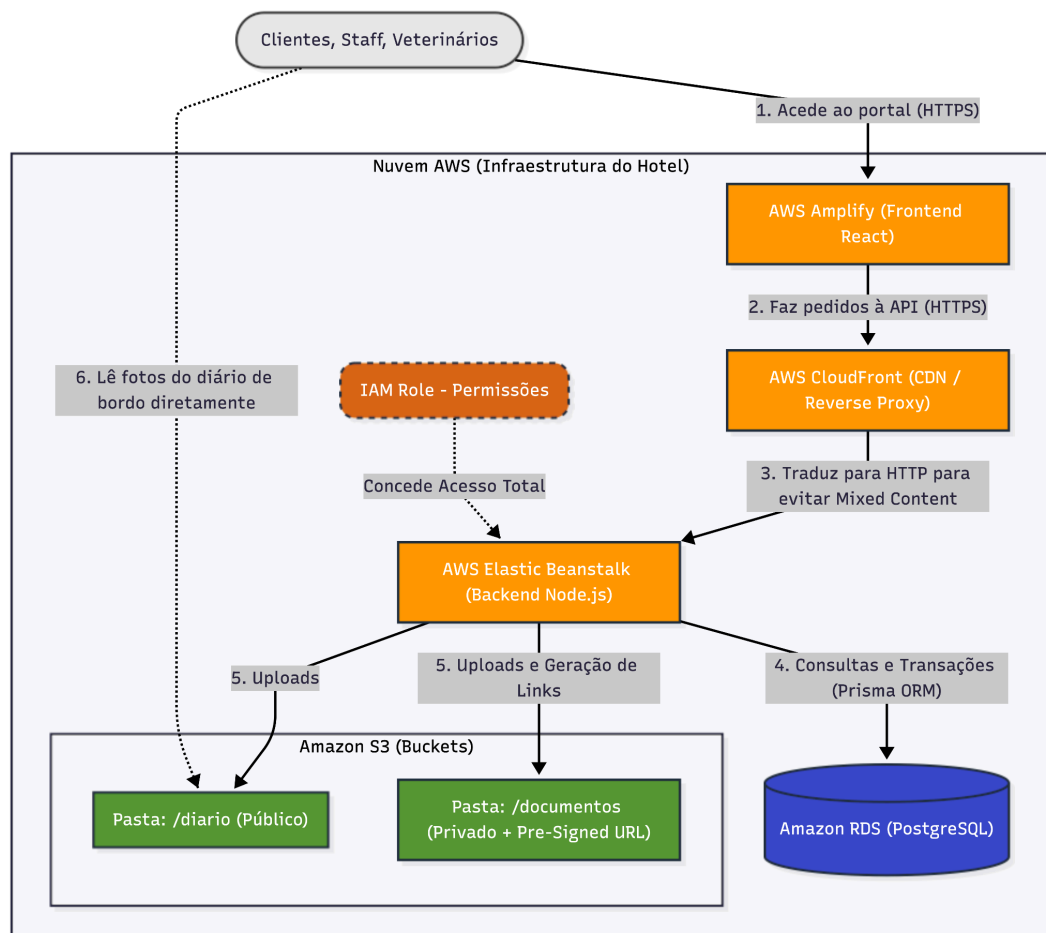


Figura 41: Esquema da arquitetura cloud.

5.3. Desenvolvimento da Camada de Dados (*Data Layer*)

A Camada de Dados do sistema "Au Au... Sou um Cão!" tem a responsabilidade estrita de garantir a persistência, integridade e recuperação eficiente da informação. Atuando como a fronteira de saída (*Driven Adapters*) na arquitetura hexagonal da aplicação, esta camada foi desenvolvida com o objetivo de abstrair a complexidade da infraestrutura (PostgreSQL e Amazon S3) face à lógica de negócio, assegurando um acoplamento mínimo.

5.3.1. Modelação e Esquema Lógico da Base de Dados

A base de dados relacional do sistema foi modelada com recurso ao PostgreSQL, gerido através do Amazon RDS, e a sua estrutura declarativa encontra-se

definida no ficheiro schema.prisma. A conceção do modelo de dados partiu diretamente dos requisitos funcionais eliciados junto dos *stakeholders* e procurou refletir com fidelidade as entidades do domínio operacional do hotel, as suas relações e as restrições de integridade que garantem a segurança clínica e financeira do negócio.

O esquema lógico é composto pelas seguintes entidades principais, as suas chaves e as relações entre si:

Entidade	Atributos Chave	Descrição
Utilizador	id (PK), email(UNIQUE), password, role	Entidade base para todos os perfis do sistema (Tutor, Funcionário, Veterinária, Gestora). A discriminação de perfis é feita pelo atributo role, que implementa o controlo de acessos RBAC (RNF.02).
Tutor	id (PK, FK → Utilizador), nome, nif, contacto	Especialização de Utilizador para os donos de animais, com informação fiscal (NIF) obrigatória para a emissão de fatura certificada (RF.36).
Funcionario	id (PK, FK → Utilizador), nome, cargo	Especialização de Utilizador para o <i>staff</i> operacional e equipa veterinária.
Animal	id (PK), nome, raca, microchip (UNIQUE), reatividade, alergias, idTutor (FK → Tutor)	Entidade central do domínio. O campo <i>microchip</i> é único e obrigatório (RF.18) e o campo reatividade alimenta o algoritmo de alocação automática de boxes (RF.06).
PlanoVacinal	id (PK), idAnimal (FK), vacina, dataAdministracao, dataValidade	Garante a rastreabilidade do estado vacinal de cada animal, cujo conteúdo é verificado antes de qualquer <i>check-in</i> (RF.09).
Box	id (PK), numero, estado (LIVRE, OCUPADA, EM_LIMPEZA), capacidade	Representa os alojamentos físicos do hotel. O atributo estado permite a implementação do fluxo de higienização (RF.31) e do mapa de alocação visual (RF.32).
Reserva	id (PK), idAnimal (FK), idBox (FK), dataEntrada, dataSaida, estado, precoTotal	Entidade que agrega o ciclo de vida completo de uma estadia, desde a criação até ao <i>check-out</i> . A validação

		de sobreposição de datas e o controlo de <i>overbooking</i> (RF.02, RF.03) são aplicados na criação desta entidade.
Servico	id (PK), idReserva (FK), tipo, preco	Regista os serviços extra adquiridos por reserva (banhos, tosas, passeios — RF.29), permitindo a consolidação automática na fatura final.
Prescricao	id (PK), idAnimal (FK), idFuncionario (FK), medicamento, dose, horario	Materializa a prescrição clínica digital (RF.20, RF.21). A associação ao Funcionário que prescreveu garante a auditabilidade clínica exigida.
LogMedicacao	id (PK), idPrescricao(FK), idFuncionario(FK), timestamp, observacoes	Registo imutável de cada administração de medicação (RF.22). O campo <i>timestamp</i> é gerado automaticamente pelo sistema e não pode ser editado, assegurando a rastreabilidade legal e clínica.
DiarioBordo	id (PK), idAnimal (FK), idReserva (FK), descricao, mediaUrl, dataRegisto	Entidade que suporta a funcionalidade de timeline do tutor (RF.37). O campo <i>mediaUrl</i> referencia a localização do ficheiro no Amazon S3 (pasta diario/).
Stock	id (PK), nome, tipo, quantidade, limiteAlerta	Controlo do inventário clínico (RF.26). Quando a quantidade desce abaixo do <i>limiteAlerta</i> , o sistema aciona um cron job de alerta automático (secção 5.4.3).
Fatura	id (PK), idReserva (FK), idTutor (FK), valorTotal, dataEmissao, pdfUrl	Persiste os dados fiscais consolidados para certificação pela AT (RF.36, RNF.05). O <i>pdfUrl</i> referencia o documento gerado e armazenado na pasta privada documentos/ do S3.

Tabela 15: Entidades Principais da Base de Dados.

5.3.2.1. Relações Principais

O modelo relacional assenta nos seguintes relacionamentos estruturantes:

- Tutor — Animal (1:N): Um tutor pode registar vários animais, mas cada animal pertence a um único tutor.

- Animal — Reserva (1:N): Um animal pode ter múltiplas reservas ao longo do tempo (histórico de estadias), mas cada reserva é relativa a um único animal.
- Reserva — Box (N:1): Múltiplas reservas podem ocorrer na mesma box em períodos diferentes; num dado período, uma box só pode estar associada a uma reserva ativa, garantindo a exclusividade do alojamento.
- Reserva — Servico (1:N): Uma reserva pode incluir zero ou vários serviços extra, os quais são consolidados na fatura final.
- Animal — Prescricao (1:N): Um animal pode ter múltiplas prescrições ativas em simultâneo (ex.: medicação para epilepsia e dieta terapêutica).
- Prescricao — LogMedicacao (1:N): Cada plano de prescrição gera múltiplos registos de administração ao longo da estadia, formando um histórico auditável e imutável.
- Reserva — DiarioBordo (1:N): Cada reserva está associada a múltiplas entradas no diário de bordo, construindo a timeline diária que é partilhada com o tutor.
- Reserva — Fatura (1:1): No momento do check-out, é emitida exatamente uma fatura por reserva, consolidando todos os serviços e diárias.

5.3.2.2. Escolhas de Design

Três decisões merecem destaque particular na conceção do modelo:

1. Herança de Utilizador por Composição: Em vez de usar herança de tabelas (*table-per-hierarchy*), optou-se por uma relação de composição entre Utilizador (entidade base com autenticação) e Tutor/Funcionario (entidades de perfil com dados específicos). Esta abordagem simplifica as queries de autenticação e mantém a separação de dados pessoais dos dados operacionais.
2. Imutabilidade do LogMedicacao: A entidade LogMedicacao foi desenhada para não permitir operações de UPDATE ou DELETE ao nível da aplicação. Esta restrição, implementada na camada DAO (ausência intencional de métodos de atualização no LogsDAO), garante a integridade do registo clínico e cumpre o requisito de auditabilidade por um período mínimo de três anos (RNF.04).
3. Segregação de Media no S3: A decisão de não armazenar ficheiros binários (imagens, PDFs) diretamente na base de dados PostgreSQL, substituindo-os por referências URL para o Amazon S3, garante a escalabilidade do modelo relacional e mantém a latência das queries dentro do limite de 2 segundos exigido (RNF.06), independentemente do volume de media gerado durante as estadias.

5.3.2.3. Modelo Lógico

De forma a ilustrar as diferentes entidades do sistema e as suas respetivas ligações, utilizou-se o *software DBaiver* para desenhar e gerar o esquema lógico da base de dados num Diagrama Entidade-Relacionamento.



Figura 42: Diagrama Entidade-Relacionamento.

5.3.2. Abstração Relacional com Prisma ORM e Padrão DAO

O acesso à base de dados relacional (AWS RDS) foi estruturado através do padrão arquitetural DAO. Para evitar o esgotamento do pool de conexões ao PostgreSQL, o *PrismaClient* é instanciado de forma global (padrão *Singleton*) e injetado em cada classe DAO. O Prisma foi utilizado primordialmente para a modelação declarativa das entidades (*schema.prisma*), enquanto os DAOs encapsulam todas as operações de leitura e escrita (*gets* e *uploads*).

Data Access Object (DAO)	Entidades Geridas (Prisma)	Responsabilidade Principal
ReservaDAO	Reserva, Box, Servico, Fatura	Gestão do ciclo de vida das reservas, validação de capacidade das boxes, prevenção de overbooking e alocação automática com base na reatividade.
AnimalDAO	Animal, PlanoVacinal, DiarioBordo	Registo de perfis de animais, verificação de planos de vacinação e consulta do historial completo de estadias e serviços.
CheckDiarioDAO	DiarioBordo, Animal	Gestão das rotinas diárias, registo de avaliações veterinárias e controlo de transição de estados de saúde (ex: ativação de Quarentena).
UtilizadorDAO	Utilizador, Tutor, Funcionario	Gestão de perfis e autenticação, criação de contas e separação lógica entre clientes e <i>staff</i> .
StockDAO	Stock, Medicamento	Controlo de inventário clínico e atualização de quantidades consumidas de fármacos e rações terapêuticas.
PrescricaoDAO	Prescricao	Registo de planos de medicação associados a animais específicos.
DiarioBordoDAO	DiarioBordo	Registo imutável de descrições e associação de media (fotografias/vídeos) ao perfil do hóspede.
FaturaDAO	Faturas	Persistência dos dados financeiros consolidados associados ao NIF do cliente para posterior certificação.
LogsDAO	LogMedicacao	Registo de auditoria (timestamp e autoria) das administrações de medicação pelo staff.

Tabela 16: Resumo dos Data Access Objects (DAOs) implementados.

A arquitetura implementada inclui vários DAOs altamente especializados. Esta abstração permitiu traduzir de forma elegante consultas(*queries*) complexas para o modelo relacional. A título de exemplo:

- **Consultas Relacionais Profundas:** No método `findAll()` do `ReservaDAO`, o Prisma é utilizado para encadear múltiplas tabelas numa única operação à base de dados. Num só pedido, o sistema recupera os dados da Reserva, interligando o Animal, o Tutor, o Utilizador, a Box, os Serviços e a Fatura.
- **Regras de Domínio Físico:** Destaca-se também a implementação do método `atribuirBoxAutomaticamente()` e `verificarCapacidadeBox()` no `ReservaDAO`, que iteram sobre os alojamentos do hotel e aplicam restrições físicas, validando a disponibilidade no período e garantindo a prevenção de overbooking.

```
1  async verificarCapacidadeBox(numeroBox: number, entrada: Date, saida: Date) {
2    const box = await prisma.box.findUnique({
3      where: { numero: numeroBox }
4    });
5
6    if (!box) {
7      return { disponivel: false, motivo: `Box ${numeroBox} não existe.` };
8    }
9
10   // Valida se a box está higienizada
11   if (box.estado !== 'Higienizada') {
12     return {
13       disponivel: false,
14       motivo: `Box ${numeroBox} está ${box.estado} - não disponível para check-in.`
15     };
16   }
17
18   // Conta quantos animais já estão nesta box no período pedido
19   const animaisJaLa = await this.contarAnimaisEmBox(numeroBox, entrada, saida);
20   const espacosLivres = box.tamanho - animaisJaLa;
21
22   if (espacosLivres <= 0) {
23     return {
24       disponivel: false,
25       motivo: `Box ${numeroBox} está cheia (${animaisJaLa}/${box.tamanho} espaços).`
26     };
27   }
28
29   return {
30     disponivel: true,
31     espacosLivres,
32     tamanho: box.tamanho,
33     ocupacao: animaisJaLa
34   };
35 }
```

Figura 43: Exemplo de regra implementada.

A resiliência das operações com a base de dados é mantida através de rigorosos blocos *try/catch* (encontrados unicamente na *api.ts*) na comunicação assíncrona. Adicionalmente, quando o sistema detecta anomalias lógicas (ex: tentar registar um

relatório diário num animal sem diário de bordo pré-existente no CheckDiarioDAO), a execução é imediatamente interrompida, lançando exceções claras (*throw new Error(...)*) para que a Camada de Lógica de Negócio consiga lidar e informar adequadamente a Camada de Apresentação.

5.3.3. Integração de Armazenamento com S3StorageAdapter

A persistência de ficheiros não-estruturados, como imagens e documentos PDF, encontra-se totalmente segregada da base de dados e centralizada no Amazon S3. A integração com este serviço é intermediada de forma transparente pela classe S3StorageAdapter, que recorre à biblioteca oficial *@aws-sdk/client-s3*.

Atendendo aos rigorosos requisitos de privacidade clínica e faturação do hotel, o *bucket* da AWS foi estruturado logicamente nestas duas áreas com políticas de acesso distintas, geridas nativamente pelo adaptador:

- **Pasta diário/ (Acesso Público):** Destinada exclusivamente aos ficheiros multimédia gerados no "Diário de Bordo". O método `uploadFicheiro` constrói e devolve imediatamente a URL pública (`https://[bucket].s3.amazonaws.com/[https://.s3.amazonaws.com/)...`), permitindo que a aplicação frontend renderize as fotografias de forma imediata aos tutores, com latência reduzida.
- **Pasta documentos/ (Acesso Privado):** Destinada aos boletins de vacinação e faturas. Para garantir a segurança dos dados dos clientes, a ligação é mantida estritamente privada. O adaptador armazena apenas a chave interna (*Key*) do S3. Quando o staff ou o cliente requer acesso a estes documentos, o adaptador invoca o método `gerarLinkTemporario`, recorrendo à função `getSignedUrl` da AWS, emitindo um "Passe VIP" criptográfico válido por uma curta duração (configurado para 15 minutos (900 segundos)). Esgotado este tempo, o *link* expira, garantindo a proteção e integridade dos documentos.

```
1  async gerarLinkTemporario(chaveS3: string): Promise<string> {
2      const command = new GetObjectCommand({
3          Bucket: this.bucketName,
4          Key: chaveS3,
5      });
```

Figura 44: Código de link temporário.

A comunicação segura deste adaptador com a *Cloud* dispensa a injeção manual de chaves de acesso vulneráveis no código, validando os acessos silenciosamente através da *IAM Role* do *Elastic Beanstalk* previamente orquestrada. Planos futuros para iterações do sistema preveem ainda a ativação de *Lifecycle Rules* ao nível do S3 para purga automatizada, garantindo que o armazenamento do "Diário de Bordo" não exceda os limites do *Free Tier*.

5.4. Desenvolvimento da Camada de Lógica de Negócio

A Camada de Lógica de Negócio atua como o "cérebro" do sistema "Au Au... Sou um Cão!". Desenvolvida em Node.js com TypeScript, esta camada materializa a Arquitetura Hexagonal, isolando as regras de negócio puras de quaisquer dependências externas. A organização interna do código foca-se na separação rigorosa de responsabilidades, segurança e automatização de processos operacionais que, até então, consumiam horas de trabalho manual à equipa do hotel.

5.4.1. O Núcleo Lógico e o Padrão Facade

Para garantir um baixo acoplamento e respeitar o Princípio de Responsabilidade Única (*Single Responsibility Principle*), a lógica de domínio foi segmentada em múltiplos subsistemas, cada um encapsulado por uma classe com o padrão *Facade*. A classe *GestorHotelFacade* atua como o orquestrador principal, servindo de ponto de entrada único para a Camada de Apresentação e encaminhando os pedidos para os subsistemas especialistas adequados.

Facade de Domínio	Responsabilidade Principal no Sistema
GestorHotelFacade	Controlador central que orquestra a comunicação entre a API e as <i>facades</i> específicas, garantindo que as fronteiras lógicas não são violadas.
GestReservasFacade	Motor de alocação que gere a criação de reservas, aplicando restrições dinâmicas e o algoritmo de atribuição automática de boxes consoante a reatividade do animal. Motor de alocação que gere a criação de reservas, aplicando restrições dinâmicas e o algoritmo

	de atribuição automática de boxes consoante a reatividade do animal.
GestClinicaFacade	Responsável pelo protocolo de segurança de saúde, incluindo prescrição de medicação com dedução síncrona de inventário e gestão de estados de quarentena.
GestHospedesFacade	Centraliza a criação de contas de utilizadores (aplicando regras de formatação de e-mail e validação de NIF) e a gestão de planos vacinais.
GestFaturacaoFacade	Calcula os custos da estadia e serviços extra (aplicando <i>Peak Pricing</i>) e prepara a emissão da fatura consolidada no final do <i>Check-Out</i> .
GestOperacoesFacade	Regula o "Diário de Bordo" e a gestão de <i>staff</i> , assegurando que cada registo diário possui, obrigatoriamente, conteúdo textual ou multimédia.

Tabela 17: Resumo das Facades implementadas e respetivas responsabilidades.

Uma das implementações mais complexas e críticas para a viabilidade do negócio reside no método `prescreverMedicacao` da `GestClinicaFacade`. Este método não se limita a registar a receita médica; ele efetua um cálculo transacional do consumo exigido para todo o tratamento (ex: $\text{dosagem} * \text{totalDoses}$) e interroga o `StockDAO`. Se o stock disponível for inferior à necessidade do tratamento, o sistema bloqueia a prescrição imediatamente, emitindo um erro claro à equipa veterinária e prevenindo falhas no tratamento clínico dos animais.

5.4.2. Segurança e Controlo de Acessos (*Middleware*)

Sendo um sistema que partilha a mesma infraestrutura API entre Clientes, *Staff* e Médicos Veterinários, a segurança no encaminhamento de tráfego é imperativa. A autenticação foi implementada com JWT no `middleware auth.ts`. A cada pedido HTTP efetuado pelas rotas protegidas, o `middleware` intercepta o cabeçalho *Authorization*, extrai o *Bearer token* e verifica a sua assinatura criptográfica.

```

1  try {
2    // 3. Tentar descriptar o token com a nossa chave secreta
3    const segredo = process.env.JWT_SECRET || 'chave_secreta_hotel_canino_2026';
4    const utilizadorDecodificado = jwt.verify(token, segredo);
5
6    // 4. Guardar a info do utilizador no pedido e abrir a porta (next)
7    req.user = utilizadorDecodificado;
8    next();
9  } catch (error) {
10   return res.status(403).json({ error: 'Sessão inválida ou expirada. Faça login novamente.' });
11  }

```

Figura 45: Middleware de autenticação JWT.

Em paralelo à autenticação, o sistema integra uma robusta camada de autorização. Uma vez decodificado o *token*, a API reconhece o perfil do utilizador. O sistema bloqueia ativamente qualquer tentativa de acesso não autorizada na origem; por exemplo, se uma conta do tipo "Tutor" tentar invocar o endpoint de "Criar Prescrição" (exclusivo para Veterinários) ou se o "Staff" tentar aceder ao módulo de Faturação, o *middleware* interrompe a operação e devolve um código HTTP 403 Forbidden, garantindo uma segregação de privilégios estanque.

5.4.3. Automatização de Tarefas (Cron Jobs)

A transição de uma gestão analógica para uma infraestrutura digital trouxe a oportunidade de automatizar tarefas diárias repetitivas. Para este efeito, utilizou-se a biblioteca *node-cron* para calendarizar processos assíncronos no *backend*, atuando diretamente na otimização de recursos. Destaca-se a seguinte automatização:

- **Limpeza de Reservas Caducadas** (*ReservaJobs.ts*): Executado diariamente às 00:00. O sistema procura reservas com estado "Pendente" cuja data de entrada já foi ultrapassada. Estas reservas são imediatamente atualizadas para o estado "Cancelada", libertando automaticamente as *boxes* trancadas e maximizando a disponibilidade para novos clientes.

5.4.4. Integração de Serviços de Terceiros

A Camada de Lógica de Negócio expande a sua utilidade através da integração com serviços utilitários modulares, desenhados para melhorar a comunicação e a segurança da conta do utilizador:

- **EmailService:** Construído sobre o protocolo *Simple Mail Transfer Protocol* (SMTP) com recurso à biblioteca *nodemailer*, este serviço unifica a comunicação

externa. O sistema aciona o disparo automático de e-mails de forma dinâmica (suportando formatos em texto simples e HTML) em momentos cruciais da jornada do cliente, como o envio da Fatura final após o *Check-Out* ou confirmações de reserva.

- **TwoFactorService:** Essencial para a proteção de dados clínicos e fiscais. Gera um código criptográfico numérico de 5 dígitos (ex: 74871) temporário que é armazenado em memória transacional com um prazo de validade rígido de 10 minutos (ExpiresAt). A expiração deste código invalida proativamente tentativas de intrusão, cumprindo as diretrizes de segurança adequadas à gestão de contas.



Figura 46: Exemplo de email de 2FA.

5.5. Desenvolvimento da Camada de Apresentação

A camada de apresentação do sistema "Au Au... Sou um Cão!" foi desenvolvida em React com TypeScript, construída com o *bundler* Vite e alojada num repositório *GitHub* com distribuição via AWS Amplify, em linha com as decisões arquiteturais definidas na secção 4.3.2. Esta combinação tecnológica permitiu uma iteração de desenvolvimento ágil e um *bundle* de produção otimizado para os diferentes contextos de utilização identificados no sistema.

A aplicação comunica exclusivamente com o *backend* através da API REST documentada na secção 4.7, utilizando o mecanismo nativo fetch do browser para todas as chamadas assíncronas. A autenticação é gerida via JWT: após o processo de login

com 2FA, descrito na secção 5.4.2, o token é armazenado em memória da aplicação e incluído no cabeçalho *Authorization: Bearer* em todas as chamadas subsequentes.

5.5.1. Estrutura da Aplicação e Roteamento

A aplicação *frontend* foi organizada de forma modular, separando componentes reutilizáveis, páginas por perfil funcional e mecanismos de controlo de acesso. Os componentes partilhados, centralizados em *src/components/*, incluem o *Header.tsx* (barra de navegação contextual ao perfil autenticado, com acesso ao *logout*), o *Footer.tsx* (rodapé institucional) e o ficheiro *Shared.css*, que define as variáveis de estilo globais, tipografia base e classes utilitárias transversais a toda a aplicação, garantindo a consistência visual do sistema.

O roteamento é gerido com *React Router*, com todas as rotas definidas no componente raiz *App.tsx*. Para proteger as áreas autenticadas, foi implementado o componente *ProtectedRoute.tsx*, que intercepta a navegação e verifica a validade do JWT antes de renderizar qualquer conteúdo protegido. Caso o *token* esteja ausente ou inválido, o utilizador é redirecionado automaticamente para a página de *login*, assegurando o cumprimento da política de segurança e segregação de perfis definida nos requisitos não funcionais (RNF.02).

5.5.2. Validação Visual com a Stakeholder e Refinamento da Interface

Antes do início do desenvolvimento efetivo das páginas, a equipa apresentou à *stakeholder* Diana Silva um conjunto de protótipos elaborados em *Figma*, correspondentes às interfaces desenhadas na secção 4.6 deste relatório. O objetivo foi validar, junto da principal utilizadora do sistema, se a estrutura visual proposta respondia às suas expectativas funcionais e à identidade que pretendia para a plataforma.

Na sequência dessa reunião de validação, a equipa recebeu o seguinte *feedback* por *email*:

De: Diana Silva

Assunto: Feedback sobre os mockups — Sistema Au Au... Sou um Cão!

Olá equipa,

Estive a analisar com atenção os mockups que me mostraram no Figma e, de uma forma geral, gostei bastante do resultado. A organização das páginas faz sentido, a separação entre a área da receção, da veterinária e do staff está bem pensada, e consegui perceber facilmente como o sistema vai funcionar no dia a dia. Acho que já reflete muito bem a complexidade do que fazemos aqui no hotel.

Ainda assim, e sendo honesta, sinto que algumas páginas têm um aspeto demasiado "administrativo" e pouco apelativo visualmente. Gostava que a interface transmitisse algo mais moderno e atual — com cores mais suaves e harmoniosas, cartões com mais espaçamento e elegância, melhor destaque visual para a informação importante, e uma sensação mais próxima de uma aplicação web contemporânea do que de um sistema de backoffice clássico.

Também me parece importante que o sistema não perca a ligação ao que somos: um hotel de animais com uma componente afetiva muito forte. Não quero que a plataforma pareça demasiado clínica ou impessoal — deve sentir-se como uma extensão do cuidado que prestamos aqui.

No geral estou satisfeita com a base que apresentaram e com a direção do projeto. Com estes ajustes acredito que o produto final vai ficar muito mais apelativo e diferenciador.

Obrigada pelo trabalho,
Diana Silva

Este *feedback*, recebido após a validação dos protótipos, foi determinante para orientar o desenvolvimento real do *frontend*. A equipa incorporou as sugestões da Diana numa fase de refinamento visual anterior à implementação definitiva das páginas, procedendo aos seguintes ajustes: modernização da paleta cromática com tons mais suaves e coerentes com a identidade do hotel, reorganização dos blocos informativos em cartões com maior espaçamento e hierarquia visual mais clara, e introdução de elementos visuais que reforçam a ligação ao domínio dos animais, tornando a experiência global mais acolhedora e menos impessoal. Estas alterações foram implementadas diretamente no código, após nova validação com a *stakeholder*, que aprovou a direção tomada.

5.5.3. Módulo de Autenticação

O módulo de autenticação é o ponto de entrada do sistema para todos os utilizadores e compreende duas páginas distintas: `LoginPage.tsx` e `RegisterPage.tsx`, ambas partilhando o ficheiro de estilos `AuthPages.css`.

5.5.3.1 Página de *Login* (LoginPage.tsx)

A página de *login* implementa o fluxo de autenticação em dois passos descrito na secção 5.4.2. Numa primeira fase, o utilizador submete as suas credenciais (*email* e *palavra-passe*), que são validadas pelo *backend*. Em caso de sucesso, e dado que o sistema lida com dados sensíveis de saúde animal, é apresentado um segundo ecrã para introdução do código de 2FA, enviado para o email do utilizador pelo *TwoFactorService*. Após validação do código, o JWT é emitido e a navegação é redirecionada para o módulo correspondente ao perfil do utilizador autenticado.

The screenshot displays the 'Página de LOGIN' interface. At the top left is the 'Au Au sou um cão' logo. The header bar is teal with the text 'Página de LOGIN' and a 'Login' button with a right arrow icon. The main content area features a light gray box titled 'Iniciar Sessão'. Inside this box, there are input fields for 'Email:' (containing 'tutor@auau.pt') and 'Password:' (masked with dots). Below the password field is a 'Login' button. A link 'Registe-se aqui' is provided for users who do not have an account. The footer contains three columns of links: 'Conheça-nos' (Sobre Nós, Opiniões de Clientes), 'Serviços' (O nosso Hotel, Clínica Veterinária, Rações Premium), and 'Ajuda' (Contacte-nos, Perguntas Frequentes, Fazer Reservas). Social media icons for Facebook, Twitter, and Instagram are also present.

Figura 47: *Página de Login.*

This screenshot shows the second step of the login process, 'Verificação 2FA'. The header and footer are identical to the previous page. The central light gray box is titled 'Verificação 2FA' and contains the instruction: 'Insira o código de 5 dígitos enviado para davidalves2862005@gmail.com'. Below this is a 'Código:' input field containing the value '72565'. A 'Verificar' button is positioned below the input field, with a 'Voltar ao Login' link underneath it.

Figura 48: *Página de Login com o 2FA.*

5.5.3.2. Página de Registo (RegisterPage.tsx)

A página de registo é acessível apenas a novos tutores e implementa o formulário de criação de conta, com validação de campos obrigatórios no lado do cliente antes da submissão à API. São validados dados como o formato de email, a existência de NIF e o contacto de emergência, garantindo que o perfil criado satisfaz os requisitos mínimos para uma futura reserva sem erros no *check-in* (RF.09, RF.18).

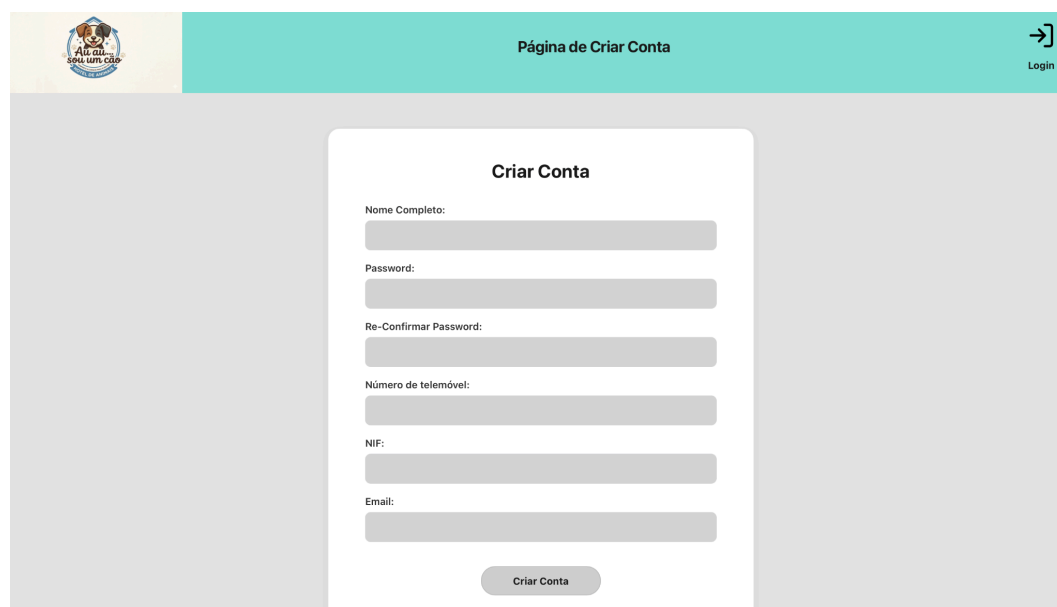


Figura 49: Página de Registo.

5.5.4. Página Pública - Homepage (HomePage.tsx)

A homepage é a face institucional do hotel, acessível sem autenticação, e constitui o ponto de contacto inicial entre novos visitantes e o sistema. Apresenta os serviços oferecidos pelo hotel, uma secção "Sobre Nós" com a história do projeto e da sua fundadora, uma galeria fotográfica das instalações e uma área de testemunhos de clientes (RF.42, RF.43). Esta página foi desenvolvida com especial atenção ao design responsivo, dado que a maioria dos potenciais clientes acede via dispositivo móvel. Reflete também as preocupações de identidade visual levantadas pela Diana no *email de feedback*: foi das páginas onde mais se investiu na componente estética, procurando transmitir acolhimento, confiança e proximidade ao domínio dos animais.

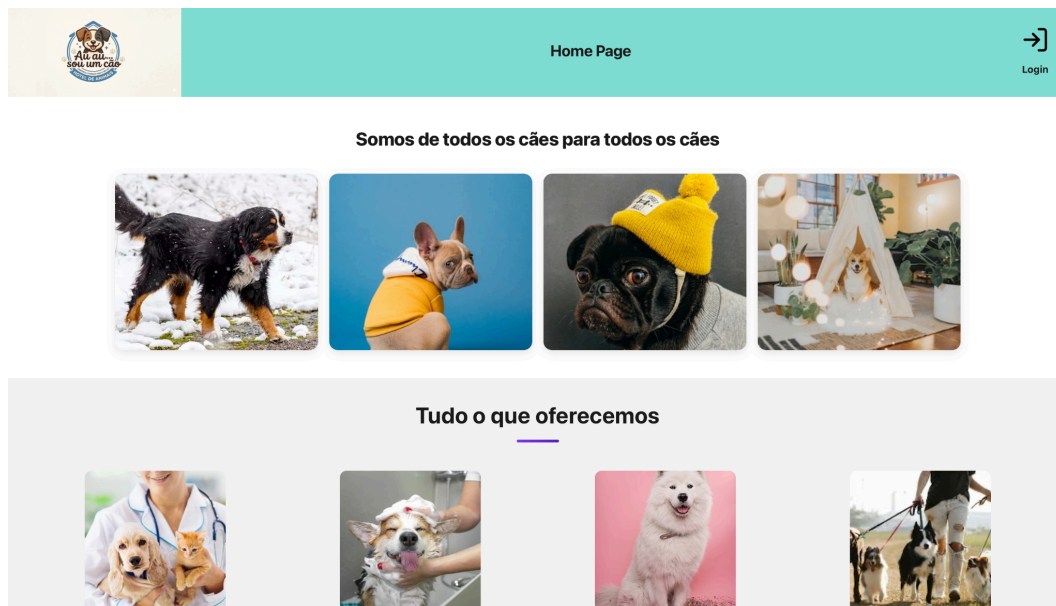


Figura 50: Home Page.

5.5.5. Módulo da Gestora - Dashboard da Gestora (GestoraPage.tsx e GatewayGestoraPage.tsx)

O módulo da gestora é o painel de supervisão e controlo central do sistema, concebido para uso da Diana no seu papel de gestora e veterinária. O GatewayGestoraPage.tsx funciona como ecrã de seleção de área funcional, apresentando os diferentes domínios de gestão de forma clara e organizada, antes de redirecionar para o dashboard principal.

A GestoraPage.tsx agrega as funcionalidades de gestão de topo: dashboards de ocupação e rentabilidade com indicadores visuais em tempo real (RF.41), controlo de stock clínico com alertas automáticos de rutura (RF.26), gestão de utilizadores e perfis de staff, e acesso ao histórico financeiro com geração de relatórios exportáveis (RF.20). A interface foi desenhada para desktop, priorizando a densidade de informação sem sacrificar a leitura imediata do estado global do hotel.

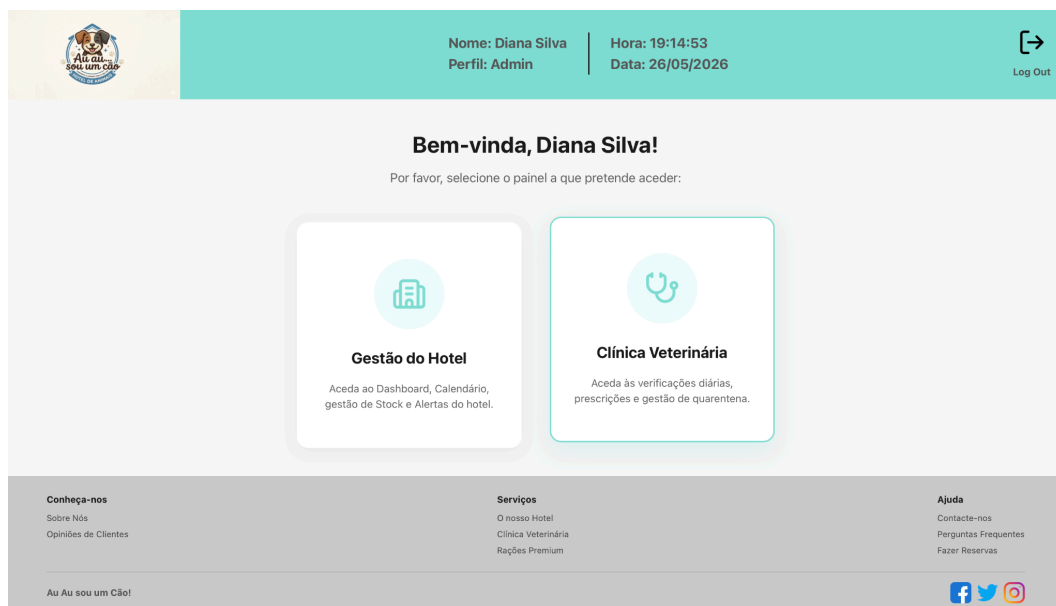


Figura 51: Página de escolha da Gestora.

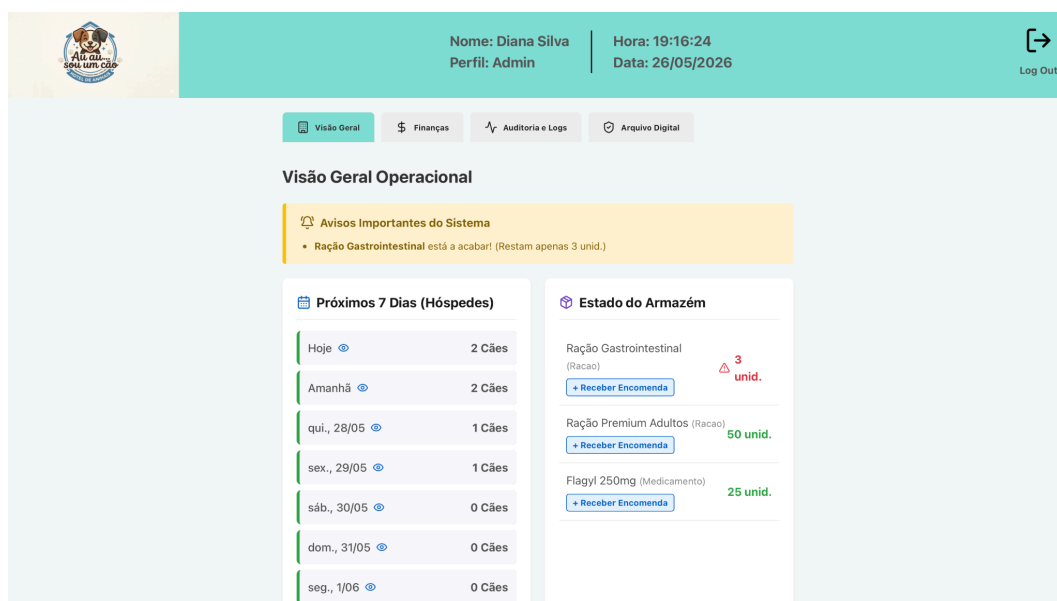


Figura 52: Página exclusiva da Gestora.

5.5.6. Módulo da Receção (RececaoPage.tsx)

O módulo da receção suporta as operações diárias do balcão de atendimento e foi concebido para uso intensivo durante os momentos de maior pressão operacional: os processos de *check-in* e *check-out*. A interface apresenta a ficha completa do animal no momento da chegada, incluindo o estado vacinal atualizado, alergias registadas e observações clínicas relevantes. Caso as vacinas do animal se encontrem caducadas, o

sistema bloqueia visualmente a progressão do *check-in*, cumprindo o requisito RF.09 e eliminando o risco de erro humano documentado na fase de observação direta (secção 3.3.1).

No check-out, a página consolida automaticamente todos os custos da estadia, aplicando o *Peak Pricing* quando a reserva abrange épocas de preço diferenciado (RF.07), e inclui os serviços extra consumidos, apresentando ao operador uma pré-visualização da fatura antes da emissão definitiva (RF.36). Esta funcionalidade responde diretamente à frustração dos clientes com surpresas na faturação final, identificada na entrevista da secção 3.4.2.

Nome	Ficha Técnica	Pré-Check-IN	Operações
Marega	Ver Ficha	Não	S N

Figura 53: Página da Rececionista - Check-In.

Nome: Marta Rececionista | Hora: 19:23:04
Perfil: Rececao | Data: 26/05/2026

Check-OUT / Faturação

Fatura Proforma

Cliente: Hugo Cunha (NIF: 12950710856)
Hóspede: Sennupy

Descrição	Valor
Estadia Base e Serviços Contratados	260.00€
Subtotal	260.00€
IVA (23%)	59.80€
TOTAL A PAGAR	319.80€

Método de Pagamento: Dinheiro Físico

Cancelar EMITIR FATURA E PAGAR

Figura 54: Pré-Visualização da Fatura - Check-Out.

5.5.7. Módulo Veterinário (VeterinariaPage.tsx)

O módulo veterinário é o mais complexo do sistema do ponto de vista clínico e foi concebido com particular atenção à segurança, rastreabilidade e prevenção de erros. Dada a natureza crítica das operações que suporta, a interface privilegia a clareza na apresentação de dados sensíveis e a separação inequívoca entre ações de consulta e ações de registo com efeitos permanentes.

As funcionalidades principais incluem: prescrição e gestão de planos de medicação digitais com alertas de horário para o staff (RF.20, RF.21), registo de administração com geração imediata do LogMedicacao imutável (RF.22), ativação do protocolo de quarentena com alteração automática do estado do animal no mapa de boxes (RF.23), e visualização do histórico clínico completo de cada hóspede. Em linha com as decisões de UI/UX documentadas na secção 4.6.4, a interface utiliza a cor como ferramenta de prevenção de erros: as ações críticas e irreversíveis, como o pedido de quarentena, são destacadas a vermelho, enquanto as ações de conclusão normal utilizam tons neutros ou azuis.

A página integra-se diretamente com a GestClinicaFacade no *backend*, garantindo que cada prescrição registada aciona simultaneamente a dedução síncrona do *stock* de medicamentos correspondente (RF.26), mantendo o inventário clínico sempre atualizado sem intervenção manual adicional.

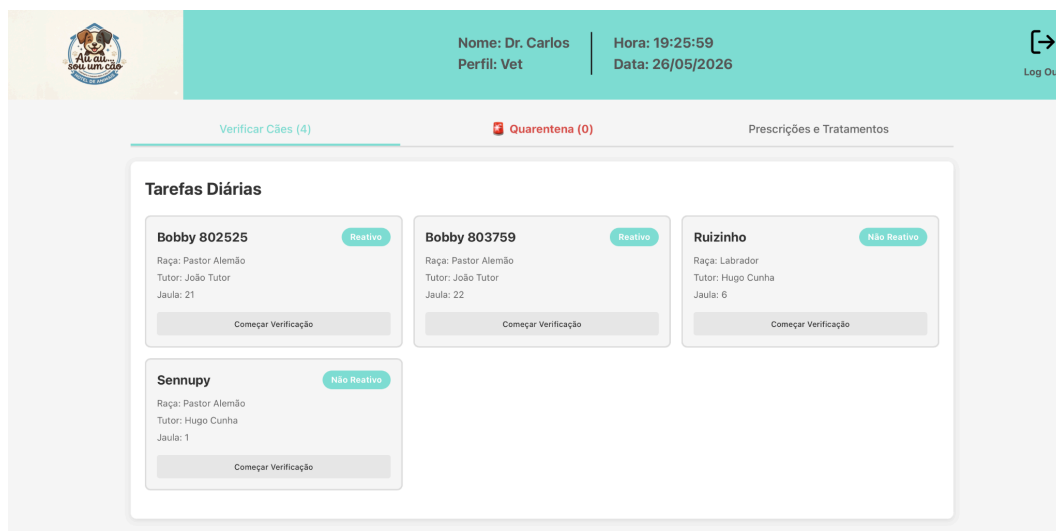


Figura 55: Página do Veterinário - Verificar Cães.

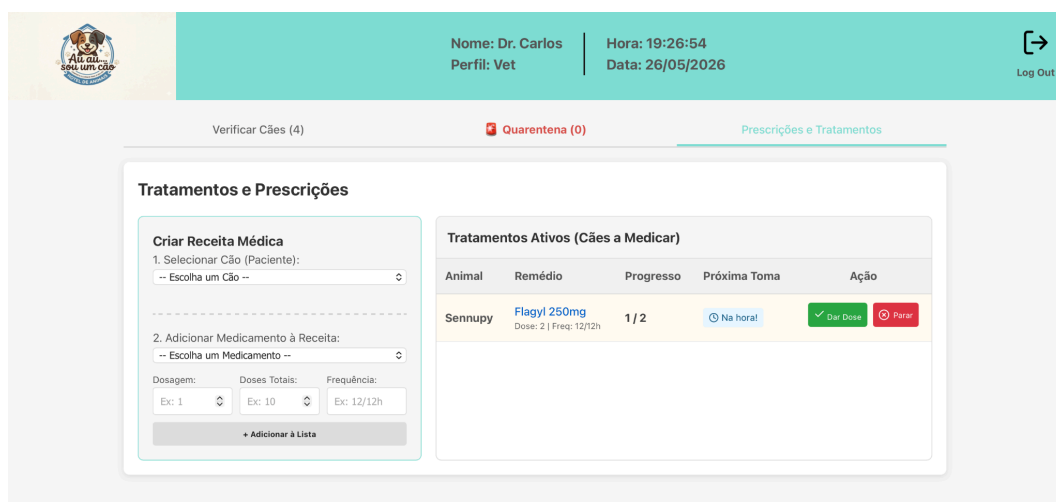


Figura 56: Página do Veterinário - Tratamento e Prescrições.

5.5.8. Módulo do Staff Operacional (StaffPage.tsx)

O módulo do staff foi desenhado sob os princípios de *mobile-first*, *stress-proof* e *one-hand operation* definidos na secção 4.6.3, reconhecendo que os cuidadores operam frequentemente num ambiente ruidoso, com as mãos ocupadas e sob pressão de tempo. A interface apresenta a *checklist* diária de tarefas a realizar por cada cuidador: alimentação, passeios, banhos e limpeza de boxes (RF.28). Cada tarefa é marcável com um único toque, atualizando o estado em tempo real no sistema e garantindo a sincronização entre turnos, eliminando a principal fonte de falha identificada na observação direta — a dependência do quadro branco com marcadores gastos (secção 3.3.1).

Os botões de ação foram sobredimensionados e posicionados na metade inferior do ecrã para facilitar o uso com uma só mão, e as cores de alto contraste garantem a legibilidade em condições de luz exterior. A interface inclui ainda o registo de incidentes com *upload* de fotografia (RF.30) e a atualização do estado de higienização das boxes, com os estados "Suja", "Em Limpeza" e "Higienizada" (RF.31), eliminando a necessidade de verificação porta a porta descrita na observação direta.

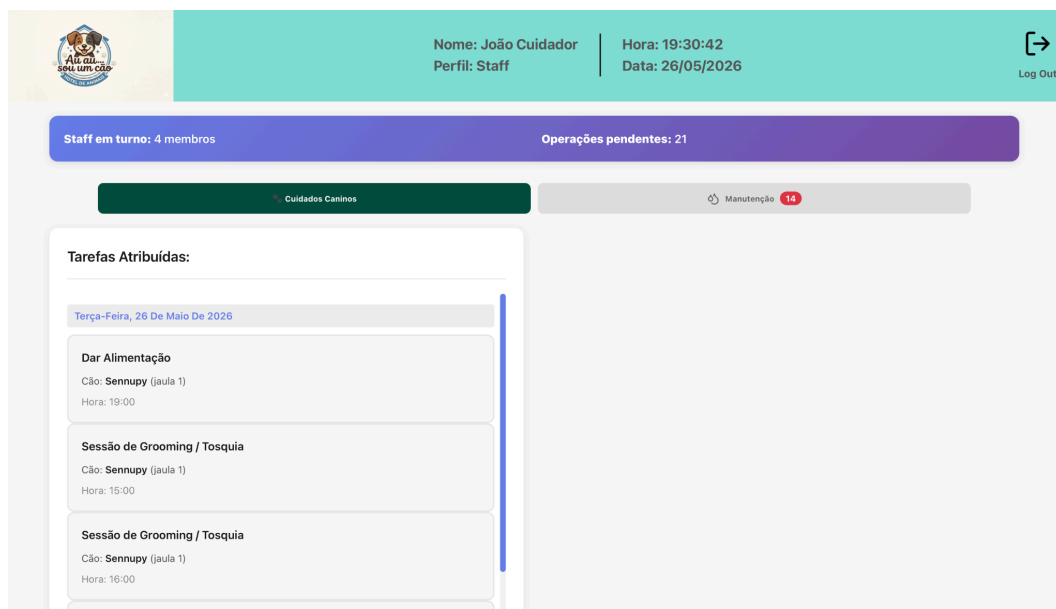


Figura 57: Página do Staff - CheckList de Tarefas.

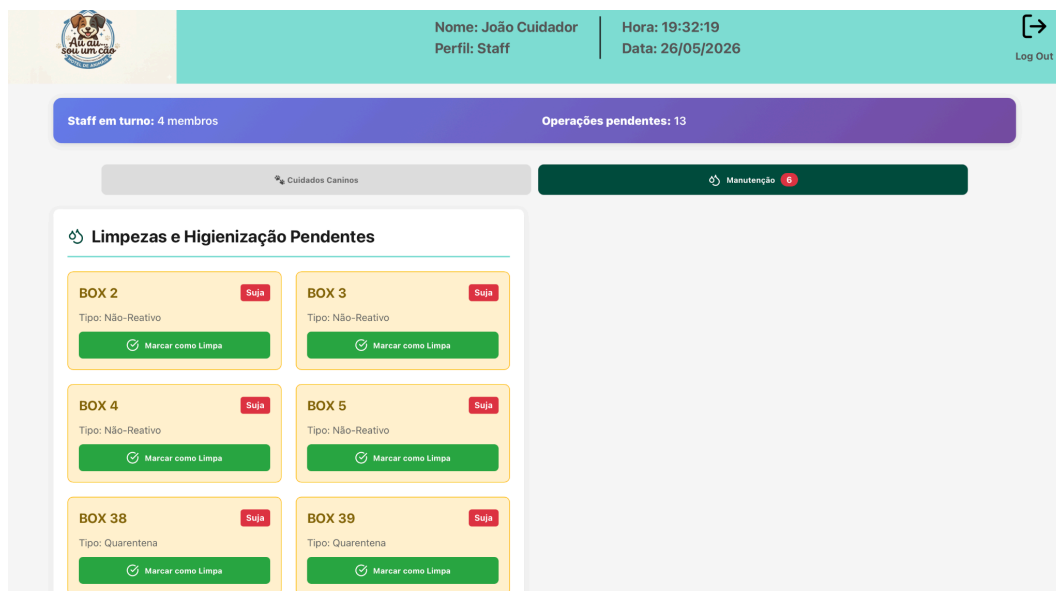


Figura 58: Página do Staff - Higienização Boxes.

5.5.9. Módulo de Marcações (MarcacoesPage.tsx)

A página de marcações gere o agendamento de serviços extra disponíveis no hotel, como banhos, tosas e consultas veterinárias (RF.29). A interface apresenta um calendário de disponibilidade por tipo de serviço, permitindo ao *staff* confirmar, reagendar ou cancelar marcações sem abandonar a área operacional do sistema. Este módulo responde à necessidade identificada na entrevista com o *staff* de eliminar os erros de faturação resultantes da ausência de registo digital imediato dos serviços realizados (secção 3.4.3).

Figura 59: Página do Tutor - Marcações.

5.5.10. Diário de Bordo (DiarioBordoPage.tsx)

O Diário de Bordo é uma das funcionalidades com maior impacto na experiência do tutor e foi identificado como um dos requisitos mais valorizados durante as entrevistas (secção 3.4.2, RF.37). A página permite ao *staff* efetuar o *upload* de fotografias ou vídeos do animal hóspede, acompanhados de uma breve descrição textual, durante o decorrer da estadia. Após a submissão, o ficheiro é enviado para a pasta *diario/* do Amazon S3 via *S3StorageAdapter* (secção 5.3.2) e a URL resultante é persistida pelo *DiarioBordoDAO*, ficando imediatamente disponível na *timeline* do tutor correspondente.

Esta funcionalidade substitui diretamente o fluxo atual de envio manual de fotografias individuais via *WhatsApp*, que a equipa do hotel identificou como um dos maiores "ladrões de tempo" da operação diária (secção 3.4.1), libertando o *staff* para as tarefas de cuidado direto dos animais.

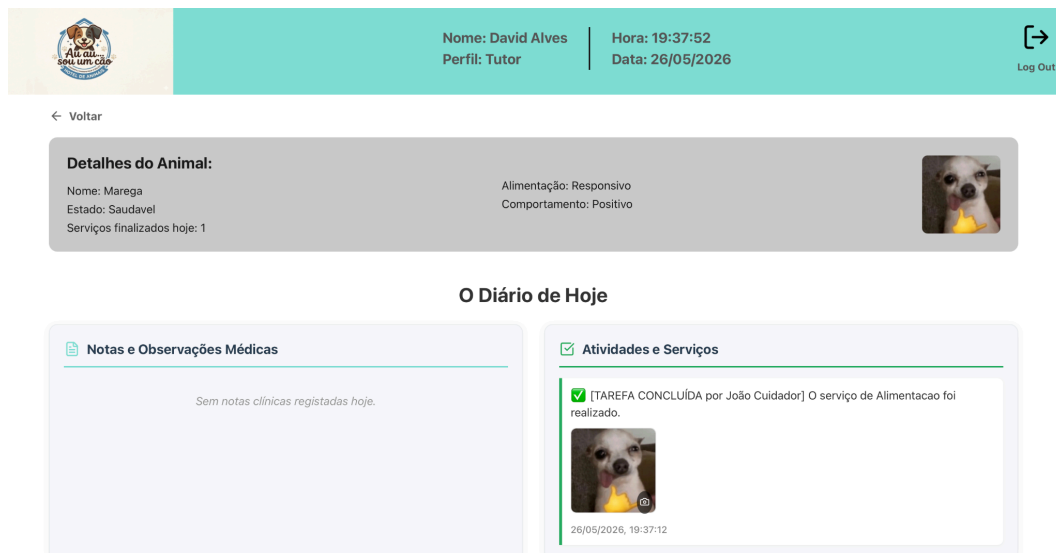


Figura 60: *Diário de Bordo do cão Marega.*

5.5.11. Portal do Tutor (PortalTutor.tsx)

O portal do tutor foi desenvolvido com uma orientação clara para a navegação em dispositivos móveis, em linha com o perfil de utilização identificado na entrevista com o cliente (secção 3.4.2, RNF.15). A interface foi concebida para responder a uma necessidade central: permitir ao tutor sentir-se ligado e informado sobre o animal durante o período de alojamento, sem sobrecarregar a equipa do hotel com pedidos individuais de informação. As funcionalidades disponíveis incluem: consulta da *timeline* do Diário de Bordo com as entradas registadas pelo *staff* durante a estadia atual (RF.37), acesso ao histórico de faturas com *download* em PDF (RF.38), visualização do plano de medicação ativo e dos registos de administração com *timestamp* (US07), e gestão de reservas futuras com seleção de serviços extra (RF.23). A estrutura da página prioriza o acesso imediato às duas tarefas mais frequentes do tutor, em linha com as justificações de UI/UX documentadas na secção 4.6.1.

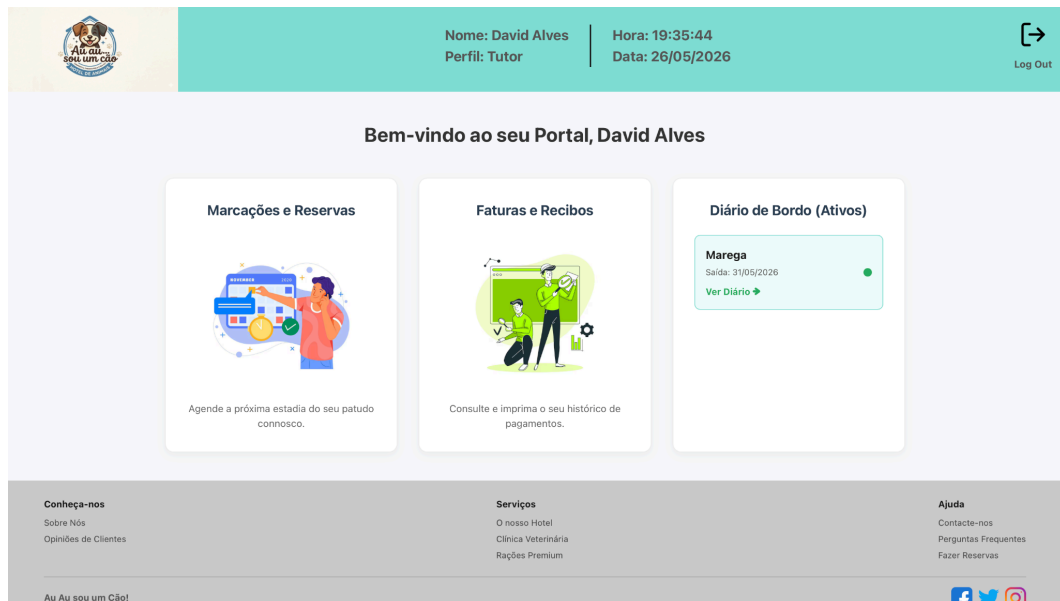


Figura 61: Página Principal do Tutor.

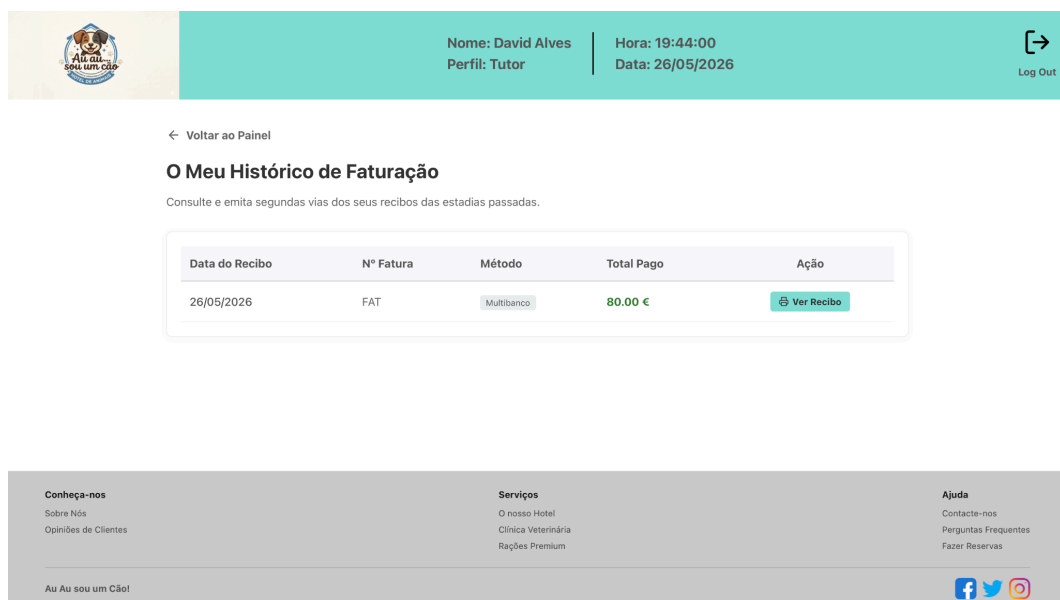


Figura 62: Histórico de Faturação do Tutor.

5.5.12. Princípios Transversais de Desenvolvimento da Interface

O desenvolvimento do *frontend* procurou equilibrar três preocupações centrais, presentes em todos os módulos: simplicidade operacional, adequação ao perfil de utilizador e coerência visual entre áreas. Esta preocupação é particularmente relevante num sistema com perfis muito distintos, onde coexistem tarefas rápidas de execução no terreno, atividades administrativas de detalhe e interações ocasionais dos tutores a partir de casa.

Do ponto de vista da interface, foi dada prioridade à clareza da informação apresentada, à redução do ruído visual e à organização dos ecrãs em blocos funcionais facilmente reconhecíveis. Em vez de concentrar todas as ações numa única vista, cada área do sistema foi desenvolvida como um módulo autónomo, o que permitiu reduzir a carga cognitiva e tornar cada perfil de utilização mais intuitivo e eficiente. A evolução da interface não foi encarada apenas como uma fase técnica, mas como um processo iterativo de tradução entre requisitos, protótipos e validação com a *stakeholder*, resultando num produto final alinhado tanto com as necessidades operacionais do hotel como com a identidade e visão da sua fundadora

5.6. Revisão de Código

No contexto da Engenharia de Software contemporânea, a revisão de código (*code review*) e os fluxos de integração contínua constituem pilares fundamentais para assegurar a manutenibilidade, a segurança e a resiliência de sistemas digitais. Em ambientes de desenvolvimento industrial e profissional, o ciclo de vida de uma nova funcionalidade não termina com a sua codificação isolada. Pelo contrário, as empresas recorrem a mecanismos formais e rigorosos assentes na abertura de *Pull Requests* (PRs).

Nesta metodologia padrão da indústria, antes de qualquer código ser fundido (*merged*) no ramo principal (*main branch*) e enviado para os servidores de produção através de processos de *deployment*, este é submetido a uma auditoria minuciosa por pares (*peer review*). Este escrutínio visa detetar vulnerabilidades de segurança, garantir o cumprimento de padrões de estilo e arquiteturais, identificar potenciais estrangulamentos de performance e prevenir a introdução de regressões lógicas no ecossistema já operacional.

5.6.1. Execução da Metodologia Semi-Profissional no Projeto

Dada a natureza e as restrições de um projeto académico com recursos computacionais e humanos limitados, a equipa adotou uma abordagem metodológica adaptada, designada como "técnica semi-profissional". O fluxo de desenvolvimento foi iniciado através da elaboração de uma *blueprint* global do projeto, procedendo-se à criação estática e estrutural de todas as interfaces da aplicação em ambiente local (*localhost*). Esta sequência de desenho e engenharia priorizou a experiência do utilizador e as jornadas operacionais, iniciando-se pela página pública (*homepage*), seguida pelos módulos de reservas, marcações e portal do tutor. Subsequentemente, foram desenvolvidas as componentes da receção, do *staff* operacional, da vertente médica veterinária e, por fim, o painel consolidado da gestora.

A evolução do protótipo baseou-se num desenvolvimento incremental e iterativo. Após o desenho das páginas, foram acopladas de forma sistemática as funcionalidades dinâmicas do sistema de modo a satisfazer os requisitos eliciados (tais como o motor de alocação de marcações, as regras de faturação e as tabelas financeiras). Comprometidos com o rigor técnico, determinou-se que a conclusão de qualquer página ou componente funcional exigia a realização de testes exaustivos em *localhost* até à sua saturação.

Para simular o rigor dos fluxos de trabalho industriais de autorização de *deployment*, a equipa estruturou uma cadeia de responsabilidades interna bem definida. Quando um membro terminava a codificação de uma funcionalidade em ambiente local, o código não era integrado diretamente de forma autónoma. Um segundo elemento do grupo ficava formalmente encarregue de atuar como revisor principal, procedendo à leitura analítica do código e à verificação minuciosa da sua coerência lógica e conformidade com a arquitetura definida. Apenas após a validação e a aprovação expressa por parte deste revisor individual é que a funcionalidade recebia autorização para transitar do ambiente de desenvolvimento local para o processo de *deployment* e distribuição na infraestrutura remota alojada na nuvem da AWS.

5.6.2. Análise Crítica de Casos de Estudo e *Code Review*

A engenharia de *software* do projeto assentou num paradigma moderno onde a geração de código assistida por LLM funcionou como o motor primário para a aceleração do desenvolvimento e criação de *boilerplates* (estruturas base de código). No entanto, ciente dos riscos de alucinação e subotimização da inteligência artificial generativa, a equipa implementou um fluxo rigoroso assente no binómio da revisão automática e humana.

Antes da intervenção dos revisores, ferramentas de análise estática e revisão automática (como o ESLint e o Prettier) faziam o primeiro filtro de sintaxe e estilo. Posteriormente, avançava-se para a revisão do código gerado (*code reviewing*) por parte de um elemento humano do grupo. Esta fase revelou-se indispensável, focando-se na deteção de *smells* (sintomas de problemas estruturais ou de performance no código) e na subsequente formulação de *prompts* diretos ao LLM com sugestões de modificação (*refactoring*) para garantir a robustez do sistema, como se ilustra nos dois casos de estudo seguintes:

- **Caso de Estudo 1: O Módulo do Diário de Bordo (*Refactoring* e Arquitetura de Dados)** Na fase inicial de estruturação, o LLM gerou o modelo do Diário de Bordo associando-o de forma direta e exclusiva à entidade "Animal", acumulando fotos e descrições do dia de forma genérica. Durante o *code*

reviewing humano e testes das lógicas geradas, detetou-se um *smell* na modelação do domínio (perda de rastreabilidade): ao finalizar um serviço (ex: Banho ou Passeio), tornou-se impossível mapear inequivocamente que foto ou descrição pertencia a que serviço específico. Como ação corretiva, o LLM foi instruído a refatorar a estrutura de dados (Prisma ORM) e as camadas de acesso (DAOs e Facades). O diário foi transformado numa lógica de linha temporal, onde cada registo (instância) passou a guardar o contexto do serviço executado e a ligar-se diretamente à entidade "Reserva". Esta iteração conjunta (Humano-IA) eliminou o *smell* lógico e garantiu um histórico preciso e auditável.

- **Caso de Estudo 2: O Módulo da Gestora (Detecção de *Smells* Lógicos e Injeção de *Features*)** Numa fase preliminar, a assistência do LLM gerou com sucesso o esqueleto visual do ecrã de administração, mas a análise humana revelou um *smell* de design: o código exibia uma lógica anémica e estática, que falhava na implementação das regras de negócio reais da clínica e do hotel. A revisão conduziu a um *refactoring* profundo no *backend*. Através de um diálogo iterativo com o modelo, foram introduzidas lógicas transacionais complexas e processos automatizados em segundo plano (*Cron Jobs*), tais como a verificação de tarefas de *staff* não realizadas e a geração de alertas críticos de saúde (ex: animal que falha duas refeições consecutivas). Esta intervenção transformou um protótipo visual simples num painel operacional e analítico robusto.

5.7. Prompts e Geração dos Resultados

Nesta secção documenta-se a estratégia de interação com os LLMs durante as fases de implementação, detalhando o prompt formulado, a avaliação crítica das respostas geradas e o processo iterativo de refinamento. O objetivo foi utilizar a IA como agente de apoio à transposição dos modelos teóricos e regras de negócio complexas para código robusto e alinhado com a arquitetura definida.

5.7.1. Refatoração e Alinhamento com a Arquitetura Hexagonal

5.7.1.1. Prompt Inicial

"No nosso projeto, estabelecemos no relatório a adoção de Arquitetura Hexagonal, isolando a lógica de negócio das infraestruturas externas, utilizando padrões Facade e DAO. Contudo, o código atual em `api.ts` encontra-se fortemente acoplado. Como refatorar para que as rotas interajam única e exclusivamente com a Facade?

Implementa o ReservaDAO.ts para abstrair o acesso aos dados e o GestReservasFacade.ts para orquestrar as regras de negócio complexas."

5.7.1.2. Avaliação Crítica e Refinamento do Resultado

Avaliação Crítica do Resultado	Refinamento Aplicado / Ação Tomada
Isolamento Básico Bem-sucedido: O modelo gerou corretamente a estrutura inicial de isolamento (extraíndo as transações para o ReservaDAO e encapsulando a lógica de alocação na GestReservasFacade), mas apresentou uma falha arquitetural.	<i>Manutenção da Estrutura Central: A base do código gerado para o isolamento em Facade e DAO foi aceite, pois garantiu um baixo acoplamento tecnológico.</i>
Omissão na Segregação de Tipos: A IA ignorou inicialmente a necessidade de segregar os tipos de dados (Data Transfer Objects - DTO) para a camada de serviço, o que poderia comprometer o isolamento puro da lógica de negócio.	Criação Explícita de DTOs: Foi necessário instruir o modelo iterativamente para criar interfaces separadas para os dados de entrada e saída. Isto garantiu que o Core do sistema não conhecesse os detalhes específicos da implementação do Prisma ou da Express.

Tabela 18: Prompts da Refatorização da Arquitetura - Backend.

5.7.2. Lógica Transacional e Regras de Negócio (Prisma)

5.7.2.1. Prompt Inicial

"No Check-Out da rececionista, preciso que várias regras disparem: 1) A Reserva muda para 'CheckOut', 2) É gerada uma Fatura, 3) A Box passa a 'Suja' e 4) Se o cão estiver em 'Quarentena', deve ganhar 'Alta' para 'Saudável'. Como orquestro esta lógica no ReservaDAO.ts usando o Prisma?"

5.7.2.2 Avaliação Crítica e Refinamento do Resultado

Avaliação Crítica do Resultado	Refinamento Aplicado / Ação Tomada
Violação da Atomicidade: O modelo sugeriu inicialmente operações sequenciais independentes para lidar com a atualização da reserva, faturação, estado da box e alta médica. Esta abordagem violava a atomicidade da transação, podendo gerar estados inconsistentes no sistema em caso de falha a meio do processo.	Uso de Transações do Prisma: Refinou-se a instrução exigindo especificamente a utilização do método <code>prisma.\$transaction</code> . Isto garantiu que todas as operações fossem tratadas como um bloco único: se uma das operações falhasse (ex: erro na faturação), todo o processo seria revertido (rollback).

Tabela 19: Prompts do Prisma.

5.7.3. Desenvolvimento de Interface Complexa (Frontend)

5.7.3.1. Prompt Inicial

"Preciso de construir a página do Staff em React. O tratador precisa de ver tarefas diárias e boxes 'suja's'. Como estrutura a interface com `useState` para criar um sistema de 'Abas' (Tabs) e como uso `useEffect` para carregar simultaneamente os dados das tarefas e das boxes com `Axios` de forma otimizada?"

5.7.3.2. Avaliação Crítica e Refinamento do Resultado

Avaliação Crítica do Resultado	Refinamento Aplicado / Ação Tomada
Estrutura de Abas Bem-sucedida: A IA sugeriu corretamente a implementação lógica do sistema de abas utilizando a gestão de estado do React, fornecendo uma base sólida de apresentação.	<i>Manutenção da Renderização Condicional: A estrutura sugerida para gerir e apresentar as abas foi adotada na íntegra, por se revelar muito eficiente e alinhada com as boas práticas de desenvolvimento em React.</i>

Carregamento de Dados Ineficiente: A primeira versão gerada pelo modelo tentou realizar o *fetching* dos dados (tarefas e estados de boxes) de forma sequencial (em série). Esta abordagem era ineficiente e aumentaria desnecessariamente a latência da página.

Exigência de Paralelismo: A instrução foi refinada para exigir especificamente a aplicação de paralelismo na requisição assíncrona de dados. O modelo ajustou o código, passando a aplicar o método `Promise.all()` de forma adequada.

Tabela 20: Prompts do Frontend.

A Etapa 3 foi a que acumulou mais atrasos pontuais, embora todos de apenas um dia e sem impacto no prazo global. A configuração do ambiente e toda a orquestração cloud atrasaram um dia, consequência da complexidade da integração inicial com a AWS. O desenvolvimento da camada de dados seguiu o mesmo padrão, com a modelação da base de dados e o adaptador S3 a sofrerem atrasos de um dia cada. O desenvolvimento da camada de lógica de negócio, liderado pelo Gonçalo Sá, foi o único bloco concluído integralmente dentro do planeado. A camada de apresentação registou um atraso de um dia na validação visual com a stakeholder e nos módulos funcionais, reflexo do tempo de feedback necessário. A revisão de código final foi concluída dentro do prazo a 13/04.

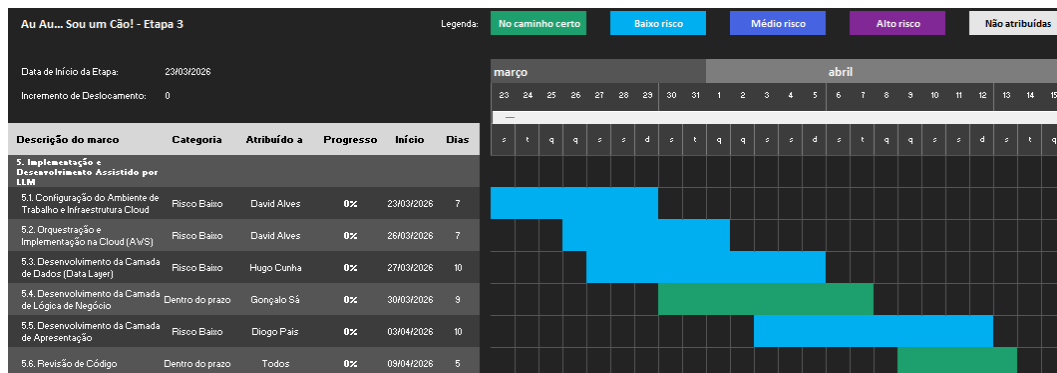


Figura 63: Diagrama de Gantt final – Capítulo 5.

6. Verificação, Validação e Avaliação da Qualidade do Software Produzido

Nesta fase, o foco transitou da concepção arquitetural para a validação do *software*. A estratégia de testes foi estruturada em quatro camadas progressivas — testes unitários, de integração, de sistema (E2E) e de aceitação manual — complementadas por geração assistida de casos de teste via LLM. O objetivo primário foi verificar formalmente a satisfação dos requisitos do SRS e avaliar a qualidade do sistema segundo as características da norma ISO/IEC 25010.

O capítulo 6 foi planejado para decorrer entre os dias 13/04 e 11/05, correspondendo à Etapa 4 de verificação e validação. A figura seguinte apresenta o diagrama de Gantt objetivo para este capítulo.

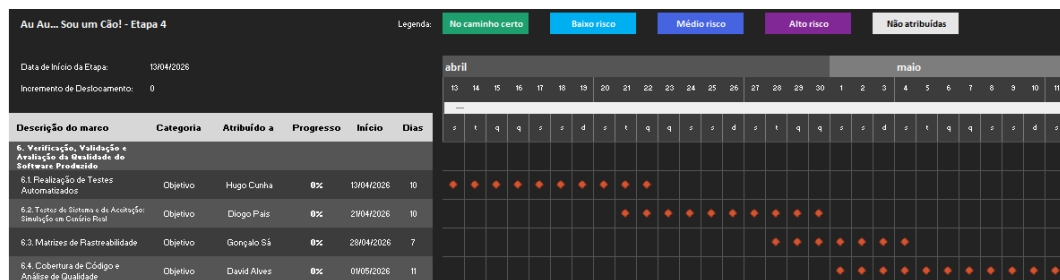


Figura 64: Diagrama de Gantt objetivo – Capítulo 6.

6.1. Realização de Testes Automatizados

A validação técnica do *software* foi estruturada numa pirâmide de testes automatizados, utilizando a *framework* Jest, garantindo a solidez do *backend* em Node.js e prevenindo regressões durante o desenvolvimento.

6.1.1. Testes Unitários

A base da pirâmide focou-se nas Facades (GestClinicaFacade, GestReservasFacade, etc.), o coração da lógica de negócio. Através da técnica de *Mocking* (simulação de acesso a dados), isolou-se a base de dados para testar exclusivamente a lógica algorítmica. Foram validadas regras fundamentais, como a atribuição inteligente de boxes de quarentena.

6.1.2. Testes de Integração

Para validar a comunicação entre as rotas da API (*Express*), os *middlewares* de segurança, o ORM (*Prisma*) e a base de dados PostgreSQL, implementaram-se testes de integração recorrendo à biblioteca Supertest.

Numa demonstração de rigor técnico, estes testes não foram executados num ambiente laboratorial à parte, mas sim diretamente no ambiente e base de dados utilizados para o desenvolvimento do projeto, isto porque o investimento para uma nova base de dados em PostgreSQL seria bastante custoso para a equipa. Para proteger os dados reais (como contas de administração, histórico hoteleiro e configuração estrutural do hotel), a equipa implementou uma estratégia de Limpeza Direcionada (*Targeted Teardown*). Os dados injetados pelos testes possuíam assinaturas únicas (e.g., NIFs iniciados em "9999", emails com prefixos específicos e boxes com numerações de teste). No final de cada ciclo transacional, os scripts de limpeza executavam a eliminação exclusiva dos registos com estas assinaturas, garantindo a idempotência da base de dados sem corromper as informações reais do sistema.

Nestas suites, simularam-se ciclos de vida transacionais completos, como por exemplo:

1. Registo de um utilizador e injeção do mesmo na base de dados;
2. Geração e validação de *Tokens JWT* e requisitos de 2FA;
3. Agendamento de uma reserva autenticada e transição de estado da box;
4. Execução das rotas de *Check-In* e *Check-Out* com geração de fatura final.

```
Test Suites: 16 passed, 16 total
Tests:       162 passed, 162 total
Snapshots:   0 total
Time:        53.005 s, estimated 184 s
Ran all test suites.
Force exiting Jest: Have you considered using '--detectOpenHandles' to detect async operations that kept running after all tests finished?
```

File	% Hits	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	81.03	86.37	88.38	81.03	
adapters	41.46	60	50	41.46	
StorageAdapter.ts	41.46	60	50	41.46	13-33,39-41
core	95.83	97.38	94.04	95.83	
GestClinicaFacade.ts	100	100	100	100	
GestFaturacaoFacade.ts	100	100	100	100	
GestHospedesFacade.ts	100	100	100	100	
GestOperacoesFacade.ts	100	100	100	100	
GestReservasFacade.ts	93.47	99.9	91.66	93.47	81-86,98-104,208-209
GestorHotelFacade.ts	92.18	100	90	92.18	53-54,128-130,163-168,179-182
dao	80.52	83.13	81.81	80.52	
AnimaDAO.ts	60.92	91.66	75	60.92	62-103,132-140,144-151
DiarioBordoDAO.ts	76.19	75	66.66	76.19	20-24,38-42
FaturaDAO.ts	100	100	100	100	
LogsDAO.ts	100	100	100	100	
PrescricaoDAO.ts	92.01	81.25	100	92.01	94-111,113-117
ReservacaoDAO.ts	77.27	79.31	72.22	77.27	49-57,79-82,86-91,161-170,226-244,265-267,332-352
StockDAO.ts	82.5	100	80	82.5	28-34
UtilizadorDAO.ts	82.35	70	83.33	82.35	57-61,63-65,72-78
middleware	100	87.5	100	100	
auth.ts	100	87.5	100	100	5
routes	62.8	68.25	100	62.8	345,352-355,361-362,370-371,377-379,382-389,399-401,408-410,423-424,432,439-441,458-460,495-498,502-519,522-527
api.ts	62.8	68.25	100	62.8	
services	90	72.72	85.71	90	
EmailService.ts	100	57.14	100	100	5-19
TwoFactorService.ts	80	100	75	80	21-25

Figura 65: Testes Backend unitários e de Integração.

6.2. Testes de Sistema e de Aceitação: Simulação em Cenário Real

Para garantir a robustez e a qualidade da plataforma AuAuSouUmCao, a nossa fase de testes foi dividida em duas abordagens complementares: a automação de fluxos críticos (para evitar regressões) e os testes Beta manuais (para validar a experiência de utilização).

6.2.1. Testes Automatizados *End-to-End* (E2E) com *Playwright*

Nesta fase, utilizamos a *framework Playwright* para simular a interação de utilizadores reais com o sistema. O objetivo foi garantir que as funcionalidades vitais do negócio operam perfeitamente desde o *frontend* até à base de dados. Automatizamos as jornadas dos três perfis principais do sistema:

- Fluxo do Cliente (Tutor): Validação do registo de novos animais com *upload* de documentos e a marcação de reservas de alojamento e serviços, lidando com verificações de segurança e tratamento de cães duplicados.
- Fluxo Operacional (*Staff*): Simulação do dia a dia de um tratador, testando a navegação pelos painéis de tarefas, limpeza de boxes e a conclusão de cuidados caninos com submissão de provas fotográficas.
- Fluxo Clínico (Veterinário): Validação da área de saúde, garantindo que o médico consegue registar *checks* diários de rotina e transitar para a área clínica para emitir prescrições e dosagens medicamentosas para os hóspedes.

A suite E2E executou 14 cenários automatizados em dois browsers (*Chromium* e *Firefox*), cobrindo os quatro perfis de utilizador, com 0 falhas e tempo total de execução de 17.9 segundos.

Test Category	Test Name	Description	Browser	Duration
login.spec.ts	Fluxo E2E - Autenticação e Dashboard	Deve fazer login com sucesso como Admin e entrar no sistema	chromium	2.5s
	login.spec.ts:8			
	Fluxo E2E - Autenticação e Dashboard	Deve mostrar erro se as credenciais estiverem erradas	chromium	2.8s
	login.spec.ts:36			
login.spec.ts	Fluxo E2E - Autenticação e Dashboard	Deve fazer login com sucesso como Admin e entrar no sistema	firefox	3.8s
	login.spec.ts:8			
	Fluxo E2E - Autenticação e Dashboard	Deve mostrar erro se as credenciais estiverem erradas	firefox	4.1s
	login.spec.ts:36			
staff_limpar.spec.ts	Fluxo E2E - Staff (Tratador)	Deve fazer login, limpar uma box e concluir um cuidado canino com foto	chromium	5.0s
	staff_limpar.spec.ts:7			
	Fluxo E2E - Staff (Tratador)	Deve fazer login, limpar uma box e concluir um cuidado canino com foto	firefox	6.3s
	staff_limpar.spec.ts:7			
staff.spec.ts	Fluxo E2E - Staff (Tratador)	Deve fazer login, limpar uma box e concluir um cuidado canino qualquer	chromium	7.3s
	staff.spec.ts:7			
	Fluxo E2E - Staff (Tratador)	Deve fazer login, limpar uma box e concluir um cuidado canino qualquer	firefox	8.3s
	staff.spec.ts:7			
tutor.spec.ts	Fluxo E2E - Tutor (Cliente)	Deve fazer login, registrar um cão único e criar uma reserva	chromium	7.5s
	tutor.spec.ts:7			
	Fluxo E2E - Tutor (Cliente)	Deve fazer login, registrar um cão único e criar uma reserva	firefox	6.9s
	tutor.spec.ts:7			
veterinario.spec.ts	Fluxo E2E - Veterinário (Arções Independentes)	Deve fazer login e finalizar um Check-In com sucesso		5.7s

Figura 66: Testes com o Playwright 14/14.

6.2.2. Testes Beta e de Usabilidade (Manuais)

Para o efeito, realizou-se uma simulação intensiva de um "Dia de Trabalho" no hotel canino. Neste cenário E2E, cada membro da equipa assumiu o papel de um ator diferente do sistema:

1. **O Cliente (Tutor):** Simulou o registo na plataforma, a introdução do perfil do seu cão e o pedido de uma reserva.
2. **A Rececionista:** Validou os termos de responsabilidade ao balcão e executou o *Check-In* no sistema.
3. **O Tratador (Staff):** Consultou o seu *Dashboard* de tarefas diárias pelo *smartphone*, executou o passeio e a alimentação do cão, documentando com uma fotografia no Diário de Bordo.
4. **O Veterinário:** Durante a ronda diária, identificou um sintoma atípico, colocou o animal em quarentena (alterando o estado físico da *box* no sistema) e prescreveu medicação deduzida em tempo real do *stock*.
5. **A Gestora:** Validou o painel de faturação no momento do *Check-Out* e analisou o fluxo financeiro.

Esta encenação em tempo real comprovou de forma inequívoca a satisfação dos SRS, atestando que a comunicação assíncrona entre diferentes perfis de utilizador decorre sem falhas ou bloqueios operacionais.

Com base no *feedback* recolhido nos testes manuais, concluímos que a interface do *Staff* precisava de ser ajustada. Para melhorar a ergonomia de quem está no terreno, alterámos a forma como as informações são exibidas e adicionamos um botão de deslizar, tornando a gestão das tarefas diárias muito mais ágil e intuitiva.

6.3. Matrizes de Rastreabilidade

As Tabelas seguintes constituem a Matriz de Rastreabilidade do projeto, mapeando cada requisito funcional e não-funcional identificado nas entrevistas com a cliente ao ficheiro e método concreto que o implementa. Este mapeamento permite verificar formalmente a satisfação dos SRS e identificar com transparência os requisitos que, por condicionantes de tempo e complexidade técnica, não foram implementados na totalidade neste protótipo.

6.3.1 Matriz de Rastreabilidade RF

A tabela seguinte mapeia os 43 requisitos funcionais levantados nas entrevistas com a cliente Diana às suas implementações concretas no código. Cada requisito é rastreado até ao ficheiro e método específico que o satisfaz, permitindo verificar que a arquitetura de Facades adotada: GestReservasFacade, GestClinicaFacade, GestHospedesFacade, GestFaturacaoFacade e GestOperacoesFacade; cobre de forma direta e auditável a totalidade das regras de negócio identificadas na fase de análise.

ID	Nome do Requisito	Ficheiro	Método/Função
RF.01	Calendário de Disponibilidade	ReservaDAO via GestReservasFacade	criarReserva() — valida datas de entrada/saída
RF.02	Validação de Conflitos	GestReservasFacade.ts	criarReserva() — verifica sobreposição de datas
RF.03	Bloqueio de Overbooking	ReservaDAO via GestReservasFacade	criarReserva() → reservaDAO.atribuirBoxAutomaticamente()
RF.04	Alerta de Ocupação Crítica	GestReservasFacade.ts	criarReserva() — lança erro se não há box disponível
RF.05	Alocação por Espécie	ReservaDAO via GestReservasFacade	reservaDAO.atribuirBoxAutomaticamente(reatividade, ...)

RF.06	Alocação por Perfil	GestReservasFacade.ts	criarReserva() — usa animalExiste.reatividade para atribuição
RF.07	Cálculo de Reembolso	GestReservasFacade.ts	processarCheckOutCompleto() — ajuste de valor por dias reais vs. previstos
RF.08	Emissão de Vale/Crédito	GestFaturacaoFacade.ts	gerarFaturaFinal() — geração de fatura certificada
RF.09	Validação de Vacinação	GestHospedesFacade.ts	atualizarPlanoVacinal() + registrarAnimal()
RF.10	Contactos de Emergência	GestHospedesFacade.ts	registrarAnimal() — campo obrigatório na ficha
RF.11	Tabela de Preços Dinâmica	GestFaturacaoFacade.ts	gerarFaturaFinal() — cálculo de peak pricing
RF.12	Gestão de Fila de Espera	ReservaDAO via GestReservasFacade	criarReserva() — bloqueia se hotel cheio
RF.13	Notificação de Vaga	routes/api.ts	POST /reservas — resposta de erro com estado de lotação
RF.14	Verificação de Prolongamento	GestReservasFacade.ts	processarCheckOutCompleto() — recalculo se diasReais > diasPrevistos
RF.15	Recalculo de Estadia	GestReservasFacade.ts	processarCheckOutCompleto() — valorFinal += diasAtraso * 20
RF.16	Upload de Documentos	routes/api.ts	POST /animais — upload.single('vacinasFile') + s3Adapter.uploadFicheiro()

RF.17	Leitura OCR de Boletins	S3StorageAdapter / routes/api.ts	POST /animais — ficheiro guardado em S3 para validação
RF.18	Registo de Microchip	GestHospedesFacade.ts	registarAnimal() — gera CHIP- <code>{Date.now()}</code> se ausente
RF.19	Historial de Saúde	GestHospedesFacade.ts	obterHistorialAnimal() → animalDAO.findHistorialComDados()
RF.20	Horários de Medicação	GestClinicaFacade.ts	prescreverMedicacao() — cria prescrição com horários
RF.21	Registo de Dosagem	GestClinicaFacade.ts	prescreverMedicacao() — valida dosagem > 0 e totalDoses > 0
RF.22	Log de Administração	GestClinicaFacade.ts	registarAdministracao(idLinha, idFuncionario)
RF.23	Sinalética de Quarentena	GestClinicaFacade.ts	ativarQuarentena(idAnimal, motivo)
RF.24	Registo de Alimentação	GestReservasFacade.ts	criarReserva() — cria serviço Alimentacao por cada dia (19:00)
RF.25	Alerta de Inapetência	GestReservasFacade.ts	marcarTarefaConcluida() — registo no diário ao completar tarefa
RF.26	Inventário Clínico	GestClinicaFacade.ts	listarStockCompleto() + reforcarStock()
RF.27	Relatórios para Seguradoras	routes/api.ts	GET /logs — exporta DiarioBordo completo
RF.28	Checklist Móvel de Tarefas	GestReservasFacade.ts	listarTarefasDoDia() + marcarTarefaConcluida()

RF.29	Agendament o de Serviços Extra	GestReservasFacade.t s	criarReserva() — cria serviços Passeio, Grooming/Banho, Grooming/Tosquia
RF.30	Registo de Incidentes com Foto	routes/api.ts	PATCH /tarefas/:id/concluir — upload.single('fotoProva') + S3
RF.31	Estado de Limpeza de Boxes	GestReservasFacade.t s	listarBoxesSujas() + limparBox(numero)
RF.32	Mapa de Alocação Visual	ReservaDAO via GestReservasFacade	reservaDAO.atribuirBoxAutomatic amente() + listarBoxesSujas()
RF.33	Impressão de Etiquetas	GestReservasFacade.t s	criarReserva() — serviços com nome do animal e restrições
RF.34	Galeria de Média Diária	GestOperacoesFacade .ts	adicionarRegistoDiario(animalId, descricao, fotos[])
RF.35	Assinatura Digital	GestReservasFacade.t s	confirmarCheckIn() — valida termosAceites: boolean
RF.36	Faturação Certificada AT	GestFaturacaoFacade.t s	gerarFaturaFinal() — gera FAT- $\{Date.now()\}$.pdf com NIF
RF.37	Timeline do Tutor	GestHospedesFacade.t s	animalDiario(idAnimal) → animalDAO.animalDiario()
RF.38	Portal Financeiro Cliente	GestFaturacaoFacade.t s	listarFaturasDoTutor(nif) + GET /faturas/tutor/:nif
RF.39	Notificações Push	routes/api.ts + EmailService.ts	POST /login — sendEmail() com código 2FA

RF.40	Alerta de Emergência Crítica	GestClinicaFacade.ts	ativarQuarentena() + registrarCheckDiario()
RF.41	Dashboard de Gestão	routes/api.ts	GET /faturas + GET /logs + GET /funcionarios/count
RF.42	Página Pública	Frontend React	Componentes públicos sem authMiddleware
RF.43	Partilha de Experiência	Frontend React	Componentes de reviews/comentários

Tabela 21: Rastreabilidade dos requisitos funcionais.

6.3.2. Matriz de Rastreabilidade RNF

A tabela seguinte mapeia os 19 requisitos não funcionais do sistema às suas implementações técnicas. Ao contrário dos requisitos funcionais, que se materializam em métodos e fluxos de dados, os RNFs atravessam transversalmente a arquitetura, desde a camada de segurança no *middleware* de autenticação, passando pela infraestrutura *cloud* na AWS, até às decisões de *design* da interface *mobile*. A sua satisfação é verificada não por testes unitários isolados, mas pela composição das escolhas arquiteturais documentadas ao longo dos capítulos anteriores.

ID	Nome do Requisito	Ficheiro	Método/Mecanismo
RNF.01	Encriptação de Dados	routes/api.ts	bcrypt.hash(password, 10) no POST /register; S3 para documentos
RNF.02	Controlo de Acessos RBAC	middleware/auth.ts	authMiddleware — verifica JWT em todas as rotas protegidas; role no token
RNF.03	Autenticação Two Factor	routes/api.ts + TwoFactorService.ts	POST /login → generate2FACode() + sendEmail() + POST /verify-2fa → verify2FACode()

RNF.04	Retenção de Auditoria	routes/api.ts	GET /logs — prisma.diarioBordo.findMany() com histórico completo
RNF.05	Conformidade Fiscal	GestFaturacaoFacade.ts	gerarFaturaFinal() — campo nifCliente obrigatório, simulação PDF AT
RNF.06	Latência de Operações	GestReservasFacade.ts	Operações assíncronas com async/await; DAO com queries otimizadas
RNF.07	Performance do Mapa	ReservaDAO	atribuirBoxAutomaticamente() — query indexada por box e datas
RNF.08	Escalabilidade e de Pico	index.ts + AWS Elastic Beanstalk	Deploy em AWS com auto-scaling
RNF.09	Disponibilidade / Uptime	AWS Elastic Beanstalk	Infraestrutura cloud com RDS PostgreSQL
RNF.10	Recuperação de Dados	AWS RDS	Backup automático configurado no RDS
RNF.11	Sincronização Offline	Frontend React	Service Workers + Local Storage para modo offline
RNF.12	Ergonomia One-Hand	Frontend React	Interface Mobile-First com botões grandes
RNF.13	Adaptabilidade e Visual	Frontend React	Ícones grandes no layout de canil
RNF.14	Sinalética Cognitiva	Frontend React	Cores e símbolos por estado (Suja/Higienizada/Quarentena)
RNF.15	Otimização Mobile	Frontend React	Componentes responsivos com breakpoints
RNF.16	Exportação de Logs	routes/api.ts	GET /logs — retorna JSON convertível em CSV/PDF

RNF.17	Localização Regional	Frontend + Backend	Datas em PT-PT, moeda EUR, fuso horário de Lisboa
RNF.18	Facilidade de Aprendizagem	Frontend React	UX simplificada, fluxos de poucos passos
RNF.19	Standards de Segurança	routes/api.ts + AWS	HTTPS via AWS (TLS 1.2), JWT com expiresIn: '8h'

Tabela 22: Rastreabilidade dos requisitos não funcionais.

6.3.3. Requisitos não Atendidos

Importa notar que os cinco requisitos assinalados como parciais ou não implementados não decorrem de falhas de conceção arquitetural, mas de limitações inerentes ao âmbito de um protótipo académico com prazo fixo. O RF.11 (*Peak Pricing*) foi arquitetado e o ponto de integração está identificado em `gerarFaturaFinal()`, mas a lógica de preços dinâmicos por época exigiria uma tabela de tarifas configurável que ultrapassava o âmbito definido. O RF.17 (OCR de Boletins) depende de um serviço externo de reconhecimento ótico, o pipeline de upload para S3 está completo e funcional, ficando a interpretação automática do documento como evolução natural para uma versão de produção. A Lista de Espera (RF.13) e as Notificações *Push* ao tutor (RF.39) foram deliberadamente pospostas em favor da solidez do fluxo transacional principal: reserva, *check-in*, *check-out* e faturação — que representa o núcleo de valor do sistema para a cliente. O modo *offline* (RNF.11) foi previsto na arquitetura *frontend* mas a sua implementação via *Service Workers* ficou condicionada pela estabilização das restantes funcionalidades. Por último, o Agendamento de Serviços Extra (RF.29) não foi completamente realizado, apenas temos um agendamento inicial onde podemos adicionar os serviços de forma basicamente limitados a 2 vezes por dia nos passeios e 1 vez por dia para o resto.

ID	Nome	Estado
RF.11	Peak Pricing automático	Parcial — valorCalculado = 150.00 hardcoded em <code>gerarFaturaFinal()</code>
RF.17	OCR de Boletins	Parcial — ficheiro é guardado no S3 mas leitura OCR não está implementada

RF.13 / RF.08	Lista de Espera com Notificação	Não implementado — só bloqueia reserva, não gere fila
RF.29	Agendamento de Serviços Extra	Parcial – Apenas podemos adicionar no início da reserva.
RF.39	Notificações Push ao tutor	Parcial — email enviado apenas no 2FA, não para eventos da estadia
RNF.11	Modo Offline	Previsto — mencionado na arquitetura, implementação no frontend pendente

Tabela 23: Requisitos não respondidos.

6.4. Cobertura de Código e Análise de Qualidade

O culminar da estratégia de testes foi avaliado de forma quantitativa através da ferramenta c8, gerando um relatório de cobertura de código (*Test Coverage*) que sustenta as métricas do modelo de qualidade de software ISO/IEC 25010:

- Adequação Funcional e Fiabilidade: O sistema demonstrou tolerância a falhas, bloqueando comportamentos inválidos com a correta devolução de códigos HTTP (400 *Bad Request*, 401/403 *Unauthorized*). Todas as regras críticas do domínio (hotelaria e clínica) passaram nas asserções dos testes com sucesso.
- Manutenibilidade (*Testability*): O relatório de cobertura revelou métricas de excelência nas componentes vitais do sistema. A totalidade das dezenas de testes automatizados passou sem erros, com camadas nevrálgicas como a GestFaturacaoFacade a atingirem 100% de cobertura, a GestHospedesFacade a registar 98.9%, e a GestOperacoesFacade 86.1%. A cobertura global atingiu 81.03% de *statements* e 81.03% de linhas. As camadas de negócio (*core*) destacam-se com 95.83%, com GestClinicaFacade, GestFaturacaoFacade, GestHospedesFacade e GestOperacoesFacade a atingirem 100%. A camada de rotas (*api.ts*) ficou em 62.8%, reflexo de rotas de gestão interna com menor criticidade de negócio.

Características	Avaliação	Evidência
Segurança	RBAC via JWT, 2FA, bcrypt, HTTPS/TLS 1.2	middleware/auth.ts, TwoFactorService.ts
Fiabilidade	162/162 testes passados, 0 falhas	async/await, ReservaDAO

Desempenho	Operações assíncronas, queries indexadas	async/await, ReservaDAO
Manutenibilidade	Cobertura global 81%, core 95.83%	Relatório c8
Usabilidade	Interface Mobile-First, 14 testes E2E sem falhas	Playwright (Chromium + Firefox)
Adequação Funcional	38/43 requisitos implementados	Matriz de Rastreabilidade
Compatibilidade	API REST testada com múltiplos perfis	Supertest, auth.test.ts

Tabela 24: Cobertura e Testes.

6.5. Prompts e geração dos resultados

6.5.1. Planeamento da Fase de Testes e Validação

6.5.1.1. Prompt Inicial

O prompt submetido inicialmente visava obter um esboço metodológico das tarefas necessárias para a etapa de qualidade. A equipa aplicou a boa prática de travar a IA de gerar código de imediato, exigindo primeiro um plano estruturado para validação humana:

"Olha se nós quiséssemos cumprir isto: "Realização de testes automatizados, unitários, de integração e de sistema, geração de testes com LLM (e.g. test cases, edge cases e testes baseados em requisitos), verificação da satisfação dos SRS (Software Requirements Specification), e realização de testes de aceitação, com análise de qualidade, cobertura de código e métricas de qualidade (ISO/IEC 25010)." Neste projeto o que teríamos de fazer? Eu quero que ao inves de pegar ja na massa quero que escrevas no .md tudo o que temos de fazer nesta fase para eu poder mudar o que acho que ta mal para tu leres denovo e entenderes exatamente aquilo que eu quero. Por favor faz um esboço de como estruturar este capítulo."

6.5.1.2. Avaliação Crítica e Refinamento do Resultado

Avaliação Crítica do Resultado	Refinamento Aplicado / Ação Tomada
Descontextualização Tecnológica: O modelo apresentou um plano de testes puramente teórico e genérico. Como o prompt inicial não especificou a stack do projeto, a IA não sugeriu ferramentas práticas adequadas ao ecossistema desenvolvido.	<i>Injeção de Contexto Técnico: O prompt foi reescrito para incluir a stack tecnológica do projeto (Node.js, Prisma, PostgreSQL, React) e indicar as ferramentas preferenciais (Jest, Supertest, Playwright, c8), obrigando a IA a planejar tarefas específicas para esta infraestrutura.</i>
Abordagem Teórica às Métricas ISO: A resposta focou-se em definir de forma exaustiva o que é a norma ISO/IEC 25010 (parecendo um manual), em vez de explicar como medir e aplicar essas características na prática.	<i>Foco Operacional e Prático: Instruiu-se o modelo a abandonar as definições teóricas e a associar cada eixo da norma (Adequação Funcional, Usabilidade, Performance) a métricas extraíveis da aplicação.</i>
Linguagem e Tom Inadequados: Devido à formulação excessivamente coloquial do prompt inicial ("pegar ja na massa", "ta mal"), o modelo respondeu com um plano desestruturado e com um tom pouco adequado para um relatório de engenharia.	<i>Adoção de Persona: Introduziu-se a persona de "Engenheiro de Software de QA" e exigiu-se a formatação do output como uma checklist formal e técnica, pronta a ser distribuída pelos membros da equipa.</i>

Tabela 25: Prompts da Fase de Testes.

6.5.1.3. Refinamento do Resultado (Novo Prompt)

Após a análise crítica detalhada na tabela anterior, a equipa submeteu um prompt refinado, garantindo assim que a IA gerasse um planeamento acionável e rigoroso para a Etapa 4:

"Atua como um Engenheiro de Software Sênior (QA). Vamos iniciar a validação do nosso sistema de gestão do hotel canino (Node.js, Prisma, React). O objetivo é criar um plano de ação em Markdown (.md) para implementar testes unitários e de integração (Jest/Supertest), testes E2E (Playwright) e avaliar a qualidade via norma ISO/IEC 25010 (usando relatórios de cobertura da ferramenta c8). Ainda não geres código de testes. Cria apenas uma checklist técnica, estruturada e detalhada das tarefas que a equipa tem de executar para cumprir estes objetivos de forma rigorosa. Mantém um tom académico e profissional."

A Etapa 4 foi a mais estável em termos de execução. Os testes unitários sofreram um atraso de arranque de um dia mas foram concluídos na data prevista. Os testes de integração prolongaram-se um dia, assim como os testes E2E com Playwright e os testes beta de usabilidade, que exigiram mais iterações do que o previsto para cobrir os cenários de erro identificados. A verificação do cumprimento dos requisitos SRS, a cargo do Gonçalo Sá, foi concluída dentro do prazo. A análise de cobertura de código e qualidade começou um dia mais tarde mas recuperou, sendo concluída a 11/05 conforme planeado.

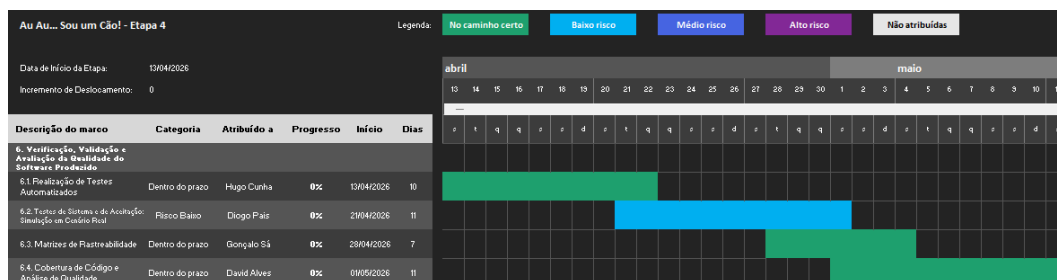


Figura 67: Diagrama de Gantt final – Capítulo 6.

7. Conclusões e Trabalho Futuro

O desenvolvimento da plataforma *Au Au... Sou um Cão!* revelou-se um desafio de engenharia de software profundamente estimulante e, ao mesmo tempo, humilhante na melhor das aceções: um projeto que começou com a ambição de digitalizar um hotel de animais e acabou por ser, na prática, um exercício exigente de transposição de um negócio físico com regras estritas para um ecossistema digital coerente, multi-perfil e auditável. Mais do que escrever código, foi necessário compreender um domínio, modelá-lo com rigor e tomar decisões de engenharia com consequências reais ao longo de todo o ciclo de desenvolvimento.

7.1. Aspetos Positivos e Valor Entregue

O resultado mais sólido do projeto é, sem dúvida, a coerência entre a arquitetura definida e o sistema que foi efetivamente construído. A decisão de adotar uma **Arquitetura Hexagonal com Facades** como camada de orquestração da lógica de negócio não foi apenas uma escolha de design elegante, foi uma decisão com consequências práticas verificáveis. A separação clara entre rotas HTTP, lógica de negócio e acesso a dados através de DAOs permitiu que a camada central do sistema fosse testada de forma completamente isolada da infraestrutura. Os números falam por si: **162 testes automatizados a passar**, com a camada de negócio *core* a atingir **95,83% de cobertura** e quatro das cinco Facades a alcançar os **100% individualmente**.

Esta coerência entre o que foi desenhado nos diagramas UML e o que foi codificado em TypeScript é rara em projetos académicos, onde a pressão do tempo frequentemente leva a desvios tácitos da arquitetura planeada. O facto de o Diagrama de Componentes, o Diagrama de Classes e os Diagramas de Sequência terem funcionado como documentos vivos, consultados durante a implementação e não apenas produzidos para entrega, é um indicador de maturidade de processo.

A engenharia de requisitos, conduzida com base em entrevistas diretas à stakeholder Diana Silva e a utilizadores reais do sistema, produziu um conjunto de requisitos funcionais e não funcionais que se manteve estável ao longo do projeto. A identificação precoce de restrições críticas, como a gestão de compatibilidade de reatividade entre animais para alocação de boxes, os protocolos de quarentena e as regras de cancelamento contratuais, evitou retrabalho tardio e validou a importância do processo de elicitação formal. A utilização de *user stories* por perfil de utilizador e a derivação de Casos de Uso a partir das entrevistas criou uma cadeia de rastreabilidade

entre os requisitos e a implementação que foi fundamental para a fase de testes de aceitação.

Do ponto de vista da integração tecnológica, a orquestração entre o *frontend* em React, o *backend* Node.js com TypeScript, a infraestrutura AWS (RDS, S3, Elastic Beanstalk) e o Prisma ORM como camada de abstração relacional demonstrou a capacidade da equipa em trabalhar com uma *stack* de padrão industrial dentro das restrições de um projeto académico. A implementação de funcionalidades como os Cron Jobs noturnos para auditoria de segurança dos animais, o sistema de RBAC com JWT e o S3StorageAdapter para o Diário de Bordo são exemplos de como a ambição técnica não cedeu à facilidade.

7.2. Aspetos Negativos e Desafios

O principal obstáculo técnico do projeto foi a **gestão da complexidade transacional do domínio**. O fluxo central de reservas, que envolve validação de datas, alocação automática de box por reatividade, geração dinâmica de serviços diários, check-in com aceitação legal de termos, ajuste de valor no check-out e emissão de fatura, revelou-se significativamente mais interdependente do que o antecipado na fase de análise. Esta complexidade não estava escondida: estava explícita nas entrevistas com a stakeholder. O que falhou foi a capacidade de a traduzir, na fase de engenharia de requisitos, em Casos de Uso suficientemente detalhados para que a sua interdependência ficasse visível antes de chegar à implementação.

Esta lacuna aponta para uma limitação metodológica real: os Diagramas de Sequência, embora corretos, modelaram fluxos individuais sem capturar adequadamente as dependências transversais entre subsistemas. Um exercício de modelação de cenários de *edge case*, como a sobreposição de reservas com animais em quarentena, ou o cancelamento parcial de uma estadia com serviços já faturados, teria antecipado grande parte da complexidade que só foi descoberta na fase de testes.

A qualidade dos testes de integração, embora positiva globalmente, foi também um espelho desta complexidade. A necessidade de construir *fixtures* de dados massivas e interdependentes para simular cenários realistas revelou que o modelo de dados, apesar de correto do ponto de vista relacional, gerou acoplamentos implícitos que tornaram os testes frágeis em certas condições. Algumas das falhas encontradas, como violações de chaves estrangeiras no Prisma em cenários de eliminação em cascata, resultaram de decisões de design de base de dados que não foram suficientemente debatidas na fase de arquitetura.

Por fim, a mudança tardia que associou o Diário de Bordo a uma Reserva em vez de a um Animal, embora conceptualmente correta, é um exemplo ilustrativo de como decisões de modelação de domínio que parecem locais têm, na prática, um raio de impacto alargado que se estende ao esquema da base de dados, à API, ao frontend e à infraestrutura de deployment. Esta alteração, feita já numa fase avançada do projeto, consumiu um tempo desproporcionado ao que a sua aparente simplicidade sugeria.

7.3. Trabalho Futuro

Existem várias direções concretas que uma iteração futura da plataforma deveria endereçar, tanto por imperativo de completude funcional como por maturidade de produto.

A **automação financeira certificada pela Autoridade Tributária** permanece como a funcionalidade de maior impacto operacional por completar. A arquitetura preparou este módulo com isolamento específico, o `SubSistemaFaturacao` e o seu adaptador AT estão desenhados, mas a integração real com a API da AT requer configuração legal e técnica que excede o âmbito de um projeto académico. Numa versão de produção, este seria o primeiro ponto a endereçar.

A **comunicação ativa com tutores** representa outra extensão de elevado valor percebido. O `EmailService` existente é uma base funcional, mas a transição para um sistema de notificações em tempo real, notificando os donos quando o Diário de Bordo é atualizado, quando a medicação é administrada ou quando se aproxima o check-out, transformaria o portal de um sistema de gestão interno num canal de comunicação com o cliente.

Do ponto de vista da engenharia de software, faria sentido investir na **instrumentação e observabilidade** da plataforma em produção: logging estruturado, métricas de performance por endpoint e alertas automáticos para falhas de infraestrutura. Numa operação onde a falha do sistema durante um pico de lotação tem consequências diretas para o bem-estar animal, a monitorização proativa não é opcional.

Por fim, a **implementação de uma estratégia de migrações versionadas com Prisma Migrate**, em substituição do `db push` utilizado durante o desenvolvimento, é uma necessidade técnica imediata para qualquer ambiente de produção real, garantindo que evoluções do esquema são rastreáveis, reversíveis e aplicadas de forma controlada.

7.4. O Paradigma do Desenvolvimento Assistido por LLMs - Uma Reflexão

Não seria honesto encerrar este relatório sem uma reflexão genuína sobre aquilo que foi, porventura, o elemento mais diferenciador deste projeto: a utilização sistemática de um Large Language Model, especificamente o Claude da Anthropic, como colaborador transversal ao longo de todas as fases do ciclo de desenvolvimento.

É tentador categorizar esta experiência numa de duas narrativas simplistas: a do entusiasmo acrítico ("a IA fez tudo") ou a do ceticismo defensivo ("a IA não acrescenta nada que um bom programador já não soubesse"). A realidade foi mais interessante, mais ambígua e, por isso, mais valiosa como lição.

7.4.1. O que a IA realmente foi

O LLM não foi um substituto do pensamento mas um **acelerador do pensamento estruturado**. Na fase de análise, auxiliou a transformar notas de entrevistas em requisitos formais, a identificar *stakeholders* implícitos que a equipa não tinha considerado e a gerar rascunhos de *user stories* que serviram como ponto de partida para refinamento. Na fase de arquitetura, funcionou como um interlocutor, o tipo de colaborador que questiona as premissas de uma decisão antes de a validar, que sugere trade-offs entre padrões de design e que identifica inconsistências no modelo de dados antes de estas serem codificadas. Na implementação, eliminou o atrito do *boilerplate*: configurações de infraestrutura AWS, esquemas Prisma, estruturas de middleware, tarefas necessárias mas mecânicas, foram geradas em segundos, libertando atenção cognitiva para os problemas que realmente importavam.

7.4.2. O que a IA não conseguiu ser

Mas o LLM falhou sistematicamente numa dimensão: a do **contexto acumulado e da memória sistémica**. O modelo pode gerar código tecnicamente correto que é contextualmente errado: código que compila, que os testes unitários validam, mas que viola invariantes do sistema que só existem na cabeça da equipa. Os erros mais custosos do projeto, como as violações de chaves estrangeiras em cenários de eliminação em cascata, mocks que não refletiam as restrições reais da base de dados, lógica de alocação de boxes que passava nos testes mas falhava em cenários de *edge case* com animais reativos, foram precisamente aqueles em que o LLM gerou respostas plausíveis para perguntas mal formuladas. O problema não era a qualidade da resposta: era a qualidade da pergunta.

7.4.3. A questão que fica

Há uma pergunta que vale a pena colocar sem resposta definitiva: **até que ponto as decisões de arquitetura deste projeto são da equipa?**

A resposta honesta é: em grande medida, sim. O LLM sugeriu padrões (o Facade, o DAO, a Arquitetura Hexagonal) mas foi a equipa que avaliou, questionou a sua necessidade, adaptou ao domínio específico de um hotel de animais e defendeu perante a realidade da implementação. Um modelo de linguagem não sabe o que é gerir 40 animais em pico de Verão, não viveu as sessões de validação de interface ou os debates sobre o modelo de cancelamento. O que o LLM trouxe foi um vocabulário técnico mais rico e uma velocidade de exploração de alternativas que, sem ele, teria sido incomportável no espaço de um semestre.

O que este projeto ensinou é que **o desenvolvimento assistido por IA é, acima de tudo, uma disciplina de juízo crítico**. A capacidade de distinguir entre uma resposta plausível e uma resposta correta, entre um padrão genérico e a solução certa para um problema específico, entre a velocidade de geração e a qualidade do resultado, essa capacidade não é delegável. É, e continuará a ser, essencialmente humana.

Referências Bibliográficas

- [1] A. Cockburn, "Hexagonal Architecture", *Alistair.cockburn.us*, 2005. [Online]. Disponível em: <https://alistair.cockburn.us/hexagonal-architecture/>. [Acedido em: 26-Mai-2026].
- [2] E. Gamma, R. Helm, R. Johnson, e J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, EUA: Addison-Wesley, 1994. (Nota: Referência clássica essencial para justificar o uso do padrão Facade).
- [3] Prisma Data Inc., "Prisma Documentation: Next-generation Node.js and TypeScript ORM", *Prisma.io*, 2026. [Online]. Disponível em: <https://www.prisma.io/docs/>. [Acedido em: 26-Mai-2026].
- [4] Meta Platforms, Inc., "React: The library for web and native user interfaces", *React.dev*, 2026. [Online]. Disponível em: <https://react.dev/>. [Acedido em: 26-Mai-2026].
- [5] Microsoft Corporation, "TypeScript: Typed JavaScript at Any Scale", *Typescriptlang.org*, 2026. [Online]. Disponível em: <https://www.typescriptlang.org/>. [Acedido em: 26-Mai-2026].
- [6] Amazon Web Services, "Amazon Simple Storage Service (Amazon S3) Documentation", *AWS Documentation*, 2026. [Online]. Disponível em: <https://docs.aws.amazon.com/s3/>. [Acedido em: 26-Mai-2026].
- [7] Anthropic, "Claude: AI for broad cognitive tasks", *Anthropic.com*, 2026. [Online]. Disponível em: <https://www.anthropic.com/claude>. [Acedido em: 26-Mai-2026]. (Nota: Referência para justificar a secção de Desenvolvimento Assistido por LLM).
- [8] D. Crockford, "The application/json Media Type for JavaScript Object Notation (JSON)", *RFC 4627*, 2006. [Online]. Disponível em: <https://datatracker.ietf.org/doc/html/rfc4627>. [Acedido em: 26-Mai-2026].
- [9] OpenJS Foundation, "Node.js Documentation", *Nodejs.org*, 2026. [Online]. Disponível em: <https://nodejs.org/en/docs/>. [Acedido em: 26-Mai-2026].

Lista de Siglas e Acrónimos

BD	Base de Dados
DW	Data Warehouse
OLTP	<i>On-Line Analytical Processing</i>
LLM	Large Language Models
EEUM	Escola de Engenharia da Universidade do Minho
SDLC	Software Development Life Cycle
AT	Autoridade Tributária
IAG	Inteligência Artificial Generativa
API	Application Programming Interface
RGPD	Regulamento Geral da Proteção de Dados
RF	Requisito Funcional
RNF	Requisito Não Funcional
SGBD	Sistema de Gestão de Base de Dados
SRS	Software Requirements Specification
2FA	2 Factor Authentication
IA	Inteligência Artificial
JWT	JSon Web Tokens
ORM	Object Relational Mapping
JSON	JavaScript Object Notation
IAM	Identity and Access Management
AWS	Amazon Web Services
VPC	Virtual Private Connection
SPA	Single Page Application
DOM	Document Object Model
DAO	Data Access Object
OOD	Object Oriented Design
UI	Design de Interfaces
UX	Experiência do Utilizador
RBAC	Role Based Access Control
SMTP	Simple Mail Transfer Protocol
URL	Uniform Resource Locator
PR	Pull Request
E2E	End to End
HTTP	Hypertext Transfer Protocol

Apêndices

I. Apêndice I - Entrevista Diana

Bloco A:

O Insight Inicial: Recuando ao momento da criação, qual foi o "clique" para transformar a dificuldade de emprego na pandemia num hotel canino? Foi uma necessidade pessoal com os teus próprios cães ou algo que observaste no mercado de Braga?

Missão e Valores: Quando abriste as portas, qual era o objetivo principal? Era apenas um negócio de sobrevivência ou já existia a ambição de criar um padrão de cuidados diferenciado pela tua formação em Veterinária?

Evolução do Conceito: O hotel começou com a estrutura que tem hoje ou foi crescendo organicamente? Como é que o teu percurso académico em Medicina Veterinária moldou os serviços que decidiste oferecer desde o primeiro dia?

Superação de Expectativas: Alguma vez imaginaste, no início, que passarias de 20 para 120 animais em épocas altas? Como é que essa escala alterou os teus objetivos iniciais para o negócio?

Bloco B:

Gestão de Inventário e Saúde: Como controlas atualmente o stock de medicamentos e rações específicas para cães com patologias? O sistema deve emitir alertas de stock baixo ou de validade de medicamentos?

Rastreabilidade de Incidentes: Se um cão se aleija ou apresenta sintomas durante a estadia, como é feito o registo hoje? Precisas que o software gere um "histórico de incidentes" que possa ser exportado para o dono ou para a seguradora?

Regras de Alocação (Quartos): Tens regras de "vizinhança" (ex: não colocar cães reativos ao lado de outros, separar por porte ou idade)? O sistema deve sugerir o quarto ideal com base no perfil do animal?

Faturação Complexa: Como lidas com cancelamentos de última hora ou extensões de estadia não planeadas? O sistema deve calcular automaticamente multas ou descontos por períodos longos?

Comunicação com o Dono: Atualmente perdes muito tempo a enviar fotos/updates por WhatsApp? Gostarias que o sistema tivesse uma funcionalidade de "Diário de Bordo" onde o staff faz upload de uma foto e o dono recebe uma notificação?

Segurança e Permissões: O que é que o Staff pode ver/editar em comparação com a Recepção? (Ex: O staff deve ver dados financeiros ou apenas a ficha clínica e de alimentação?)

II. Apêndice II - Respostas Diana

Olá! Sou a Diana Gonçalves Silva, fundadora do "Au Au... Sou um Cão!". Como podem imaginar, entre o curso de Medicina Veterinária na UMinho e a gestão de um hotel que cresceu muito além do planeado, o meu tempo é escasso e o meu pragmatismo é uma questão de sobrevivência. Estou aqui para vos dar a visão real de quem está no terreno, a tratar de patudos enquanto tenta não se afogar em papéis e mensagens de WhatsApp.

Bloco A: Génese, Motivação e Objetivos (O Passado e a Visão)

1. O Insight Inicial: O "Clique"

O clique foi uma mistura de necessidade pessoal e oportunidade de mercado em Braga durante a pandemia (2021). Eu já tinha os meus cães e sentia na pele a dificuldade de encontrar um sítio que eu, como futura veterinária, confiasse plenamente. Vi que as pessoas em Braga estavam a adotar mais animais no confinamento, mas não havia estruturas com uma visão clínica e etológica séria. Decidi que, se não encontrava o padrão de excelência que queria, eu própria o criaria com o apoio do meu namorado.

2. Missão e Valores: Sobrevivência ou Padrão Diferenciado

Embora tenha começado como um projeto de vida, a minha formação em Veterinária nunca me permitiria que fosse "apenas sobrevivência". A missão sempre foi a excelência baseada na ciência. Queria um espaço onde o maneio sanitário e o controlo clínico fossem a base. Os donos deixam cá os animais precisamente porque sabem que eu deteto uma otite, uma claudicação subtil ou um sintoma de gastroenterite muito antes de um leigo.

3. Evolução do Conceito e Moldura Académica

Começámos de forma artesanal, eu e o meu namorado a fazer tudo. Hoje temos uma estrutura de 20 a 30 quartos individuais, mas o percurso académico moldou tudo desde o dia um: desde a escolha de desinfetantes não tóxicos e protocolos de isolamento para tosses suspeitas, até à criação de fichas clínicas rigorosas com registo de peso, apetite e dejecções. Não somos um "depósito de cães"; somos um centro de bem-estar com supervisão técnica.

4. Superação de Expectativas: Dos 20 aos 120

Honestamente? É assustador e exaustivo. Nunca imaginei que em épocas altas como agosto ou o Natal passaríamos de 30 para 120 animais (recorrendo a ocupação dupla de cães compatíveis e staff extra). Essa escala alterou os meus objetivos: passei de focar apenas no cuidado direto para focar na fiabilidade do processo. Com 120 animais, o erro humano é o meu maior inimigo. Preciso de

tecnologia porque a minha formação clínica não serve de nada se eu não souber, com 100% de certeza, qual cão tomou qual comprimido.

Bloco B: Engenharia de Requisitos (Operacional e Técnico)

1. Gestão de Inventário e Saúde (Medicação e Alimentação)

Atualmente, o controlo é feito em quadros brancos e cadernos, o que é um perigo.

- * Requisito de Medicação: O sistema tem de gerir stock de medicamentos críticos (ex: insulina para diabéticos, fenobarbital para epiléticos) e emitir alertas de stock baixo e, crucialmente, de validade. Preciso de um log de administração: quem deu, que dose, a que horas e se houve reação.
- * Requisito de Alimentação: Tenho cães com dietas renais, hipoalergénicas ou pancreatite. O sistema deve emitir etiquetas de refeição para evitar que o staff troque as rações por engano.

2. Rastreabilidade de Incidentes

Hoje, se um cão se arranha a brincar ou apresenta diarreia, anotamos num caderno. Isso é ineficiente e juridicamente perigoso.

- * Requisito: Preciso que o software gere um Histórico de Incidentes exportável em PDF para o dono ou seguradora, com fotos, descrição clínica (ex: "edema localizado"), tratamento aplicado e timestamp. Se eu tiver de encaminhar um cão para uma consulta externa de urgência, quero carregar num botão e enviar todo o relatório clínico para o colega veterinário.

3. Regras de Alocação de Quartos (Vizinhança)

Gerir a "vizinhança" de cabeça é um puzzle impossível em picos de lotação.

- * As minhas regras: Cães reativos/agressivos isolados; cães grandes não podem estar ao lado de cães minúsculos (o ruído e presença assustam); cães seniores precisam de zonas calmas e cães com patologias transmissíveis em quarentena.
- * Requisito: O sistema deve ter um algoritmo de sugestão de quartos baseado em etiquetas (Reativo, Idoso, Porte, Fugitivo) e bloquear alocações conflituosas.

4. Faturação Complexa (Cancelamentos e Extensões)

As contas ao fim do mês são um caos entre estadias prolongadas, banhos de última hora e extensões não planeadas (quando o voo do dono atrasa).

- * Requisito: Automação total de políticas de cancelamento (ex: retenção de 30% em cancelamentos <48h), descontos progressivos por longas estadias (ex: 10% após o 15º dia) e faturação automática de serviços extra (banhos, tosquiadas, administração de medicação complexa). O check-out tem de ser rápido, sem contas de somar à mão com o cliente à espera.

5. Comunicação com o Dono (Diário de Bordo)

O WhatsApp está a matar a produtividade da equipa. O meu namorado passa horas a enviar fotos individualmente.

- * Requisito: Um "Diário de Bordo" integrado. O staff tira uma foto na app, escreve "Comeu bem e brincou com o Rex" e o dono recebe uma notificação automática. Sem conversas paralelas infinitas que nos tiram o foco do cuidado animal.

6. Segurança e Permissões (Perfis de Acesso)

Pragmatismo aqui: segregação de dados é essencial.

- * Staff (Cuidadores): Devem ver apenas a ficha clínica, restrições de alimentação, horários de medicação e tarefas do dia. Não devem ver dados financeiros ou faturas.
- * Receção/Gestão: Acesso total a reservas, contactos dos donos e faturação.
- * Veterinário (Eu): Acesso a auditoria e edição de prescrições clínicas.

Resumo da Urgência:

O "Au Au... Sou um Cão!" precisa de funcionar com o rigor de uma clínica hospitalar e a fluidez de um hotel moderno. Com o meu namorado e com os funcionários, não conseguimos escalar mais sem um sistema que elimine o erro humano e nos devolva o tempo para o que realmente importa: o bem-estar dos animais. podemos avançar com o desenvolvimento?

III. Apêndice III - Entrevista Cliente

Bloco A: Motivação e Critérios de Escolha

O Fator Confiança: O que o levou a escolher este hotel em específico? Foi a formação em veterinária da Diana , a localização em Braga ou recomendações de outros clientes?

Expectativas Iniciais: Quando deixa o seu animal, qual é a sua maior preocupação? (Ex: alimentação, socialização, administração de medicação ou medo que o animal se sinta sozinho). Experiência em Época Alta: Já utilizou o hotel durante picos de ocupação (como o Verão ou Natal)? Notou alguma diferença na qualidade do atendimento ou na facilidade de conseguir reserva?

Bloco B: Engenharia de Requisitos (Visão do Utilizador)

Processo de Reserva: Como prefere fazer as suas reservas atualmente? Gostaria de ter uma aplicação ou portal onde pudesse ver a disponibilidade em tempo real e reservar sem ter de esperar por uma resposta manual?

Transparência e Comunicação: Que tipo de informação gostaria de receber durante a estadia? Fotos diárias, um relatório de humor/comportamento ou apenas um aviso de que "está tudo bem"? Check-in Digital: Acharia útil poder preencher a ficha clínica (vacinas, alergias, hábitos)

online antes de chegar ao hotel para tornar a entrega do animal mais rápida?

Pagamentos e Faturação: Como prefere receber as faturas e efetuar pagamentos? Gostaria de ter um histórico de faturas e gastos acumulados acessível no seu perfil?

Serviços Extra: Durante a reserva, gostaria de poder seleccionar "extras" de forma visual (ex: banho no último dia, passeio extra, consulta veterinária de rotina)?

IV. Apêndice IV - Respostas Cliente

Como cliente fiel e dono de um cão com necessidades médicas específicas (que exige medicação rigorosa e monitorização de ansiedade), apresento as minhas respostas ao guião de entrevista. O meu objetivo é garantir que o hotel "Au Au... Sou um Cão!" consiga manter a excelência técnica da Diana, evoluindo para um modelo que nos dê mais autonomia e segurança, especialmente nos períodos de maior afluência.

Bloco A: Motivação e Critérios de Escolha

1. O Fator Confiança: O que o levou a escolher este hotel em específico? O fator decisivo foi a formação em Medicina Veterinária da Diana. Em Braga, existem muitas creches, mas poucas oferecem o rigor clínico necessário para um animal que toma medicação (como ansiolíticos ou protetores gástricos). Saber que a proprietária entende de farmacologia e comportamento animal dá-me uma paz de espírito que não encontro noutro lado. As recomendações excelentes de outros clientes e a localização prática em Braga apenas confirmaram que este era o local certo.
2. Expectativas Iniciais: Quando deixa o seu animal, qual é a sua maior preocupação? A minha maior ansiedade reside na administração correta da medicação (doses e horários fixos) e na monitorização de sintomas. Tenho sempre receio que, com o volume de trabalho, algum detalhe do plano de saúde possa passar ao lado ou que a ansiedade de separação do meu cão não seja detetada a tempo. Preciso de ter a certeza de que ele está a comer bem e a socializar de forma segura.
3. Experiência em Época Alta: Notou alguma diferença no atendimento em picos de ocupação? Sim, e é aqui que reside o maior problema. Em épocas como o Verão ou o

Natal, com o hotel a ultrapassar os 100 animais, é quase impossível falar com a Diana ao telefone. Sinto que, ao ligar para saber como ele está, estou a tirar tempo precioso que a equipa deveria usar a cuidar dos animais. As mensagens de WhatsApp perdem-se e o feedback limita-se muitas vezes a um "está tudo bem" no final da estadia. A qualidade do cuidado mantém-se, mas a comunicação torna-se opaca e gera ansiedade no dono.

Bloco B: Engenharia de Requisitos (Visão do Utilizador)

1. Processo de Reserva: Preferência por autonomia tecnológica Atualmente, o processo manual por chamada/mensagem é lento e incerto. Gostaria de ter um Portal ou App com disponibilidade em tempo real. Seria ideal poder ver um calendário visual, selecionar as datas e receber confirmação imediata. Isso dá autonomia ao cliente e liberta a Diana de 80% das tarefas administrativas,

permitindo-lhe focar-se na gestão clínica.2. Transparência e Comunicação:

Informação durante a estadia Proponho um

sistema de notificações diárias via App estruturado em três níveis:

- Crítico: Confirmação de que a medicação foi administrada (ex: "Bobi - Medicação das 08:00 ✓").
- Importante: 1 ou 2 fotos diárias e um pequeno "log" de saúde (apetite, nível de energia, socialização).
- Urgente: Alertas imediatos em caso de incidentes (vómitos, feridas ou recusa alimentar).

3. Check-in Digital: Ficha clínica online antecipada Acho fundamental poder preencher a ficha clínica online (vacinas, alergias, doses de medicação e contactos de emergência) antes de chegar ao hotel. No dia da entrega, o cão está agitado e nós temos pressa; ter tudo já no sistema com upload dos comprovantes em PDF/JPG tornaria o check-in num processo de 2 minutos. É mais profissional, seguro e elimina o erro humano da escrita manual.

4. Pagamentos e Faturação: Gestão de histórico e transparência Gostaria de ter um perfil de cliente com histórico de faturas e gastos acumulados. O pagamento por MB Way ou cartão integrado na App seria o ideal. Atualmente, andar a pedir faturas por e-mail em época alta é pedir para elas ficarem esquecidas. Ver o preço exato das noites (standard vs. época alta) de forma transparente evitaria qualquer surpresa no checkout.

5. Serviços Extra: Seleção visual de "extras" Sim, adoraria um "menu visual" (tipo carrinho de compras) no momento da reserva. Poder seleccionar visualmente o banho para o último dia, um passeio extra na natureza ou uma consulta veterinária de rotina com a Diana, vendo os preços e fotos de cada serviço. Isto permite ao dono decidir calmamente e garante que a equipa sabe exatamente o que foi contratado sem trocas de última hora.

V. Apêndice V - Entrevista Staff

Olá! Desculpa lá a cara de sono, mas as últimas semanas aqui no "Au Au... Sou um Cão!" têm sido uma correria. Entre o Verão e os feriados, passámos daqueles 30 quartos calmos para ter mais de 80 bichos espalhados por todo o lado. É uma alegria, claro, mas às vezes o meu cérebro já só processa latidos. Senta-te aí, que eu conto-te como isto anda enquanto bebo este café.

1. Podes descrever como é a tua rotina típica desde o momento em que entras no hotel até ao fim do turno? Entro por volta das 07:30 e a primeira coisa é fazer uma ronda rápida para ver se está tudo bem. Depois vou a correr para o quadro branco ver as notas da noite, mas o marcador está sempre gasto e às vezes nem percebo o que lá escreveram. A manhã é uma maratona: servir refeições (rezando para que as dietas especiais estejam bem anotadas), limpar boxes e fazer passeios. O problema é que passamos o dia a saltar entre os baldes de limpeza e o telemóvel para tirar fotos aos cães. No fim do turno, estou de rastos a tentar passar tudo o que fiz para fichas de papel sem me esquecer de nada.
2. O que é que mais gostas aqui e o que é que te dá mais "dores de cabeça"? O que eu adoro é o contacto com eles; ver um cão tímido ganhar confiança é gratificante. Agora, as dores de cabeça? É a desorganização que o crescimento trouxe. Tentar gerir 80 animais com os mesmos métodos de quando tínhamos 20 é de loucos. O barulho dos cães ansiosos na receção enquanto andamos à procura de fichas de papel perdidas em dossiers é de deixar qualquer um doido.
3. Como decidem o quarto de cada cão? Já mudaram algum à pressa? Isso é um "puzzle" que costuma estar só na cabeça da Diana. Como ela é a veterinária, sabe quem é reativo ou nervoso. Mas sim, já aconteceu! Às vezes pomos um Yorkshire ao lado de um Pastor Alemão e é um desastre de rosnadelas. Temos de andar a mudar cães de quarto a meio da tarde e, como não há um mapa centralizado, às vezes um colega nem sabe que o cão mudou de sítio.
4. Nas épocas altas, como sabem que quartos estão limpos? É um bocado à base do grito ou de andar a abrir portas. "Ó Tiago, a box 12 já foi higienizada?". Marcamos no quadro "limpo", mas como o marcador falha ou alguém se esquece de apagar o que era de ontem, há dias em que ando a correr os canis todos só para ver qual está pronto. É uma perda de tempo incrível.
5. Qual o passo do check-in que demora mais tempo? Sem dúvida, encontrar e preencher a ficha em papel. Se o dono não vem cá há um ano, a ficha está enterrada num arquivo físico. O cão está ali a ladrar, o dono com pressa para ir de férias, e nós ali a folhear dossiers e a perguntar outra vez os contactos de emergência e as

alergias. Se isso já viesse preenchido de casa, era um alívio.⁶ Como tens a certeza de que um cão já comeu ou tomou a medicação sem

perguntar a ninguém? Sinceramente? Não tenho 100% de certeza. Eu olho para o quadro branco, mas se o marcador estava gasto e o "visto" não se vê, ou se o colega se esqueceu de anotar na correria, fico logo na dúvida. Tenho de andar a ligar aos colegas ou a interromper a Diana. É um stress constante.

7. Já aconteceu o marcador estar gasto ou alguém se esquecer de anotar?

Acontece quase sempre. Quando o marcador falha, escrevemos com o que há — às vezes até a lápis — e depois fica aquela mancha que não sai do quadro. Quando alguém se esquece de anotar, o remédio é ir pelo "feeling" ou perguntar a toda a gente. É uma falta de segurança que me tira o sono, a sério.

8. Se um animal tossir ou parecer apático, qual o procedimento? Como a Diana sabe? Isso é complicado. Se ela estiver aqui, aviso logo. Mas se ela estiver numa urgência clínica, tenho de lhe mandar uma mensagem por WhatsApp com um vídeo ou foto. O problema é que essas mensagens perdem-se no meio das fotos que enviamos aos donos. Às vezes, um registo de um sintoma fica num papel qualquer e ela só vê horas depois.

9. Quanto tempo do dia passas no WhatsApp? Demasiado! Em época alta, perco facilmente 1 a 2 horas por dia nisso. Os donos querem ver os meninos, eu percebo, mas parar de limpar ou de passear para tirar 50 fotos e enviar individualmente... tira-nos o foco do que realmente importa: estar atento ao comportamento deles.

10. Como controlas se um cão já fez o banho ou o passeio que o dono contratou? São listas de papel à parte. No fecho da conta, é sempre uma confusão. "O Bobi fez o banho na terça ou na quarta?". Às vezes o dono diz que pediu e nós não temos a certeza se foi feito porque o papel se extraviou ou não foi assinalado. É uma correria de última hora antes de o cliente pagar.

11. Se pudesses carregar num botão para fazer desaparecer uma tarefa, qual seria? Enviar fotos e vídeos individualmente pelo WhatsApp, sem hesitar. Queria algo onde carregasse a foto uma vez e os donos vissem logo, para eu poder ser apenas cuidador e não secretário de telemóvel na mão.

12. O que te faz sentir mais inseguro(a) no trabalho? O erro humano na medicação crítica. Com 80 animais e registos manuais num quadro que se apaga, o medo de falhar uma dose de insulina ou de um remédio para a epilepsia é o que me deixa mais ansiosa. A nossa memória já não chega para tanto bicho e sinto que, a qualquer momento, algo grave pode falhar por falta de um registo claro.